The Chatman Equation

A Thesis Program

A six-part development of admission algebra, receipt cryptography, planning geometry, and projection at scale.

Sean Chatman

2026

Contents

The Chatman Equation:

A Receipt Calculus for Reality-Addressed Agentic AI

Sean Chatman
Independent Researcher

Genesis — Foundations, Paper 00

Abstract

This paper introduces the Chatman Equation, $\mathcal{A} = \mu(\mathcal{O}^*)$ with $\mathcal{R} = \text{receipt}(\mathcal{A})$, as a receipt calculus for agentic AI systems. The equation separates raw observation from admitted observation: an action is not manufactured from arbitrary model output, logs, claims, or tool responses, but only from an admitted subspace produced by finite, decidable obligations. This boundary is motivated by Rice's theorem: arbitrary semantic truth cannot be decided by a total program, so execution must proceed through a restricted admission language rather than through unbounded interpretation.

The paper formalizes five objects: observation space, admitted space, manufacturing morphism, action/artifact space, and receipt space. It defines admission as a computable retraction, refusal as a first-class value, refusal composition as a denial monoid, and lifecycle governance as a typestate category whose illegal transitions are uninhabited. Manufacture is modeled as a deterministic bounded morphism from admitted observations to artifacts, while receipts are collision-committed projections of execution traces sized for bounded verification.

The enforced form, the Bounded Receipted Chatman Equation, requires four invariants: no actuation from raw or refused observations, bounded deterministic manufacture, total receipt emission, and conformance replay. Under these invariants, the paper proves conservation of consequence: every standing action has an admitted cause and a receipt-chain position, while receiptless actuation is excluded by construction. The result is a formal foundation for reality-addressed agentic systems in which scalable trust is obtained through admission, manufacture, receipt, and replay rather than through semantic omniscience or post hoc explanation.

Contents

Chapter 1

Introduction: One Equation, Five Objects

1.1 The claim in one line

A prior paper in this program [1] advanced a single governing law, the *Chatman equation*

$$\mathcal{A} \;=\; \mu(\mathcal{O}), \tag{1.1}$$

read there as: an action state $\mathcal{A}$ is the deterministic image, under a measurement function μ, of a typed knowledge graph $\mathcal{O}$. That paper demonstrated the law at enterprise scale — knowledge hooks over RDF/SHACL, the KNHK hot-path executor, the `ggen` bounded projection, and Lockchain provenance — and reported operational constants (sub-two-nanosecond rule checks, full coverage of the forty-three workflow patterns of van der Aalst et al. [4], cryptographically receipted runs). It established that deterministic projection of typed knowledge into verifiable action is not a simulation but a deployed reality.

This paper does not restate that result; it *grounds* it. Equation (1.1) silently assumes its input is already typed — already lawful. The engineering that makes $\mathcal{O}$ typed, that decides which raw records may enter μ at all, and that emits the receipts by which a bounded human trusts the output, is exactly the machinery this series formalizes. We therefore refine (1.1) by *factoring* the projection into an admission stage and a manufacturing stage, exposing the admitted sub-space $\mathcal{O}^*$ and the receipt space $\mathcal{R}$ that (1.1) left implicit. The refined identity, which governs the whole series, is

$$\boxed{\mathcal{A} \;=\; \mu(\mathcal{O}^*)} \qquad \text{with} \qquad \mathcal{O}^* = \operatorname{im}(\alpha), \quad \alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\bot\}. \tag{1.2}$$

In words: nothing is manufactured from a raw observation; only from an *admitted* one; and every manufacture leaves a receipt. The five objects of the equation are named once, here, and used unchanged throughout the series.

Definition 1.1 (The five objects)**.** The Chatman equation (1.2) relates five objects.

(i) $\mathcal{O}$, the *observation space*: arbitrary finite records — logs, model outputs, sensor frames, human claims, tool responses — carrying no decidable semantics (Axiom 2.1).

(ii) $\mathcal{O}^* = \mathcal{O}^* \subseteq \mathcal{O}$, the *admitted space*: the decidable sub-collection onto which a finite battery of obligations is computable, together with the distinguished refusal $\bot$ (Definition 3.1).

(iii) $\mu : \mathcal{O}^* \to \mathcal{A}$, the *manufacturing morphism*: a deterministic, bounded, computable map from admitted observations to artifacts (Definition 6.1).

(iv) $\mathcal{A}$, the *artifact/action space*: the actuated outputs, exhausted by $\operatorname{im}\mu$ (Definition 6.1).

(v) $\mathcal{R}$, the *receipt space*: bounded, collision-committed projections of the execution that produced an artifact, sized for a human verifier (Definition 6.2).

Remark (Relation to the prior μ). The measurement function of (1.1) is the composite of this paper's two stages: $\mu_{\text{prior}} = \mu \circ \alpha$, with α the admission of Definition 3.1 and μ the manufacture of Definition 6.1. Nothing in [1] is retracted; the ingress guards H of that paper *are* the obligation battery, and its cryptographic receipts *are* the objects of $\mathcal{R}$. This series simply refuses to leave α and $\mathcal{R}$ implicit.

1.2 Why comprehension is the wrong standard, in one paragraph

The systems this program serves exceed any human's working-memory capacity, a fixed small integer we write κ (canonically $\kappa \approx 4$ following Miller and Cowan [7, 8]). A system whose faithful description needs more than κ chunks cannot be trusted by comprehension. The response of this series is not to shrink the system but to insert, between it and the human, a map of codomain dimension $\leq \kappa$ whose fidelity is guaranteed by mechanism rather than by the reader — the receipt. That is the subject of the keystone paper *The Projection Principle*; here we build the algebraic and type-theoretic floor beneath it.

1.3 What this paper proves

Chapter 2 axiomatizes observation and proves the Rice specialization that forces admission to be a retraction, not a decision; it defines the admission map, refusal, and the denial monoid, and states the totality property of the eight-category refusal taxonomy. Chapter 5 builds the lifecycle typestate category and proves the illegal-transition-is-a-type-error theorem. Chapter 6 defines the manufacturing morphism and the receipt, states the Chatman equation as a factoring, and gives the enforced form BRCE with its conservation corollary. Chapter 7 works a contract-claim law object end to end. Chapter 8 is the map of the series. Throughout, a claim is a definition, a theorem with proof, or a citation; there is no fourth kind of sentence.

Chapter 2

The Computability Boundary: Rice Quarantine

2.1 The observation axiom

We begin with the least structure that makes "lawful action" expressible.

Axiom 2.1 (Observation)**.** There is a set $\mathcal{O}$, the observation space, whose elements are arbitrary finite records. $\mathcal{O}$ carries no decidable semantics: the predicate "does this observation mean what it purports to mean" is not assumed computable. Concretely, an element of $\mathcal{O}$ is any finite byte string a caller may submit; in the `praxis` implementation it is the JSON payload handed to the `law judge` verb, before any obligation has been evaluated.

Axiom 2.1 is not a limitation we impose; it is forced by a classical result, which we specialize to observations.

Theorem 2.1 (Rice, specialized to observations)**.** *Let $\mathcal{U}$ be a universal model of computation and let observations in $\mathcal{O}$ range over finite encodings that may denote arbitrary $\mathcal{U}$-programs (equivalently, the partial functions they compute). Let P be any non-trivial semantic property of those denoted functions — non-trivial meaning some observation has it and some does not, and P depends only on the denoted function, not on the encoding. Then the set $\{o \in \mathcal{O} : P(o)\}$ is undecidable.*

Proof. This is Rice's theorem [2] instantiated at $\mathcal{O}$. Rice's theorem states that every non-trivial property of the partial function computed by a Turing machine is undecidable. Since observations are unrestricted finite encodings, the map $e : \mathcal{O} \to \{\mathcal{U}\text{-programs}\}$ sending an observation to the program it denotes is onto a set closed under the standard constructions, and P factors through the denoted function by hypothesis. Suppose a decider D existed for $\{o : P(o)\}$. Fix observations o_+ with P and o_- without (non-triviality). Given any machine M and input x, build the observation $o_{M,x}$ denoting the program "run $M(x)$; if it halts, behave as $e(o_+)$; else diverge." Then $o_{M,x}$ denotes a function with property P iff $M(x)$ halts, so $D(o_{M,x})$ decides the halting problem, a contradiction. Hence no such D exists. $\square$

Corollary 2.2 (Admission cannot be semantic decision)**.** *There is no algorithm that admits an observation $o \in \mathcal{O}$ by deciding a non-trivial semantic property of what o denotes. Any total,* *terminating admission procedure must therefore decide a* syntactic*, decidable surrogate — it retracts $\mathcal{O}$ onto a decidable sub-language rather than deciding meaning.*

Proof. Immediate from Theorem 2.1: a total admission that decided a non-trivial semantic property would be a decider for an undecidable set. A total terminating procedure exists only for decidable predicates; the largest such structure available is a decidable sub-collection, onto which admission maps. □

Corollary 2.2 is the reason the entire apparatus exists. One does not admit an observation by understanding it. One admits it by *retracting* it onto a sub-language — the *Rice quarantine* — on which the properties of interest are, by construction, decidable.

Chapter 3

Admission Algebra: Refusal as Data

3.1 The admission map and refusal as data

Definition 3.1 (Admitted space, obligations, admission). Fix a decidable sub-collection $\mathcal{O}^* \subseteq \mathcal{O}$ and a finite battery of *obligations* $g_1, \dots, g_m : \mathcal{O} \to \{0, 1\}$, each total and computable. The *admission map* is the partial retraction

$$\alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\perp\}, \qquad \alpha(o) = \begin{cases} \rho(o) \in \mathcal{O}^*, & \text{if } \bigwedge_{i=1}^{m} g_i(o) = 1, \\ \perp, & \text{otherwise,} \end{cases}$$

where $\rho : \mathcal{O} \rightharpoonup \mathcal{O}^*$ is a computable normalization fixing $\mathcal{O}^*$ pointwise ($\rho|_{\mathcal{O}^*} = \mathrm{id}_{\mathcal{O}^*}$) and $\perp$ is the distinguished *refusal*. Admission is idempotent on its image: $\alpha \circ \alpha = \alpha$.

Proposition 3.1 (Admission is a retraction). *α restricted to $\mathcal{O}^*$ is the identity, and $\alpha \circ \alpha = \alpha$ as partial maps. Hence $\mathcal{O}^*$ is a retract of* $\mathrm{dom}(\alpha)$*, and re-admitting an already-admitted observation neither changes it nor can newly refuse it.*

Proof. For $x \in \mathcal{O}^*$ every $g_i(x) = 1$ (obligations are decidable on the admitted language by construction), so $\alpha(x) = \rho(x) = x$. Thus $\alpha|_{\mathcal{O}^*} = \mathrm{id}_{\mathcal{O}^*}$. For general $o \in \mathrm{dom}(\alpha)$ with $\alpha(o) \neq \perp$ we have $\alpha(o) \in \mathcal{O}^*$, whence $\alpha(\alpha(o)) = \alpha(o)$; and $\alpha(\perp) = \perp$ by convention. Idempotence follows on all of $\mathrm{dom}(\alpha)$. □

Remark (Refusal is a value, not an exception). $\perp$ is a first-class output of a correctly functioning admission: the system computed that an obligation failed and recorded *which*. In `praxis` this is the type `Andon`, whose `Halted` variant carries both the unmet obligations and their refusal classification:

```
Andon::Halted{ unmet: Vec<Obligation>, refusals: Vec<RefusalScenario>, at: u64 }.
```

A refusal is data with a timestamp and a cause, not a thrown error. This is the formal content of the program's slogan *refusal is data.*

The obligations are themselves first-class, hashable values, not opaque closures. The `praxis` `Obligation` enum realizes exactly three decidable kinds, matching the three ways an observation can fail to be admitted:

`Obligation` variant	meaning	g_i decides
`Precondition{predicate_id, params_hash}`	a named predicate must hold	predicate membership
`BlockingConstraint{reason}`	a hard block until lifted	a flag
`EvidenceRequired{evidence_type}`	external evidence must be present	presence of a typed recor

Each is a syntactic, terminating check — a legitimate g_i under Corollary 2.2 — never a semantic decision about what the payload "really means."

3.2 The denial monoid and the eight-category taxonomy

Obligation checking composes, and the composition carries the algebra used throughout the series' geometry.

Construction 3.1 (Denial monoid)**.** Let $D = (\{0,1\}^n, \vee, \mathbf{0})$ be the commutative idempotent monoid of *denial words*: n independent lanes, componentwise OR, identity $\mathbf{0}$ ("admitted, all lanes clear"). Each refusal cause contributes a lane map $d_i : \mathcal{O} \to \{0,1\}^n$ with $d_i(o) = \mathbf{0} \iff$ cause i is absent. The *total denial* of an observation is $d(o) = \bigvee_i d_i(o)$, and $\alpha(o) \neq \bot \iff d(o) = \mathbf{0}$.

Proposition 3.2 (D is a bounded join-semilattice; denial is monotone)**.** *$(\{0,1\}^n, \vee, \mathbf{0})$ is a commutative idempotent monoid — the join-semilattice of the Boolean lattice $2^{[n]}$ with bottom $\mathbf{0}$. Consequently adding obligations can only enlarge the denial (turn admits into refusals), never the reverse.*

Proof. Idempotence $x \vee x = x$, commutativity, and associativity hold coordinatewise; a product of semilattices is a semilattice, with $\mathbf{0}$ the identity. For monotonicity, if $G' \supseteq G$ are cause sets then $d_{G'}(o) = d_G(o) \vee \bigvee_{i \in G' \setminus G} d_i(o) \succeq d_G(o)$ coordinatewise, and $d(o) = \mathbf{0}$ is the unique minimum, so $\{o : d_{G'}(o) = \mathbf{0}\} \subseteq \{o : d_G(o) = \mathbf{0}\}$. □

This construction is realized in code. The `bcinr_powl_receipt::denial::DenialPolarity` type is a newtype over `u64` with a zero element `ADMITTED` and seven named non-zero lanes; its `compose` operation is bitwise OR; and `praxis`'s `compose_denials` folds a set of refusal scenarios through `compose` starting from `ADMITTED` — literally the monoid fold $\bigvee_i d_i$ with the identity as the empty product. A refusal *category* is then the coarse classification of a lane.

Definition 3.2 (Refusal taxonomy)**.** A *refusal taxonomy* is a pair (S, cat) where S is a finite set of concrete refusal scenarios and $\text{cat} : S \to C$ is a total map onto a fixed finite set C of categories. In `praxis`, C is the eight-element set

$$C = \{\mathsf{Identity}, \mathsf{Capacity}, \mathsf{Topology}, \mathsf{Temporal}, \mathsf{Lifecycle}, \mathsf{Authorization}, \mathsf{Prerequisites}, \mathsf{Reserved}\},$$

S is the `RefusalScenario` enum (obligation-driven halts, the seven `DenialPolarity` lanes, and the prolog8 kernel / andon-ring denials), and $\text{cat} =$ `RefusalScenario::category`.

Proposition 3.3 (Totality is mechanized)**.** *The category map $\text{cat} : S \to C$ is total, and its totality is enforced by the host type system: `RefusalScenario::category` is a `match` with no wildcard arm, so a new scenario variant added without a category is a compile error. Moreover the lane assignment $\text{lane} : S \to D$ (`denial_lane`) and the single-lane inverse $\text{scn} : D \rightharpoonup S$ (`scenario_for_denial_lane`) satisfy $\text{lane} \circ \text{scn} = \text{id}$ on each of the seven named lanes, i.e. scn is a section of lane over the single-lane words.*

Proof. Totality of a wildcard-free exhaustive `match` over a closed enum is exactly the compiler's exhaustiveness guarantee: the program does not type-check unless every variant is covered, so `category` denotes a total function $S \to C$. The section identity is the round-trip property verified by the module's test `category_mapping_is_total_over_denial_lanes`: for each of the seven single-lane constants ℓ, $\mathrm{scn}(\ell)$ is defined and $\mathrm{lane}(\mathrm{scn}(\ell)) = \ell$. That scn is only a *partial* section (undefined on composed multi-lane words and on `ADMITTED`) reflects that a composed denial has no single scenario; it is a join $\bigvee$ of several, per Construction 3.1. □

Proposition 3.3 is the algebraic backbone of the admission pillar (Chapter 8): the taxonomy is not documentation but a total function whose totality the compiler certifies, and the denial lattice is the semantic domain of that function.

Chapter 4

The BRCE Byte: Machine Algebra of Standing

4.1 CPU notation for the BRCE byte

When O^* is projected into the machine's fastest lawful state, it becomes a single byte. We now assign notation to the physical operations by which the Bounded Receipted Chatman Equation (BRCE) achieves execution at the picosecond scale.

Let $\mathbb{B} = \{0, 1\}$ and let the BRCE standing byte be $b \in \mathbb{B}^8$. Write $0 = 00000000_2$ and $1 = 11111111_2$. Each basis bit is $e_i \in \mathbb{B}^8$, for $0 \le i < 8$, where e_i has a 1 in lane i and 0 everywhere else. A standing byte is $b = b_0 e_0 \vee b_1 e_1 \vee \cdots \vee b_7 e_7$. In the canonical instance, the lanes represent:

$$
\begin{array}{llll}
b_0 = \text{admitted}, & b_1 = \text{evidenced}, & b_2 = \text{budgeted}, & b_3 = \text{authorized}, \\
b_4 = \text{healthy}, & b_5 = \text{conformant}, & b_6 = \text{receipted}, & b_7 = \text{replayable}.
\end{array}
$$

Define the register-level CPU operation alphabet $\mathcal{I}_8$:

$$\mathcal{I}_8 = \{\wedge_8, \vee_8, \oplus_8, \neg_8, \lll_k, \ggg_k, =_c, \mathrm{z}, \mathrm{nz}, \mathrm{sel}, \mathrm{pop}_8\}.$$

These correspond to the primitive machine instructions acting directly on fixed-width state: bitwise AND ($\wedge_8$), OR ($\vee_8$), XOR ($\oplus_8$), NOT ($\neg_8$), constant comparison ($=_c$), zero test (z), nonzero test (nz), branchless select (sel), and population count (pop_8). A lawful BRCE hot path is a word over $\mathcal{I}_8$.

4.2 The machine algebra of lawful action

With the CPU alphabet defined, every governance operation reduces to a bounded Boolean map over $\mathbb{B}^8$.

Admission and Denial. The denial byte is the complement of the standing byte: $D(b) = \neg_8 b$. No denial exists when $D(b) = 0$. Admission is exactly the zero-test of the denial byte:

$$A_{\text{gate}}(b) = \mathrm{z}(D(b)) \implies A_{\text{gate}}(b) = 1 \iff b = 1.$$

Refusal is the nonzero test: $H(b) = \mathrm{nz}(D(b))$. If $d^{(1)}, \ldots, d^{(n)}$ are denial bytes from independent obligations, their composed denial is simply the bitwise OR: $d_\Sigma = d^{(1)} \vee_8 d^{(2)} \vee_8 \cdots \vee_8 d^{(n)}$. Admission exists exactly when $d_\Sigma = 0$.

Policy masks and repairs. Let $m \in \mathbb{B}^8$ be a required policy mask. The policy predicate is $P_m(b) = [(b \wedge_8 m) = m]$, meaning every lane required by m is present in b. The missing lanes are isolated by $M_m(b) = m \wedge_8 \neg_8 b$, so $P_m(b) = 1 \iff M_m(b) = 0$. A policy is just a mask; a violation is simply "mask AND NOT byte". Repairing a lane i is $b' = b \vee_8 e_i$.

Completeness and Branchless Selection. Completeness is the population count $C(b) = \mathrm{pop}_8(b) \in \{0, \ldots, 8\}$. Repair distance is $R(b) = 8 - \mathrm{pop}_8(b)$. This allows the system to determine that an object is "missing one lane" without inspecting raw meaning. For a predicate bit $p \in \mathbb{B}$, define its full-byte mask $\hat{p} = 1$ if $p = 1$ else 0. Branchless select is $\mathrm{sel}(p, u, v) = (\hat{p} \wedge_8 u) \vee_8 (\neg_8 \hat{p} \wedge_8 v)$.

Action selection. Define the byte-to-integer decoder $\nu(b) = \sum_{i=0}^{7} b_i 2^i$, mapping $\mathbb{B}^8 \to \{0, \ldots, 255\}$. Reserving zero ($\nu(b) = 0 \implies \perp$), the action selector is:

$$\sigma(b) = \begin{cases} \perp, & \nu(b) = 0, \\ a_{\nu(b)}, & 1 \leq \nu(b) \leq 255. \end{cases}$$

Thus $|\,\mathrm{im}(\sigma) \setminus \{\perp\}| \leq 255$. Eight bits can select 255 non-null lawful actions without branching.

The constant-depth eligibility theorem. Let τ be the cycle quantum of the target machine. For primitive register operations, $\mathrm{cost}(i) = 1$ for $i \in \mathcal{I}_8$. The execution depth $\mathrm{depth}(f)$ of a composed expression f is its longest dependency chain in $\mathcal{I}_8$. The bounding constraint is $T(f) \approx \mathrm{depth}(f) \cdot \tau$. The admission check $G(b) = [b = 1]$ has depth 1. The policy mask $P_m(b)$ has depth 2. From first principles, BRCE proves that admission, denial composition, policy masking, and action selection are finite Boolean maps with depth $\in \{1, 2, 3\}$. The hot-path eligibility checks compile to constant-depth Boolean expressions over fixed-width registers. On machines whose primitive register operations retire within a sub-nanosecond cycle budget, the admission predicate is picosecond-eligible.

Thus BRCE does not merely claim that eight bits are compact; it gives a CPU-level algebra in which lawful standing, denial, policy, repair, and action selection are all words over the machine's primitive register operations. BRCE opens high-frequency domains by making law executable as byte algebra.

Chapter 5

The Lifecycle as a Typestate Category

5.1 Stages, transitions, and the free path category

An admitted observation does not become a receipt in one step. It traverses a fixed lifecycle, and the *illegality of skipping steps* is the first structural guarantee a newcomer must trust. We make it a theorem.

Definition 5.1 (Lifecycle category)**.** Let **Life** be the category with four objects, the lifecycle stages

$$\mathsf{Raw}, \quad \mathsf{Validated}, \quad \mathsf{Admitted}, \quad \mathsf{Receipted},$$

and with non-identity morphisms generated by exactly three arrows

$$j : \mathsf{Raw} \to \mathsf{Validated}, \qquad a : \mathsf{Validated} \to \mathsf{Admitted}, \qquad r : \mathsf{Admitted} \to \mathsf{Receipted},$$

(*judge*, *admit*, *receipt*), closed under composition and identities. **Life** is the free category on the linear quiver $\mathsf{Raw} \xrightarrow{j} \mathsf{Validated} \xrightarrow{a} \mathsf{Admitted} \xrightarrow{r} \mathsf{Receipted}$.

Proposition 5.1 (Hom-sets of **Life**)**.** *For stages X, Y, the hom-set* $\mathbf{Life}(X, Y)$ *has exactly one element if Y is reachable from X along the quiver (including $X = Y$, the identity) and is empty otherwise. In particular* $\mathbf{Life}(\mathsf{Raw}, \mathsf{Receipted}) = \{r \circ a \circ j\}$ *is a singleton, and there is* no *morphism* $\mathsf{Raw} \to \mathsf{Admitted}$ *that is not $a \circ j$, no morphism* $\mathsf{Validated} \to \mathsf{Receipted}$ *other than $r \circ a$, and no backward or skipping morphism at all.*

Proof. In the free category on a quiver, morphisms $X \to Y$ are directed paths from X to Y modulo nothing (paths are the morphisms). The quiver here is a simple directed path with no branches and no cycles, so between any two vertices there is at most one directed path: the sub-path of the unique maximal path, if Y lies after X, and none otherwise. Hence each nonempty hom-set is a singleton and each "illegal" hom-set (backward, or skipping an intermediate vertex) is empty. □

5.2 Illegal transition is a type error

The `praxis` implementation represents a law object as a type carrying its stage as a phantom parameter:

`LawObject<Payload, S: Stage, Law>`,

where `Stage` is a *sealed* trait implemented only by the four zero-sized types `Raw`, `Validated`, `Admitted`, `Receipted`, and the phantom fields `_stage`, `_law` are private to the defining module. The three lifecycle arrows are the only stage-changing operations exposed: `Judge::judge` has type $\mathsf{Raw} \to \mathsf{Validated}$ (returning the halted `Raw` on refusal), `Admit::admit` has type $\mathsf{Validated} \to \mathsf{Admitted}$, and `receipt` is a method available *only* on `LawObject<_,Admitted,_>`, with signature $\mathsf{Admitted} \to \mathsf{Receipted}$, consuming its receiver. We now state the correspondence precisely.

Theorem 5.2 (Illegal lifecycle transitions are uninhabited)**.** *Let $\mathcal{T}$ be the host type system and interpret each stage X as the type family `LawObject<P,X,L>` and each exposed stage-changing operation as a term of the corresponding function type. Then the set of stage-changing operations denotes exactly the generating arrows $\{j, a, r\}$ of* **Life***, and consequently:*

(i) every well-typed stage-changing program is a composite of j, a, r, i.e. a morphism of **Life** *(Proposition 5.1);*

(ii) for any pair (X, Y) with $\mathbf{Life}(X, Y) = \varnothing$ *— a backward or step-skipping transition — there is no term of type `LawObject<P,X,L>` $\to$ `LawObject<P,Y,L>` constructible from the exposed interface; such a program does not type-check;*

(iii) no law object can be receipted twice: r consumes its `Admitted` receiver, so the term is linear in its argument and $\mathsf{Receipted}$ *has no outgoing stage-changing arrow.*

Proof. (i) The only public constructor produces a `Raw` object; the only public stage-changing operations are `judge`, `admit`, `receipt`, whose types are j, a, r respectively. The stage transition helper that rebuilds an object at a new stage is crate-private (`pub(crate)`), and the phantom fields are private, so no external term can fabricate a `LawObject` at a chosen stage. Hence every externally well-typed stage-changing program is a composition of j, a, r, which by definition is a morphism of the free category **Life**. (ii) Suppose $\mathbf{Life}(X, Y) = \varnothing$. By Proposition 5.1 there is no path $X \to Y$ in the quiver, so no composite of j, a, r has type `LawObject<P,X,L>` $\to$ `LawObject<P,Y,L>`. Since (i) shows composites of j, a, r are the only such terms, the required term does not exist; a program asserting it fails to type-check. Concretely, `receipt` is impl'd only on the `Admitted` type, so calling it on `Raw` or `Validated` is a missing-method type error, and `admit` takes `Validated` by value, so feeding it a `Raw` is a type mismatch. (iii) `receipt` takes `self` by value (move), consuming the `Admitted` object; the returned `Receipted` type has no method producing a further stage. Thus the argument is used exactly once and the terminal object $\mathsf{Receipted}$ emits no stage-changing arrow, so a second receipting is both unconstructible (no arrow out of $\mathsf{Receipted}$) and unavailable (the receiver was moved). □

Corollary 5.3 (The admissible sub-category is what compiles)**.** *The sub-category of "admissible morphisms" — those a program may actually perform — is precisely* **Life***, and it coincides with the set of well-typed stage-changing programs over the `LawObject` interface. "Illegal transition is a type error" is therefore not a runtime check to be tested but a statement about which hom-sets are inhabited, discharged by the type checker before the program runs.*

Proof. Combine Theorem 5.2(i)–(ii): the inhabited stage-changing types are exactly the morphisms of **Life**, and the uninhabited ones are exactly the empty hom-sets. The admissible sub-category is thus **Life** itself, verified statically. □

Remark (Where the parallel to admission lives). Chapter 2 retracted an undecidable observation space onto a decidable admitted language; here the type checker retracts the space of *all conceivable stage sequences* onto the decidable (indeed, trivially decidable) language of paths in **Life**. Both are Rice quarantines: an undecidable or unwieldy space of behaviors is replaced by a small decidable sub-language whose membership is mechanically certified. Admission does it at values; typestate does it at programs.

5.3 Judgment carries its refusal

The arrow $j : \mathsf{Raw} \to \mathsf{Validated}$ is total in a specific sense: it never throws, it either advances the object or returns it halted, carrying its denial. In `praxis`,

$$\texttt{Judge::judge} : \texttt{LawObject<P,Raw,L>} \longrightarrow \texttt{Result<LawObject<P,Validated,L>, LawObject<P,Raw,L>>},$$

where the `Err` branch is the *same object*, now in `Andon::Halted` with its `unmet` obligations and `refusals`. This is Definition 3.1 at the type level: $\alpha(o)$ is either an element of $\mathsf{Validated} \subseteq \mathcal{O}^*$ or the refusal $\perp$, and the refusal is a value one can inspect, not control flow one must catch. The admit arrow $a : \mathsf{Validated} \to \mathsf{Admitted}$ is analogous, returning `Err(Andon)` when admission is denied.

5.4 The 3-Node SEQ Lifecycle Token Game

The lifecycle transitions of the `LawObject` parameters represent a sequential path: `judge` $\to$ `admit` $\to$ `receipt`.

Definition 5.2 (3-Node SEQ POWL Token Game)**.** The lifecycle process is a safe Petri net with state $M \in \{0,1\}^4$ and transitions $T = \{j, a, r\}$ representing the judge, admit, and receipt actions. The token state is:

$$M = (\texttt{TOK\_START},\ \texttt{TOK\_JUDGED},\ \texttt{TOK\_ADMITTED},\ \texttt{TOK\_DONE}), \tag{5.1}$$

with firing rules enabled by coordinatewise subtraction and addition:

$$M_{\text{next}} = (M \setminus \boldsymbol{m}^-) \cup \boldsymbol{m}^+.$$

Fitness $\varphi \in [0,1]$ is computed in Q16.16 format. An out-of-order transition triggers a token shortfall and results in $\varphi < 1.0$.

Chapter 6

Manufacture, Receipt, and the Enforced Equation

6.1 The manufacturing morphism

Definition 6.1 (Manufacturing morphism)**.** A *manufacturing morphism* is a deterministic computable map $\mu : \mathcal{O}^* \to \mathcal{A}$ subject to two laws:

(M1) **Determinism / reproducibility:** $\mu(x)$ depends only on x; equal admitted inputs yield bit-identical artifacts.

(M2) **Boundedness:** μ factors through a representation of size bounded by fixed structural constants, so μ terminates with a priori bounded cost.

μ is undefined on $\mathcal{O} \setminus \mathcal{O}^*$ by construction, and refusal propagates: $\mu(\bot) = \bot$.

The governing identity of Definition 1.1, $\mathcal{A} = \mu(\mathcal{O}^*)$, is now precise: operationally $a = \mu(\alpha(o))$ whenever $\alpha(o) \neq \bot$, and undefined (a receipted refusal) otherwise. In `praxis`, one instance of μ is the manufacture of a PDDL8 planning domain and problem from a `pdl:` RDF ontology (the `mfg pddl` verb over `bcinr-pddl`): the ontology graph is hashed with BLAKE3 (a `graph_hash` witnessing determinism), and identical ontologies manufacture identical domain text, exactly (M1). The grounding-and-solving stage is (M2)'s bounded representation: a finite lifted domain grounded to a finite action set the planner searches.

Proposition 6.1 (Determinism makes μ a function on receipts)**.** *Under (M1), the assignment $x \mapsto \mu(x)$ is a function (single-valued), hence the composite $o \mapsto \mu(\alpha(o))$ is a partial function on $\mathcal{O}$. Two runs on the same admitted input therefore agree bit-for-bit, and any disagreement is evidence that the input, not the morphism, differed.*

Proof. (M1) is exactly single-valuedness. Composition of the partial function α with the function μ is a partial function. Bit-identity of outputs on equal inputs is the contrapositive of the last clause. $\square$

6.2 Receipts

A manufacture leaves an *execution trace* $\sigma \in \Sigma$: every intermediate marking and sub-artifact. Traces live in a space of unbounded dimension relative to κ. The receipt is the bounded, collision-committed projection of that trace.

Definition 6.2 (Receipt and chain)**.** Fix a collision-resistant hash $\mathsf{H} : \{0,1\}^* \to \{0,1\}^{256}$ (in `praxis`, BLAKE3 [5]). For a manufacture with payload bytes b, previous chain value h_-, and metadata θ (instruction index, activity, node kind, timestamp, denial word), the *frame* is a fixed-width encoding $\mathsf{fr} = \langle \theta,\ \mathsf{H}(b) \rangle$ and the *chained receipt* is

$$h_+ \ = \ \mathsf{H}\big(h_- \,\|\, \mathsf{fr}\big). \tag{6.1}$$

The *receipt* is the tuple $r = (\text{verdict}, h_+, \varphi, \text{reason}) \in \mathcal{R}$ of at most κ chunks: a Boolean verdict, the 256-bit chain value, a scalar conformance fitness φ, and a refusal reason (empty on accept).

Equation (6.1) is the `OcelCausalFrame` construction in `praxis`: the 32-byte BLAKE3 payload hash is packed into the frame's eight object-reference words as little-endian `u32`s (repurposing the OCEL [6] object slots as a content commitment), the honest denial word from Chapter 2 is written into the frame's `denial/fired_mask`, and the chain step is `chain_hash`$(t{+}1)$ = $\mathsf{H}($`chain_hash`$(t) \,\|\,$ `frame_bytes`$(t{+}1))$. Determinism is tested directly: identical inputs produce identical chain hashes; differing payloads, previous hashes, instruction ids, or denial words produce differing chain hashes.

Definition 6.3 (Conformance fitness)**.** Let $\varphi \in [0,1]$ be one minus the fraction of tokens a replay was forced to consume on unenabled transitions of the lifecycle process model. $\varphi = 1$ iff the replay never attempted a disabled firing. In `praxis`, the lifecycle is the fixed three-node sequence `judge` $\to$ `admit` $\to$ `receipt`, replayed by `PowlReplayVerifier`; a lawful in-order trace yields fitness `0x0001_0000` — the value 1.0 in Q16.16 fixed point — while any out-of-order or step-skipping trace is a `TokenNotEnabled` violation with $\varphi < 1$.

The full theory of the receipt — that a bounded verifier reading $O(1)$ symbols rejects any perturbed trace except with negligible probability — is the subject of the receipt cryptography pillar (Chapter 8) and the keystone's Faithful Projection Theorem. We state here only the commitment property we need for conservation.

Lemma 6.2 (Chain commitment)**.** *Under the collision resistance of* H*, the chain value* h_+ *of* (6.1) *is a binding commitment to the pair* (h_-, fr)*: producing a distinct* (h'_-, fr') *with the same* h_+ *requires exhibiting a collision of* H*. By induction along the chain,* h_+ *binds the entire causal prefix.*

Proof. If $\mathsf{H}(h_- \| \mathsf{fr}) = \mathsf{H}(h'_- \| \mathsf{fr}')$ with distinct arguments, the two arguments are a collision of H, which occurs with negligible probability under collision resistance. Since h_- is itself such a commitment to the previous frame, an induction on chain length shows h_+ commits to every frame in the prefix. □

6.3 The Bounded Receipted Chatman Equation

The Chatman equation $\mathcal{A} = \mu(\mathcal{O}^*)$ is a mathematical identity; its *enforced* form is the conjunction of invariants a running system maintains. We name it once for the series.

Definition 6.4 (BRCE: Bounded Receipted Chatman Equation)**.** A system satisfies the *Bounded Receipted Chatman Equation* if for every actuated artifact $a \in \mathcal{A}$ it maintains all four invariants:

(B1) **Admission gate:** $a = \mu(x)$ for some $x = \alpha(o) \neq \perp$; nothing is actuated from a raw or refused observation ($\operatorname{im}\mu$ exhausts the actuated set).

(B2) **Bounded manufacture:** μ satisfies (M1)–(M2); manufacture is deterministic and a priori cost-bounded.

(B3) **Receipt totality:** every actuation appends exactly one chained receipt (6.1); there is no unreceipted actuation.

(B4) **Conformance:** the actuation's lifecycle replay has fitness $\varphi = 1$ (Definition 6.3); the trace stays within the process model.

Theorem 6.3 (Conservation of Consequence)**.** *Under* BRCE*, the map $a \mapsto h_+(a)$ from actuated artifacts to receipt-chain positions is well defined and injective up to hash collision, and every actuated artifact is the image under μ of exactly one admitted observation. Equivalently: consequence is neither created without an admitted cause (B1) nor destroyed without a receipt (B3), and the chain preserves order.*

Proof. By (B1) actuation implies $a = \mu(x)$ with $x = \alpha(o) \neq \perp$, so $\operatorname{im}\mu$ is the whole actuated set and its complement is never actuated. By (B2)/(M1) and Proposition 6.1, $x \mapsto \mu(x)$ is single-valued, so each actuated a has a determined admitted preimage x (unique as the input that produced it under a deterministic μ; distinct admitted inputs yielding the same artifact are identified by the artifact, which is all conservation asserts). By (B3) the actuation appends exactly one chain position, so $a \mapsto h_+(a)$ is a well-defined total map on actuated artifacts. Injectivity up to collision is Lemma 6.2: if two distinct actuations shared a chain value, their frames would collide under H. Order preservation is the recursive dependence of h_+ on h_- in (6.1). □

Corollary 6.4 (The antibody clause)**.** *Define the* overclaim set *as actuations lacking a valid receipt. Under* BRCE *the overclaim set is empty as an invariant: an artifact without a receipt satisfying (B3) is not actuated. Hence the admissible magnitude of any claim a system makes is bounded by what its mechanism can receipt, not by its rhetoric.*

Proof. (B3) requires a receipt per actuation; an actuation without one violates BRCE and is, by (B1)'s gate, never emitted. So the set of receiptless actuations is empty, and every standing claim corresponds to a receipted artifact whose verification cost is $O(\kappa)$ (keystone). □

This is the formal statement of the program's discipline: *a grand system without receipts is grandiosity; a grand system that receipts its own limits is machinery.*

Chapter 7

A Worked Example: A Contract-Claim Law Object

We instantiate the whole apparatus on one object, tracing it Raw $\to$ Receipted with its obligations, an andon halt and its lift, and the resulting chain hash. The example is a *contract claim*: a payload asserting that a counterparty owes a delivery under a signed contract, submitted for lawful processing.

7.1 Raw

The caller submits an observation $o \in \mathcal{O}$, a JSON payload, to `law judge`:

```
{ "claim": "delivery-owed", "contract_id": "C-4471", "amount": 12000,
"counterparty": "acme", "evidence": null }
```

The governing `Law` attaches three obligations (Definition 3.1), each a decidable g_i:

g_1: `Precondition{predicate_id:"contract_active", params_hash: h(C-4471)}` — the referenced contract must be in the active set.

g_2: `EvidenceRequired{evidence_type:"signed_contract"}` — a signed-contract record must be present.

g_3: `BlockingConstraint{reason:"amount_within_authority"}` — the claimed amount must lie within the desk's authority.

At submission the object is `LawObject<Claim,Raw,ContractLaw>` with `andon` = `Green` and `chain_hash` = `None`.

7.2 Judgment and an andon halt

`Judge::judge` evaluates g_1, g_2, g_3. The contract `C-4471` is active ($g_1 = 1$), but `evidence` is `null`: the signed-contract record is absent ($g_2 = 0$). By Definition 3.1, $\alpha(o) = \perp$. The arrow j returns the `Err` branch — the same object, in

`Andon::Halted{ unmet:[EvidenceRequired{"signed_contract"}], refusals:[MissingEvidence{...}], at`

By Proposition 3.3, cat(`MissingEvidence`) = Prerequisites and lane(`MissingEvidence`) = `PRECONDITION_FAILED`; the total denial word is $d(o) = \texttt{PRECONDITION\_FAILED} \neq \mathbf{0}$. This is a lawful output: a refusal with a category, a lane, and a timestamp — data, not an exception (Chapter 2). No manufacture occurs, because μ is undefined on $\perp$ (Definition 6.1); the line is halted, andon-red.

7.3 Evidence, re-judgment, admission

The caller supplies the missing evidence and resubmits:

```
{ ..., "evidence": { "signed_contract": "blob:9f2c..." } }
```

Now $g_2 = 1$ and $g_3 = 1$ (the amount 12000 is within authority), so $\bigwedge_i g_i = 1$ and $\alpha(o) = \rho(o) \in \mathcal{O}^*$: the object advances $j : \mathsf{Raw} \to \mathsf{Validated}$ to `LawObject<Claim,Validated,ContractLaw>` with `andon` = `Green`. Then $a : \mathsf{Validated} \to \mathsf{Admitted}$ admits it to `Admitted`. By Theorem 5.2 nothing could have jumped from `Raw` straight to `Admitted`; the two arrows were mandatory and in order.

7.4 Manufacture and receipt

The admitted claim is manufactured into its artifact (the actuated obligation-to-deliver record) and receipted. `receipt` consumes the `Admitted` object and appends the frame of Definition 6.2. With previous chain value h_-, a deterministic timestamp, instruction id, and the honest denial word `ADMITTED` (the halt was lifted, so the receipt records a clean admission), the payload is hashed $b \mapsto \mathsf{H}(b)$ and the chain advances

$$h_+ = \mathsf{H}\big(h_- \parallel \mathsf{fr}(\theta, \mathsf{H}(b))\big).$$

The object is now `LawObject<Claim,Receipted,ContractLaw>` carrying `chain_hash` = `Some`(h_+). A lifecycle replay of `judge` → `admit` → `receipt` yields fitness `0x0001_0000` = 1.0 (Definition 6.3); all four BRCE invariants hold. The boundary projection the system exposes is the κ-sized tuple

$$r = \big(\text{verdict} = \mathsf{pass},\ h_+ = \texttt{<256-bit hex>},\ \varphi = 1.0,\ \text{reason} = \varnothing\big),$$

and by Theorem 6.3 this receipt is the unique chain position of this actuation, injective up to collision. A verifier trusts the delivery obligation by reading r — four chunks — never by re-deriving the manufacture.

Remark (The agent's byte). In the membrane deployment, each connected agent carries an eight-bit projection of its lifecycle status — the `agent8` byte — whose flags are set by exactly these events: a halt sets a blocked bit, a receipt sets a receipted bit. The worked object above drove one session's byte from blocked (at the andon halt) to receipted (after the chain advanced). The byte is the smallest receipt of all: a single octet, popcount-summable across a fleet, projecting a whole lifecycle into eight bits a scheduler can read in one instruction.

Chapter 8

Map of the Series

This foundations paper fixes the five objects, the Rice quarantine, the typestate category, and BRCE. Four pillar papers develop each face in depth; the reader should approach them in any order after this one.

Pillar I — Admission Algebra. Deepens Chapter 2: the denial monoid D (Construction 3.1), the eight-category taxonomy as a total function (Proposition 3.3), and the algebra of obligation composition, including the join-irreducible structure of refusal categories and the prolog8 kernel's proof-carrying admission path. It is the algebra beneath "refusal is data."

Pillar II — Receipt Cryptography. Deepens Chapter 6, § 2: the BLAKE3 `OcelCausalFrame` chain (6.1), the Faithful Projection Theorem (a bounded verifier reads $O(1)$ symbols and rejects any perturbation except with negligible probability), the affidavit `verify` pipeline (`decode`, `check_format`, `chain_integrity`, `continuity`, `verify_commitments`, `evaluate_profile`), and receipt signing. It is the cryptography beneath Lemma 6.2 and Theorem 6.3.

Pillar III — Planning and Geometry. Develops manufacture as search and conformance as geometry: the `bcinr-pddl` planner that grounds and solves a manufactured PDDL8 domain, the marking polytope and state equation of token-flow execution, conformance as membership with Farkas separating-hyperplane certificates, and `PowlReplayVerifier` fitness (Definition 6.3) as normalized distance to the polytope. It is the geometry beneath the conformance invariant (B4).

Pillar IV — The Projection Principle (keystone). The already-standing keystone *The Projection Principle* unifies the above around the Comprehension–Verification Gap: verification cost is $O(\kappa)$ per receipt while comprehension cost is $\Omega(\dim \Sigma)$, so trust in a system of unbounded interior is available at constant human cost. BRCE (Definition 6.4) is that paper's operating premise made into a checklist.

Prior work — [1]. The published paper *The Chatman Equation and the Industrial Revolution of Knowledge* (v1.2.0) demonstrated $\mathcal{A} = \mu(\mathcal{O})$ at enterprise scale: knowledge hooks $h = (\text{trigger}, \text{check}, \text{act}, \text{receipt})$ over RDF/SHACL, the KNHK hot-path executor ($\leq 2\,\text{ns}$ rule checks), the `ggen` bounded projection, Lockchain provenance, and full coverage of the forty-three workflow patterns [4]. This series factors that paper's μ into admission α and manufacture μ, names the receipt space $\mathcal{R}$ it produced, and supplies the first-principles proofs its scale rested on.

Chapter 9

Conclusion

We set out to give the series its floor. The Chatman equation $\mathcal{A} = \mu(\mathcal{O}^*)$ relates five named objects; admission is a computable retraction rather than a semantic decision, because Rice's theorem forbids the latter (Theorem 2.1, Corollary 2.2); refusal is a first-class value carrying its category through a denial monoid whose taxonomy is a compiler-certified total function (Proposition 3.3); the lifecycle $\mathsf{Raw} \to \mathsf{Validated} \to \mathsf{Admitted} \to \mathsf{Receipted}$ is a free path category in which every illegal transition is an uninhabited hom-set, so "illegal-transition-is-a-type-error" is a theorem discharged before runtime (Theorem 5.2); manufacture is deterministic and bounded; the receipt is a collision-binding projection; and the enforced form BRCE yields conservation of consequence (Theorem 6.3) and its antibody corollary. A single contract-claim object exhibited all of it, halting on missing evidence and advancing to a receipted chain position once the evidence arrived.

The standard was never comprehension. It is a lawful admission, a bounded manufacture, and a faithful receipt — each small enough to hold, each guaranteed by mechanism. Everything downstream in this series is a deeper reading of one of those three.

Appendix A

Notation

Symbol	Meaning
$\mathcal{O},\ \mathcal{O}^*$	observation space; admitted (decidable) sub-space $\mathcal{O}^*$
$\alpha,\ \perp$	admission retraction; refusal (first-class value)
$g_i,\ d(o)$	obligations; total denial word
$D = (\{0,1\}^n, \vee, \mathbf{0})$	denial monoid (join-semilattice)
$\mu,\ \mathcal{A}$	manufacturing morphism; artifact/action space
$\Sigma,\ \mathcal{R}$	execution traces; receipt space
$\mathsf{H},\ h_\pm$	collision-resistant hash (BLAKE3); chain values
φ	conformance fitness $\in [0,1]$ (Q16.16 in code; 1.0 =`0x0001_0000`)
κ	human working-memory capacity (chunks), a fixed small constant
Life	lifecycle category on $\mathsf{Raw} \to \mathsf{Validated} \to \mathsf{Admitted} \to \mathsf{Receipted}$
j, a, r	judge, admit, receipt arrows
BRCE	Bounded Receipted Chatman Equation (B1–B4)

Appendix B

Code Correspondence

Every abstract object above is anchored to a location in the `praxis` repository.

Mathematical object	Code artifact	Location
$\mathcal{O}$, raw observation	`law judge` payload	`src/verbs/law.rs`
α, admission / $\perp$	`Judge/Admit`, `Andon`	`crates/praxis-core/src/law.rs`
g_i, obligations	`Obligation` enum (3 kinds)	`crates/praxis-core/src/law.rs`
D, denial monoid	`DenialPolarity`, `compose_denials`	`crates/praxis-core/src/refusal.rs`
cat, 8 categories	`RefusalCategory`, `::category`	`crates/praxis-core/src/refusal.rs`
Life, typestate	`LawObject<P,S:Stage,L>`, sealed `Stage`	`crates/praxis-core/src/lifecycle.r`
μ, manufacture	`mfg pddl` over `bcinr-pddl`	`src/verbs/mfg.rs`
h_+, receipt chain	`OcelCausalFrame`, `build_admission_frame`	`crates/praxis-core/src/law.rs`
φ, fitness	`PowlReplayVerifier`, `replay_lifecycle`	`crates/praxis-core/src/replay_adap`
BRCE verify	`verify` affidavit pipeline	`crates/praxis-core/src/verify.rs`
`agent8` byte	resident `AgentByte` over MCP	`src/bin/mcp_lawobject_server.rs`

Bibliography

[1] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.

[2] H. G. Rice, *Classes of recursively enumerable sets and their decision problems*, Transactions of the American Mathematical Society **74** (1953), 358–366.

[3] R. E. Strom and S. Yemini, *Typestate: A programming language concept for enhancing software reliability*, IEEE Transactions on Software Engineering **SE-12**(1) (1986), 157–171.

[4] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.

[5] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, *BLAKE3: One function, fast everywhere*, 2020.

[6] A. F. Ghahfarokhi, G. Park, A. Berti, and W. M. P. van der Aalst, *OCEL: A standard for object-centric event logs*, in New Trends in Database and Information Systems, Springer, 2021.

[7] G. A. Miller, *The magical number seven, plus or minus two: some limits on our capacity for processing information*, Psychological Review **63**(2) (1956), 81–97.

[8] N. Cowan, *The magical number 4 in short-term memory: A reconsideration of mental storage capacity*, Behavioral and Brain Sciences **24**(1) (2001), 87–114.

The Admission Monoid:

An Algebra of Refusal

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Pillar I, Paper 01

Abstract

The foundations paper of this series established that admission cannot be a semantic decision and must be a computable retraction, with *refusal* promoted from an exception to a first-class value [?]. This paper develops the algebra of that value. We take as our object the *denial word* — the machine-checkable record of *why* an observation was not admitted — and prove that the space of denial words is a bounded, commutative, idempotent monoid (a join-semilattice) whose identity is the clean word ADMITTED and whose product is componentwise disjunction, realized in `praxis` as the `DenialPolarity` bitfield under `compose`. We show that obligations are *lane maps* into this monoid, that total denial is their join, and that admission is *antitone* in the obligation set: adding obligations can only shrink the admitted set (monotonicity). We then formalize the eight-bucket `RefusalCategory` taxonomy as a total classification $\mathrm{cat} : \mathcal{S} \to \mathcal{C}$ and prove a *mechanized totality theorem*: cat is realized by a wildcard-free `match`, so it is compiler-certified total, its image is exactly the seven inhabited buckets, and the eighth (`Reserved`) carries an empty preimage by design — an honest slack coordinate, not an overclaim. We identify precisely where the type system's totality guarantee ends and a runtime test must continue: the lane decoder `scenario_for_denial_lane` is a *partial* map because its domain is an open 2^{64} newtype rather than a closed enum. We prove that admission composes across a manufacturing pipeline as the unique monoid homomorphism from the free monoid on stages into the denial monoid, and that this composition is order- and multiplicity-independent, so a pipeline admits exactly when every stage admits. Finally, we refine admission at the logical layer: the `prolog8` kernel decides bounded Horn queries by SLD-resolution and returns a *proof-carrying trichotomy* — an `Answered` query ships a positive proof DAG, a `Denied` query ships a negative proof tree, and an `Invalid` query ships a `RejectionCode`; we prove this trichotomy and show it embeds back into the denial monoid. Every object is anchored to a line of the `praxis` repository, so a reader can trace each theorem to the code that enforces it.

Contents

Chapter 1

Introduction: The Algebra a Refusal Needs

1.1 From "refusal is data" to an algebra of refusal

The prior published work [**?**] established the governing identity $\mathcal{A} = \mu(\mathcal{O}^*)$ and demonstrated it at enterprise scale. The foundations paper of this series (Paper 00) grounded it: it proved a specialization of Rice's theorem [**?**] to observations, concluded that admission *cannot* decide meaning and *must* retract onto a decidable sub-language, and promoted the *refusal* $\perp$ from a thrown exception to a returned value. The keystone thesis introduced, in a single Construction, the *denial monoid* $D = (\{0,1\}^n, \vee, \mathbf{0})$ and one proposition about it. This paper pays that Construction its full debt.

The reason a refusal needs an algebra, rather than merely a name, is scale. Among a fleet of $\sim 10^{12}$ agents, each with bounded context κ and none able to read another's interior, a refusal must satisfy four demands simultaneously:

(D1) **Composition.** A manufacture passes through many admission stages; each may refuse; the fleet needs *one* verdict that faithfully aggregates all of them.

(D2) **Classification.** A refusal must land in a *finite, shared, legible* vocabulary, so that a recipient who never saw the sender's interior can nonetheless act on the *kind* of refusal.

(D3) **Totality.** No refusal may fall through a crack: every failure mode must map to a class, and the mapping must be checkable by machine, not by hope.

(D4) **Reason-carrying.** A refusal must carry *why*; at the logical layer this sharpens to carrying a *proof*.

Composition (D1) wants a monoid. Classification (D2) wants a surjection onto a small set. Totality (D3) wants a mechanized exhaustiveness argument. Reason-carrying (D4) wants, at its limit, proof-carrying evaluation. This paper supplies each in turn — Chapters **??** and **??** for composition, Chapter **??** for classification and totality, and Chapter **??** for proof-carrying refusal — and anchors each to running code.

1.2 First principle

Axiom 1.1 (Refusal is data)**.** There is a distinguished refusal value $\perp$ and a space of *reasons* $\mathcal{R}_f$. A refusal is not the absence of an output but the pair $(\perp, r)$ with $r \in \mathcal{R}_f$ a machine-checkable reason. Admission is therefore *total* as a map into $\mathcal{O}^* \cup (\{\perp\} \times \mathcal{R}_f)$: a correctly functioning admission always returns, either an admitted observation or a refusal-with-reason.

Axiom **??** is the design decision made concrete in `praxis` by the `Andon` type, whose `Halted` variant carries both the unmet obligations and their refusal-taxonomy classification:

`Andon::Halted`{ `unmet: Vec<Obligation>, refusals: Vec<RefusalScenario>, at: u64` }.

The remainder of this paper is the study of the structure that $\mathcal{R}_f$ carries. We fix, once, the code objects under study: the `DenialPolarity` word and its `compose` operation (`crates/praxis-core`, re-exported from `bcinr-powl-receipt`); the `Obligation` enum and the `Judge`/`Admit` traits (`law.rs`); the `RefusalScenario`/`RefusalCategory` taxonomy and its total maps (`refusal.rs`); and the `prolog8` `Kernel::query` logic gate (`ops.rs`). Each theorem below names the artifact it governs.

Chapter 2

The Denial Semilattice

2.1 The abstract denial word

Definition 2.1 (Denial word space)**.** Fix $n \in \mathbb{N}$ *lanes.* The *abstract denial word space* is $\mathbb{D}_n = \{0,1\}^n$ with componentwise disjunction $\vee$ and the all-zero word $\mathbf{0}$. A lane i is *set* in d when $d_i = 1$; the *support* $\operatorname{supp} d = \{i : d_i = 1\}$ names the fired lanes. Write $d \preceq d'$ for the componentwise order $\forall i\,(d_i \leq d'_i)$.

Proposition 2.1 ($\mathbb{D}_n$ is a bounded idempotent commutative monoid)**.** $(\mathbb{D}_n, \vee, \mathbf{0})$ *is a commutative monoid in which every element is idempotent ($d \vee d = d$). It is the join-semilattice of the Boolean lattice $(2^{[n]}, \subseteq)$ under the support isomorphism $d \mapsto \operatorname{supp} d$; $\mathbf{0}$ is its least element (bottom), and the all-ones word $\mathbf{1}$ its greatest. Consequently $d \vee d'$ is the least upper bound of d, d', and $\preceq$ is exactly the algebraic order $d \preceq d' \iff d \vee d' = d'$.*

Proof. Disjunction is associative, commutative, and idempotent per coordinate, with unit 0 per coordinate; a finite product of such structures inherits all four laws, giving a bounded commutative idempotent monoid. The map $d \mapsto \operatorname{supp} d$ carries $\vee$ to $\cup$ and $\mathbf{0}$ to $\varnothing$, and is a bijection $\mathbb{D}_n \to 2^{[n]}$, hence a monoid isomorphism onto $(2^{[n]}, \cup, \varnothing)$. In any join-semilattice the identity $d \preceq d' \iff d \vee d' = d'$ holds by antisymmetry of $\subseteq$; least/greatest elements are $\varnothing$ and $[n]$, i.e. $\mathbf{0}$ and $\mathbf{1}$ [**?**, Ch. I]. $\square$

Proposition **??** is the algebra behind "a refusal category is a join-support" in the keystone: $\operatorname{supp} d(o) \subseteq [n]$ is a subset, and subsets join by union. We now show that the `praxis` implementation realizes exactly this structure.

2.2 The concrete word: `DenialPolarity`

Definition 2.2 (The polarity word)**.** In `praxis` the denial word is `DenialPolarity`, a `#[repr(transparent)]` newtype over `u64` carved into eight byte-lanes. The clean word is $\mathsf{ADMITTED} =$ `DenialPolarity(0)`. Seven named nonzero constants each occupy one distinct byte lane:

`PRECONDITION_FAILED`	(lane 1)	`RESOURCE_EXHAUSTED`	(lane 4)
`SLA_BREACH`	(lane 2)	`OBJECT_LIFECYCLE_VIOLATION`	(lane 5)
`AUTHORIZATION_DENIED`	(lane 3)	`CONFORMANCE_GATE_FAILED`	(lane 6)
		`WATCHDOG_DRAINED`	(lane 7).

The product is $\texttt{compose}(a, b) = \texttt{DenialPolarity}(a.0 \mid b.0)$, bitwise OR; the admission predicate is $\texttt{is\_admitted}(d) \iff d.0 = 0$.

Proposition 2.2 (The polarity word is the denial semilattice)**.** *Let* $\mathbb{D} = (\{\textit{DenialPolarity}\}, \textit{compose}, \mathsf{ADMITTED}$
Then $\mathbb{D}$ *is a bounded commutative idempotent monoid, and* ***is_admitted*** *is exactly the predicate "is the bottom element." The submonoid* $\langle L\rangle$ *generated by the seven named nonzero constants together with* ADMITTED *is isomorphic to* $\mathbb{D}_7$ *of Definition* **??** *via "which named lanes are nonzero."*

Proof. Bitwise OR on u64 is associative, commutative, idempotent, with identity the zero word; compose inherits these, so $\mathbb{D}$ is a bounded commutative idempotent monoid whose bottom is ADMITTED, and $\texttt{is\_admitted}(d) \iff d.0 = 0 \iff d = \mathsf{ADMITTED}$. Each named constant c_i $(1 \le i \le 7)$ has $c_i.0 = \texttt{0xFF} \ll 8i$ occupying lane i alone, so for any subset $S \subseteq \{1, \ldots, 7\}$ the composite $\bigvee_{i\in S} c_i$ has byte lane i nonzero iff $i \in S$. The map $\bigvee_{i\in S} c_i \mapsto \mathbf{1}_S \in \mathbb{D}_7$ (the indicator of S) is therefore a well-defined bijection $\langle L\rangle \to \mathbb{D}_7$ carrying compose to $\vee$ and ADMITTED to $\mathbf{0}$: a monoid isomorphism. □

Remark. Definition **??** uses eight byte-lanes but only seven carry a named denial; lane 0 is the region of the clean word and is never set by a denial constant. Thus the *word* has seven informative lanes, while the *category* taxonomy of Chapter **??** has eight buckets, the eighth (Reserved) being an uninhabited slack coordinate. The two "eights" are the byte width, not a claim of eight active denials; we are careful never to conflate them.

2.3 The fired-mask projection

The keystone's agent byte and its planetary admission sweep require collapsing a wide, possibly multi-lane word onto a fixed-width bit vector without losing *which* lanes fired. This is to_fired_mask.

Definition 2.3 (Fired-mask)**.** $\pi_f : \mathbb{D} \to \{0, 1\}^8$ sends a word d to the bit vector whose bit j equals the nonzero-indicator of byte lane j of d: $\pi_f(d)_j = \mathbf{1}[(d.0 \gg 8j)\ \&\ \texttt{0xFF} \neq 0]$. The praxis realization computes this branchlessly via the identity $\mathbf{1}[x \neq 0] = (x \mid (-x)) \gg 63$ on each lane.

Theorem 2.3 (Fired-mask is a semilattice homomorphism)**.** π_f *is a homomorphism of bounded commutative idempotent monoids:* $\pi_f(\mathsf{ADMITTED}) = \mathbf{0}$ *and, for all* $a, b \in \mathbb{D}$*,*

$$\pi_f\big(\textit{compose}(a, b)\big) = \pi_f(a) \vee \pi_f(b).$$

Restricted to $\langle L\rangle$ *it is a monoid isomorphism onto the seven-bit image* $\{0\} \times \{0, 1\}^7 \subseteq \{0, 1\}^8$.

Proof. $\pi_f(\mathsf{ADMITTED}) = \mathbf{0}$ since every lane of the zero word is zero. Byte lane j of $\texttt{compose}(a, b) = a.0 \mid b.0$ is $a_j \mid b_j$ (bitwise OR acts lane-locally), and for byte values $\mathbf{1}[a_j \mid b_j \neq 0] = \mathbf{1}[a_j \neq 0] \vee \mathbf{1}[b_j \neq 0]$ because a disjunction of bytes is nonzero iff some operand is; hence $\pi_f(a \mid b)_j = \pi_f(a)_j \vee \pi_f(b)_j$ for every j, which is the claimed homomorphism law. On $\langle L\rangle$, Proposition **??** identifies elements with subsets $S \subseteq \{1, \ldots, 7\}$, and π_f sends such an element to the indicator of S in bits $1 \ldots 7$ (bit 0 always zero); this is a bijection onto $\{0\} \times \{0, 1\}^7$ preserving join and bottom. □

Theorem **??** is the projection principle at the algebra layer: an arbitrarily composed denial word — the aggregate of an entire pipeline (Chapter **??**) — projects losslessly, as far as *which lanes fired*, onto at most eight bits. This is the homomorphism that makes the keystone's word-parallel fleet-admission sweep sound (Proposition **??**).

Chapter 3

Obligations as Lane Maps

3.1 Obligations and total denial

Definition 3.1 (Obligation and lane map)**.** An *obligation* is a first-class, hashable value the payload must satisfy before admission. In `praxis` the `Obligation` enum has exactly three kinds: `Precondition{predicate_id, params_hash}`, `BlockingConstraint{reason}`, and `EvidenceRequired{evidence_type}`. Each obligation g induces a *lane map* $\delta_g : \mathcal{O} \to \mathbb{D}$ with

$$\delta_g(o) = \begin{cases} \mathsf{ADMITTED} & \text{if } g \text{ is satisfied by } o, \\ \ell(g) & \text{otherwise,} \end{cases}$$

where $\ell(g) \in \mathbb{D}$ is the lane g fires on failure. For an obligation *set* $G = \{g_1, \dots, g_m\}$ the *total denial* is the join

$$d_G(o) \;=\; \texttt{compose\_denials}\big(\{\delta_{g_i}(o) : g_i \text{ unmet}\}\big) \;=\; \bigvee_{i=1}^{m} \delta_{g_i}(o).$$

The right-hand fold is literally `compose_denials` in `refusal.rs`: it maps each refusal scenario to its lane via `denial_lane` and folds with `compose` from the seed $\mathsf{ADMITTED}$. Because $\mathsf{ADMITTED}$ is the monoid identity (Proposition **??**), an empty set of unmet obligations composes to $\mathsf{ADMITTED}$ — the admitted word — which is exactly the documented behavior ("empty input composes to `ADMITTED`").

Proposition 3.1 (Bottom characterizes admission)**.** *$\alpha_G(o) \neq \bot \iff d_G(o) = \mathsf{ADMITTED} \iff$ every $g_i \in G$ is satisfied by o.*

Proof. $d_G(o) = \bigvee_i \delta_{g_i}(o) = \mathsf{ADMITTED}$ iff every summand is $\mathsf{ADMITTED}$ (a join of elements of a join-semilattice equals the bottom iff each element is the bottom, by Proposition **??**), i.e. iff no obligation is unmet. Admission returns non-refusal exactly in that case (Axiom **??**). □

3.2 Monotonicity of admission

Theorem 3.2 (Admission is antitone in the obligation set)**.** *Let $G \subseteq G'$ be obligation sets. Then for every observation o,*

$$d_G(o) \;\preceq\; d_{G'}(o), \qquad \textit{hence} \qquad \mathcal{O}^*_{G'} \;\subseteq\; \mathcal{O}^*_{G},$$

where $\mathcal{O}_G^* = \{o : d_G(o) = \mathsf{ADMITTED}\}$ *is the admitted set under* G*. Adding obligations can only enlarge denial and shrink admission; it can never admit an observation a smaller obligation set refused.*

Proof. $d_{G'}(o) = \bigvee_{g\in G'} \delta_g(o) = \left(\bigvee_{g\in G} \delta_g(o)\right) \vee \left(\bigvee_{g\in G'\setminus G} \delta_g(o)\right) = d_G(o) \vee d_{G'\setminus G}(o) \succeq d_G(o)$, since in a join-semilattice $x \preceq x \vee y$ always (Proposition ??). If $o \in \mathcal{O}_{G'}^*$ then $d_{G'}(o) = \mathsf{ADMITTED}$, the bottom; from $d_G(o) \preceq d_{G'}(o) = \mathsf{ADMITTED}$ and minimality of $\mathsf{ADMITTED}$ we get $d_G(o) = \mathsf{ADMITTED}$, i.e. $o \in \mathcal{O}_G^*$. □

Corollary 3.3 (Admission is an antitone map of lattices)**.** *The assignment* $G \mapsto \mathcal{O}_G^*$ *is an order-reversing map from the lattice of obligation sets* $(2^{\mathcal{G}}, \subseteq)$ *to the lattice of admitted subsets* $(2^{\mathcal{O}}, \subseteq)$*:* $G \subseteq G' \Rightarrow \mathcal{O}_{G'}^* \subseteq \mathcal{O}_G^*$*. In particular the empty obligation set admits everything* ($\mathcal{O}_\varnothing^* = \mathcal{O}$)*, and enlarging obligations monotonically tightens the gate.*

Proof. Order-reversal is Theorem ??. With $G = \varnothing$ the join is empty, so $d_\varnothing(o) = \mathsf{ADMITTED}$ for all o and $\mathcal{O}_\varnothing^* = \mathcal{O}$. □

3.3 The obligation-to-scenario map is total

Proposition 3.4 (Obligation classification is compiler-certified total)**.** *The map* scn : `Obligation` $\to$ $\mathcal{S}$ *implemented by* `From<&Obligation> for RefusalScenario` *is a total function realized by a wildcard-free* `match` *over the three* `Obligation` *kinds:*

$$\mathtt{Precondition} \mapsto \mathtt{UnsatisfiedPrecondition}, \quad \mathtt{BlockingConstraint} \mapsto \mathtt{BlockingConstraint}, \quad \mathtt{EvidenceRe}$$

Because the `match` *has no wildcard arm, adding a fourth* `Obligation` *kind without extending* scn *is a compile error; totality is therefore a static property of the program, not a runtime hope.*

Proof. A Rust `match` on a closed enum type-checks only if every constructor is covered or a wildcard is present; the cited `match` covers all three constructors and uses no wildcard, so the compiler certifies exhaustiveness (this is the mechanized-totality mechanism formalized in Paper 00). Each arm returns a specific `RefusalScenario`, so scn is a total function. □

Proposition ?? is the first half of the totality argument: every obligation *failure* becomes a scenario. The second half — every scenario becomes a category — is Chapter ??.

Chapter 4

The Eight-Bucket Taxonomy as a Surjection

4.1 Categories and scenarios

Definition 4.1 (The taxonomy)**.** The *category set* is the eight-element enum

$\mathcal{C} = \{\texttt{Identity}, \texttt{Capacity}, \texttt{Topology}, \texttt{Temporal}, \texttt{Lifecycle}, \texttt{Authorization}, \texttt{Prerequisites}, \texttt{Reserved}\}.$

The *scenario set* $\mathcal{S}$ is the thirteen-variant `RefusalScenario` enum, partitioned by origin into: three *obligation-driven* variants (`BlockingConstraint`, `MissingEvidence`, `UnsatisfiedPrecondition`); seven *denial-lane* variants, one per named nonzero lane of Definition **??** (`WatchdogDrained`, `PreconditionFailed`, `SlaBreach`, `AuthorizationDenied`, `ResourceExhausted`, `ObjectLifecycleViolation`, `ConformanceGateFailed`); and three *logic/andon* variants (`KernelDenied`, `KernelInvalid`, `AndonInvariantViolated`). The *category map* $\mathrm{cat} : \mathcal{S} \to \mathcal{C}$ is `RefusalScenario::category`.

Scenario	cat(·)	origin
`KernelInvalid`	`Identity`	logic
`ResourceExhausted`	`Capacity`	denial lane
`ConformanceGateFailed`	`Topology`	denial lane
`AndonInvariantViolated`	`Topology`	andon
`WatchdogDrained`, `SlaBreach`	`Temporal`	denial lanes
`BlockingConstraint`, `ObjectLifecycleViolation`	`Lifecycle`	obligation / lane
`AuthorizationDenied`, `KernelDenied`	`Authorization`	lane / logic
`MissingEvidence`, `UnsatisfiedPrecondition`, `PreconditionFailed`	`Prerequisites`	obligation / lane
(none)	`Reserved`	—

4.2 The totality theorem

Theorem 4.1 (Mechanized totality of classification)**.** *The category map* $\mathrm{cat} : \mathcal{S} \to \mathcal{C}$ *satisfies:*

(i) ***Totality.*** cat *is a total function: every one of the thirteen scenarios has exactly one category.*

(ii) ***Mechanization.*** cat *is realized by a wildcard-free* `match`*, so its totality is compiler-certified: adding a scenario variant without extending* cat *fails to compile. (The* `praxis` *test suite additionally pins each variant's value.)*

(iii) ***Image.*** $\operatorname{im}\mathrm{cat} = \mathcal{C}\setminus\{\texttt{Reserved}\}$*: exactly the seven buckets* $\{\texttt{Identity}, \texttt{Capacity}, \texttt{Topology}, \texttt{Temporal}, \texttt{Lif}$ *are inhabited, and* `Reserved` *has empty preimage.*

Consequently cat *is surjective onto the inhabited sub-taxonomy but is* not *surjective onto* $\mathcal{C}$*; the deficiency* $\{\texttt{Reserved}\}$ *is exactly one bucket.*

Proof. (i)–(ii) A Rust `match` on the closed `RefusalScenario` enum type-checks only under exhaustiveness; the `category` method matches all thirteen constructors with no wildcard arm, so the compiler certifies that every scenario is assigned, and each arm returns a single `RefusalCategory`, making cat total and single-valued. (iii) Reading the thirteen arms (Definition **??** table) exhibits a preimage for each of the seven listed buckets — e.g. $\texttt{KernelInvalid} \in \mathrm{cat}^{-1}(\texttt{Identity})$, $\texttt{ResourceExhausted} \in \mathrm{cat}^{-1}(\texttt{Capacity})$, and so on down the table — so all seven are in the image; and no arm returns `Reserved`, so $\mathrm{cat}^{-1}(\texttt{Reserved}) = \varnothing$. Hence $\operatorname{im}\mathrm{cat}$ is precisely the seven-element complement of $\{\texttt{Reserved}\}$. □

Remark (Honesty of the reserved bucket). Theorem **??**(iii) is deliberately *not* a claim of 8/8 surjectivity. The codomain names a bucket the mechanism does not yet inhabit, and the mathematics records that faithfully rather than inflating coverage. This is the doctrine's antibody clause at the type level: a reserved coordinate is declared, unused, and provably unused, so no reader can be misled into thinking eight kinds of refusal are in force when seven are. When a future enforcement lane populates `Reserved`, the theorem's image statement is what must be re-verified.

4.3 Where the type guarantee ends: the partial lane decoder

Not every total map in this taxonomy is a wildcard-free `match`, and the exception is instructive: it marks the precise boundary between what the type system certifies and what a runtime test must certify.

Definition 4.2 (Single-lane set and the lane decoder)**.** Let $L = \{\mathsf{ADMITTED}, c_1, \ldots, c_7\} \subseteq \mathbb{D}$ be the clean word together with the seven named single-lane constants. The *lane decoder* $\mathrm{scn}_\ell : \mathbb{D} \rightharpoonup \mathcal{S}$ is `scenario_for_denial_lane`: it returns `None` on $\mathsf{ADMITTED}$, returns the matching denial-lane scenario on each c_i, and returns `None` on every word not in L.

Proposition 4.2 (The lane decoder cannot be a total `match`, and why)**.** scn_ℓ *is a partial map, definite (as* `Some`*) only on* $L \setminus \{\mathsf{ADMITTED}\}$*. It is* not *realizable as a wildcard-free exhaustive* `match`*, because its domain* $\mathbb{D}$ *is a* `#[repr(transparent)]` *newtype over* `u64` *— an open value space of cardinality* 2^{64}*, not a closed enum — so the compiler cannot enumerate its cases. In particular any composed multi-lane word* $d = c_i \vee c_j \notin L$ *(a legitimate element produced by* `compose`*) has* $\mathrm{scn}_\ell(d) = \texttt{None}$*: the decoder recognizes single lanes only. Exhaustiveness over the seven single lanes is therefore a* runtime test obligation *(discharged by the* `category_mapping_is_total_over_denial_lanes` *test), not a static type obligation.*

Proof. Rust's exhaustiveness checker operates on closed sum types; a newtype over `u64` exposes 2^{64} inhabitants with no finite constructor set, so no wildcard-free `match` on its value is well-typed — the implementation is necessarily an `if/else` chain over the eight known constants with a `None` fallback. On $\mathsf{ADMITTED}$ it returns `None` by the first guard; on each c_i it returns the corresponding `Some` scenario; on any other word (including every $c_i \vee c_j$ with $i \neq j$, which lies outside L) it falls through to `None`. Thus $\mathrm{dom}^+ \mathrm{scn}_\ell = L \setminus \{\mathsf{ADMITTED}\}$ where dom^+ denotes the `Some`-domain. □

Proposition 4.3 (The lane encoder is total and a section)**.** *The* lane encoder lane : $\mathcal{S} \to \mathbb{D}$ *implemented by* ***`denial_lane`*** *is a total function realized by a wildcard-free* **`match`** *over all thirteen scenarios. Restricted to the seven denial-lane scenarios $\mathcal{S}_\ell$, it and* scn_ℓ *form a section–retraction pair:*

$$\mathrm{scn}_\ell \circ \mathrm{lane} = \mathrm{id}_{\mathcal{S}_\ell} \quad \textit{and} \quad \mathrm{lane} \circ \mathrm{scn}_\ell = \mathrm{id}_{L \setminus \{\mathsf{ADMITTED}\}},$$

so the seven single lanes and the seven denial-lane scenarios are in bijection. The remaining six scenarios (obligation-driven and logic/andon) are mapped by lane *onto* closest-fit *lanes and are therefore not in the range of that bijection.*

Proof. lane matches all thirteen constructors with no wildcard, hence is total (same mechanism as Theorem **??**). For each of the seven denial-lane scenarios s, $\mathrm{lane}(s) = c_i$ is its own lane, and $\mathrm{scn}_\ell(c_i) = s$ by Definition **??**, giving $\mathrm{scn}_\ell(\mathrm{lane}(s)) = s$; conversely $\mathrm{lane}(\mathrm{scn}_\ell(c_i)) = \mathrm{lane}(s) = c_i$. Both round-trips are the identity on their respective seven-element sets, the round-trip pinned by the cited test. The six non-lane scenarios map by lane onto lanes already claimed by the seven (e.g. `MissingEvidence`$\mapsto$ `PRECONDITION_FAILED`), so lane is not injective overall and those six are outside the bijection. □

Corollary 4.4 (The assembled totality statement)**.** *Composing the total maps of Propositions* **??** *and* **??** *with the total classification of Theorem* **??***:* every unmet obligation *(via* $\mathrm{cat} \circ \mathrm{scn}$*) and* every non-$\mathsf{ADMITTED}$ single denial lane *(via* $\mathrm{cat} \circ \mathrm{scn}_\ell$ *on* $L \setminus \{\mathsf{ADMITTED}\}$*) maps to exactly one of the seven inhabited categories. No obligation failure and no fired single lane is left unclassified.*

Proof. Immediate by composition of the three total (respectively `Some`-total) maps, each single-valued into its codomain, landing in $\mathrm{im}\,\mathrm{cat} = \mathcal{C} \setminus \{\texttt{Reserved}\}$ (Theorem **??**). □

Corollary **??** is precisely the totality claim demanded of a fleet vocabulary (D3): a refusal never falls through a crack, and its class is legible to any recipient who holds the eight-element enum, regardless of the sender's interior.

Chapter 5

Composition Across a Pipeline as a Monoid Action

5.1 Pipelines and their aggregate denial

A manufacture rarely faces a single gate. It traverses a *pipeline*: `Judge` evaluates obligations; `Admit` may deny; a `prolog8` kernel gate may refuse a logical side-condition (Chapter **??**); an optional andon second-gate ring may fire. Each stage emits a denial word; the fleet needs one.

Definition 5.1 (Pipeline and stage denial)**.** Let Stage be the set of admission stages. Fix an observation o and let $\varphi_o : \mathsf{Stage} \to \mathbb{D}$ send a stage to the denial word it emits on o (via its scenarios and `denial_lane`). A *pipeline* is a finite sequence $w = s_1 s_2 \cdots s_k \in \mathsf{Stage}^*$, an element of the free monoid on Stage under concatenation with unit the empty sequence ε. Its *aggregate denial* is

$$\Phi_o(w) \;=\; \texttt{compose\_denials}\big(\varphi_o(s_1), \ldots, \varphi_o(s_k)\big) \;=\; \bigvee_{i=1}^{k} \varphi_o(s_i), \qquad \Phi_o(\varepsilon) = \mathsf{ADMITTED}.$$

Theorem 5.1 (Pipeline denial is the free-monoid homomorphism)**.** $\Phi_o : (\mathsf{Stage}^*, \cdot, \varepsilon) \to (\mathbb{D}, \texttt{compose}, \mathsf{ADMITTED})$ *is the unique monoid homomorphism extending* φ_o *along the inclusion* $\mathsf{Stage} \hookrightarrow \mathsf{Stage}^*$*. Because the target is commutative and idempotent,* Φ_o *factors through the finite join-semilattice generated by* $\{\varphi_o(s) : s \in \mathsf{Stage}\}$*; hence* $\Phi_o(w)$ *depends only on the* set *of distinct stage-denials occurring in* w *— it is invariant under reordering the stages and under repeating them.*

Proof. $\Phi_o(\varepsilon) = \mathsf{ADMITTED}$ and $\Phi_o(w \cdot w') = \bigvee_{s \in w} \varphi_o(s) \vee \bigvee_{s \in w'} \varphi_o(s) = \Phi_o(w) \vee \Phi_o(w') = \texttt{compose}(\Phi_o(w), \Phi_o(w'))$ by associativity of $\vee$; so Φ_o is a monoid homomorphism, and it agrees with φ_o on length-one words. Uniqueness is the universal property of the free monoid: any homomorphism agreeing with φ_o on generators is determined on all of Stage^* by the homomorphism laws. For the factorization, commutativity gives order-independence ($a \vee b = b \vee a$) and idempotence gives multiplicity-independence ($a \vee a = a$), so $\Phi_o(w) = \bigvee_{s \in \mathrm{set}(w)} \varphi_o(s)$ depends only on the underlying set of distinct denials. $\square$

Corollary 5.2 (A pipeline admits iff every stage admits)**.** $\Phi_o(w) = \mathsf{ADMITTED} \iff \varphi_o(s_i) = \mathsf{ADMITTED}$ *for every stage* s_i *in* w*. A single refusing stage refuses the whole pipeline, and the aggregate word records* every *lane that fired anywhere along* w*.*

Proof. A join in a join-semilattice equals the bottom iff every joinand is the bottom (Proposition **??**); apply to $\Phi_o(w) = \bigvee_i \varphi_o(s_i)$. The support of the aggregate is $\operatorname{supp}\Phi_o(w) = \bigcup_i \operatorname{supp}\varphi_o(s_i)$, the union of the per-stage fired lanes. □

Corollary **??** is the correctness statement for `ReceiptMeta.denial`: when a manufacture is receipted, the honest denial word written into the `OcelCausalFrame` is the aggregate $\Phi_o(w)$, so the receipt records exactly what was waved through — not a fiction of clean admission when a non-fatal lane fired. Composition (D1) is thereby a theorem, not a convention.

5.2 The fleet sweep is word-parallel

Proposition 5.3 (Byte-packed fleet admission)**.** *Let a fleet of N manufactures produce aggregate words $\Phi^{(1)}, \ldots, \Phi^{(N)}$, and pack each into a byte $b^{(r)} = \pi_f(\Phi^{(r)}) \in \{0,1\}^8$. Then:*

(i) $\pi_f(\Phi_o(w)) = \bigvee_i \pi_f(\varphi_o(s_i))$*: the fleet byte of a pipeline equals the bitwise* OR *of its stage bytes (fired-mask commutes with aggregation).*

(ii) A fleet admission sweep — "are all agents clean?" — is one linear pass computing $\bigvee_r b^{(r)}$, eight agents per 64*-bit word, and returns* **0** *iff every agent is admitted.*

Proof. (i) π_f is a homomorphism (Theorem **??**) and Φ_o is a homomorphism (Theorem **??**); their composite carries `compose` to $\vee$, so $\pi_f(\bigvee_i \varphi_o(s_i)) = \bigvee_i \pi_f(\varphi_o(s_i))$. (ii) The pass folds $\vee$ over N bytes; by Corollary **??** applied fleet-wide, the fold is **0** iff each $b^{(r)} = \mathbf{0}$ iff each aggregate is ADMITTED iff every agent is admitted. Packing eight bytes per machine word makes the fold word-parallel. □

Proposition **??** is the algebraic underpinning of the keystone's "agent as eight bits" construction and its memory-bandwidth-bounded planetary sweep: the reason a fleet of 10^{12} agents can be admission-swept by masked ORs is that both aggregation (over stages) and projection (to bits) are semilattice homomorphisms, so the cheap bit-level operation computes the same verdict as the expensive per-stage evaluation.

Chapter 6

Proof-Carrying Admission via SLD-Resolution

6.1 The logical quarantine

Some admissions are not Boolean batteries but *logical queries*: "does this atom follow from the admitted facts and rules?" `praxis` routes these to the `prolog8` kernel (`run_kernel_query` in `ops.rs`). Admission here is derivation, and — unlike a bare Boolean gate — it can carry a *proof*. First the language must be quarantined.

Definition 6.1 (Logic admission; the Rice quarantine for Horn logic)**.** `prolog8` admits a query, fact block, or rule only if it lies in a bounded, decidable Horn fragment: arity ≤ 8, rule body ≤ 8 atoms, ≤ 8 variables, proof fan-in ≤ 8; no cut, no dynamic mutation (`assert`/`retract`), stratified negation, bounded or declared recursion, no runtime text parsing, no side effects, interned (non-string) terms. Violations are enumerated by `RejectionCode` (a closed enum of $\sim$30 structured codes: `ArityCapExceeded`, `RuleBodyCapExceeded`, `CutNotAdmitted`, `UnstratifiedNegation`, `DynamicMutationNotAdmitted`, ...). `admit_atom` and `admit_rule` run before any query touches the kernel.

Definition **??** is Rice's theorem [**?**] answered at the logic layer: full first-order or unrestricted Prolog derivability is undecidable, so the kernel retracts onto a fragment whose byte caps make the search space finite, and everything outside the fragment is refused with a code. This is the same move as $\alpha : \mathcal{O} \to \mathcal{O}^* \cup \{\perp\}$ (Paper 00), specialized to logic and carrying, additionally, a proof on the inside.

6.2 The proof-carrying trichotomy

Definition 6.2 (Query result)**.** `Kernel::query` returns `QueryResult`, one of: `Answered(Vec<Decision>)`, `Denied(Box<Decision>)`, or `Invalid(RejectionCode)`. A `Decision` carries a *proof* (a `Vec<ProofNode>` DAG, positive or negative) and a *receipt* sufficient for deterministic replay.

Theorem 6.1 (Proof-carrying admission trichotomy)**.** *For a query q evaluated against an admitted kernel (admitted catalog, fact blocks, and rules), exactly one of the following holds:*

(a) ***Answered.*** *q passes* `admit_atom` *and unifies with a stored fact or is derivable by a depth-one rule application; then* `query`(q) = `Answered`$(D_1, \dots, D_p)$ *with $p \geq 1$, each D_j carrying a* positive *proof DAG whose every node has fan-in ≤ 8, plus a receipt.*

(b) ***Denied.*** *q passes* `admit_atom` *but no fact and no admitted rule derive it; then* `query`(q) = `Denied`(D)*, where D carries a* negative *proof — a finite, closed witness that the depth-bounded SLD search was exhausted without success — plus a receipt.*

(c) ***Invalid.*** *q fails* `admit_atom`*; then* `query`(q) = `Invalid`(c) *with $c \in$* `RejectionCode`*, and* no proof is produced*.*

Thus a `Denied` *query yields a proof tree; an* `Invalid` *query yields a rejection code.*

Proof. Trace `Kernel::query`. Its first action is the admission gate: if `admit_atom`(q) returns `Err`(c) the kernel returns `Invalid`(c) immediately, before any search, establishing (c) and its no-proof clause. Otherwise the query is in the admitted fragment and the kernel runs three phases. *Phase 1* scans fact blocks for rows unifying with q under its bound positions; a nonempty match set is assembled into `Answered` decisions, each with a positive proof rooted at the matched fact hash. *Phase 2*, on empty Phase 1, applies each rule whose head unifies with q by depth-one backward chaining with backtracking over body atoms (a single SLD-resolution step against the fact base); any successful derivation is assembled into `Answered`, giving (a). The fan-in bound holds because `admit_rule` rejects any rule or proof node exceeding fan-in 8 (`ProofFanInExceeded`), so every emitted proof node has ≤ 8 children. *Phase 3* is reached iff both searches are empty; the kernel then assembles a `Denied` decision with a *negative* proof recording the exhausted search, giving (b). The three phases are mutually exclusive by construction (each returns), so exactly one branch fires. Soundness and completeness of the depth-bounded resolution — that Phase 3 is reached *only* when q is underivable within the fragment — is SLD soundness and completeness for definite/stratified Horn programs [**?**, **?**], restricted to the finite search space that the byte caps of Definition **??** guarantee. ☐

Proposition 6.2 (Boundedness gives a bounded certificate)**.** *Every proof node emitted by the kernel has fan-in ≤ 8 and every rule body has ≤ 8 atoms, so a proof (positive or negative) is a bounded-degree DAG; its rolling* `proof_root` *is a single fixed-width hash field of the receipt. A bounded-context verifier checks a query outcome by recomputing* `proof_root` *and comparing — cost $O(\kappa)$, independent of the size of the fact base — and the deterministic replay path recomputes exactly this root.*

Proof. Fan-in and body caps are enforced by `admit_rule` (`ProofFanInExceeded`, `RuleBodyCapExceeded`), so proof nodes have bounded out-degree; the receipt commits the proof by its root hash (a projection of the DAG onto one field), and replay recomputes the root from the stored decision, agreeing iff untampered — the Faithful Projection mechanism of the keystone applied to the proof DAG. The verifier reads the fixed-width root, not the DAG, so its cost is $O(\kappa)$. ☐

Proposition **??** is the projection principle at the logic layer: an arbitrarily large search over facts and rules projects onto one root hash a bounded verifier can check, so proof-carrying admission scales the same way byte-level admission does.

6.3 Logic refusals rejoin the monoid

The logical layer does not live apart from Chapters ??–??; its refusals are ordinary denial words.

Proposition 6.3 (Kernel outcomes embed into the denial monoid)**.** *The refusing branches of Theorem* ?? *embed into* ($\mathbb{D}$, `compose`, ADMITTED) *via the taxonomy:* `run_kernel_query` *maps a* `Denied` *outcome to the scenario* `KernelDenied` *(category* `Authorization`*) and an* `Invalid` *outcome to* `KernelInvalid` *(category* `Identity`*); by* `denial_lane` *both compose into the lane* `CONFORMANCE_GATE_FAILED`*. Hence a logical refusal is an element of* $\mathbb{D}$ *subject to monotonicity (Theorem* ??*), classification (Theorem* ??*), pipeline composition (Theorem* ??*), and fired-mask projection (Theorem* ??*), exactly as any obligation refusal.*

Proof. `run_kernel_query` constructs `RefusalScenario::KernelDenied` on the `Denied` branch and `RefusalScenario::KernelInvalid` on the `Invalid` branch (Definition ??); their categories are as stated by Theorem ??, and `denial_lane` sends both to `CONFORMANCE_GATE_FAILED` (Proposition ??, closest-fit clause). Being elements of $\mathbb{D}$, they are acted on by all the structure of Chapters ??–??. □

Remark (The epistemic distinction, made precise)*.* The trichotomy separates three epistemic states. `Invalid` means *outside the language*: a reason (a `RejectionCode`) but no proof — the query was never well-formed enough to have a truth value in the fragment. `Denied` means *in the language and false*: a negative proof witnesses underivability. `Answered` means *in the language and true*: a positive proof witnesses derivability. Only membership-in-the-language is a type-like gate (`admit_atom`); truth or falsity within the language is decided by bounded search and *always* ships a witness. This is the sharpest form of “refusal is data”: at the logic layer even a denial carries its proof, so a bounded verifier can trust the “no” without re-running the search.

Chapter 7

The Boundary Observation Algebra and the Domain Proposer

7.1 Untrusted observations and the boundary

A subtle consequence of the admission algebra is that observations arriving from outside a system boundary carry no authority by default. The `praxis-proposer` crate formalizes this by distinguishing two regimes.

Definition 7.1 (Observation authority)**.** Let $\mathcal{O}$ be the raw observation space (Axiom **??**). An observation $o \in \mathcal{O}$ is *authoritative* ($o \in \mathcal{O}^*$) if it was produced by an admitted actuation with a chained receipt — equivalently, if it carries a committed payload digest inside an `OcelCausalFrame`. All other observations are *untrusted* ($o \in \mathcal{O} \setminus \mathcal{O}^*$). The proposer's admission map α_{prop} retracts $\mathcal{O}$ onto $\mathcal{O}^*_{\text{prop}}$ by treating every inbound observation as untrusted until its receipt chain satisfies the obligation battery G_{prop}.

Proposition 7.1 (Proposer boundary separation)**.** *Under Definition* **??***, the `praxis-proposer` enforces the invariant* $\operatorname{im}\mu_{\text{prop}} \subseteq \mathcal{O}^*_{\text{prop}}$*: no proposal is manufactured from an unadmitted observation. Operationally, the generic `Domain<D>` proposer evaluates its `Obligation` battery on every inbound record before calling `Admit::admit`; a record that fails any obligation is refused with its denial word and never reaches the manufacturing step.*

Proof. The `Domain` struct holds a `Law<Obligation>` attached to its typed domain `D`. Proposing calls `judge` then `admit` in sequence (the lifecycle arrows j, a of the foundations paper). An `Andon::Halted` on either step returns the `Err` branch with its denial word; the manufacturing step (`D::manufacture`) is unreachable from the `Err` branch. Hence $\operatorname{im}\mu_{\text{prop}}$ is contained in `Admitted` objects, elements of $\mathcal{O}^*_{\text{prop}}$. □

7.2 Maximum Reachable Revenue: additive separation over accounts

Definition 7.2 (Maximum Reachable Revenue)**.** Let A be a set of client accounts. For each $a \in A$, let $\text{lawful\_targets}(a)$ be the evidence-gated target stages for account a, and let realized(a, s) be the revenue function under target stage s. The *Maximum Reachable Revenue* (MRR) of the fleet is

$$\text{MRR} \;=\; \max_{\text{plan}} \sum_{a \in A} \text{realized}(a) \;.$$

Theorem 7.2 (MRR additively separates over independent accounts)**.** *If the accounts in A are independent — no account's target stages share resources or evidence with another's — then*

$$\text{MRR} \;=\; \sum_{a \in A} \max_{s \in \text{lawful\_targets}(a)} \text{realized}(a, s) \;,$$

reducing the joint plan search from exponential ($\prod_a |\text{lawful\_targets}(a)|$) to linear ($\sum_a |\text{lawful\_targets}(a)|$).

Proof. Independence means the feasibility of account a's targets is unaffected by the targets of any $b \neq a$. Hence the joint optimization $\max_{\text{joint}} \sum_a r(a)$ decomposes into $\sum_a \max_{s_a} r(a, s_a)$ by linearity of the sum and independence of the maxima. In `praxis-proposer` this is realized in `mrr.rs`: `compute_mrr` iterates over accounts, calls `max_realized_revenue(a)` for each, and sums the per-account maxima. No cross-account joint optimization is performed. □

Corollary 7.3 (Boundary independence is the MRR precondition)**.** *Theorem* **??** *holds exactly when no obligation in G_{prop} couples distinct accounts (e.g. a shared authority token or a mutually exclusive resource). If such coupling exists, the proposer's obligation battery must be extended with a `BlockingConstraint` on the shared resource, which by Theorem* **??** *only tightens the admitted set and preserves the denial algebra.*

Proof. Immediate from Theorem **??**: adding an obligation enlarges denial and shrinks admission. With the extended battery the coupled accounts are refused by the coupling constraint and the remaining independent ones decompose additively as before. □

Remark (The generic `Domain<D>` proposer)*.* The `Domain` struct in `praxis-proposer` is a generic adapter parameterized by `DomainSpec`, a sealed trait that supplies the domain's typed observation schema and `manufacture`. Any domain (revenue, scheduling, compliance) inherits the full denial algebra of Chapters **??**–**??** for free: its refusals compose under `DenialPolarity`, are classified by `RefusalCategory`, and are swept byte-parallel across a fleet. The MRR theorem (Theorem **??**) is the domain-neutral consequence of this inheritance: account-level independence is a structural claim about the obligation battery, independent of what the domain manufactures.

Chapter 8

Conclusion

We set out to give the refusal object the algebra its role in a trillion-agent fleet demands, and we have discharged each of the four demands of Chapter ?? as a theorem anchored to `praxis`.

Composition (D1) is Theorem ??: pipeline denial is the unique monoid homomorphism from the free monoid on stages into the bounded commutative idempotent monoid ($\mathbb{D}$, `compose`, $\mathsf{ADMITTED}$) (Propositions ??, ??), so aggregation is associative, order-independent, and multiplicity-independent, and a pipeline admits iff every stage admits (Corollary ??). *Classification (D2)* is the category map cat, and its *totality (D3)* is Theorem ??: a compiler-certified total classification onto exactly seven inhabited buckets plus a provably empty `Reserved` slack coordinate, assembled with the total obligation and lane maps into Corollary ?? — no refusal is unclassified. We located the exact boundary of the type system's guarantee: the lane decoder is partial precisely because its domain is an open 2^{64} newtype, so its exhaustiveness is a test obligation, not a type obligation (Proposition ??). And *reason-carrying (D4)*, at its logical limit, is the proof-carrying trichotomy of Theorem ??: an admitted-but-false query ships a negative proof tree and an ill-formed query ships a rejection code, both bounded (Proposition ??) and both rejoining the denial monoid (Proposition ??).

Monotonicity (Theorem ??) ties the whole together: enlarging obligations only tightens the gate, so the algebra is safe to grow. The fired-mask homomorphism (Theorem ??, Proposition ??) is what lets the keystone's planetary admission sweep compute the true verdict with a bitwise OR. What began, in the keystone, as one Construction and one Proposition is now a complete small theory: the space of "why not" is a join-semilattice, its classification is a mechanized surjection, its aggregation is a free-monoid homomorphism, and its logical instance carries proof.

A refusal is not the absence of an answer; it is an answer with an algebra.

Appendix A

Notation and Code Anchors

Symbols

Symbol	Meaning
$\mathbb{D},\ \vee,\ \mathsf{ADMITTED}$	denial-word monoid; join (`compose`); bottom (`ADMITTED`)
$\mathbb{D}_n,\ \operatorname{supp} d$	abstract n-lane word space; fired-lane support
π_f	fired-mask projection $\mathbb{D} \to \{0,1\}^8$ (`to_fired_mask`)
$\delta_g,\ d_G(o)$	obligation lane map; total denial (join over G)
$\mathcal{O}^*_G$	admitted set under obligation set G
$\mathcal{C},\ \mathcal{S}$	category set (8); scenario set (13)
$\mathrm{cat},\ \mathrm{scn},\ \mathrm{lane},\ \mathrm{scn}_\ell$	category map; obligation→scenario; scenario→lane; lane decoder
$\Phi_o,\ \mathsf{Stage}^*$	pipeline aggregate denial; free monoid on stages
$\perp,\ \kappa$	refusal value; bounded-verifier context capacity

Mathematics to code

Object	Code artifact	Location
$(\mathbb{D}, \vee, \mathsf{ADMITTED})$	`DenialPolarity`, `compose`	`bcinr-powl-receipt/src/denial.rs`
π_f	`DenialPolarity::to_fired_mask`	`bcinr-powl-receipt/src/denial.rs`
d_G, `compose_denials`	`compose_denials`	`crates/praxis-core/src/refusal.rs`
cat (Thm **??**)	`RefusalScenario::category`	`crates/praxis-core/src/refusal.rs`
scn (Prop **??**)	`From<&Obligation>`	`crates/praxis-core/src/refusal.rs`
scn_ℓ (Prop **??**)	`scenario_for_denial_lane`	`crates/praxis-core/src/refusal.rs`
lane (Prop **??**)	`denial_lane`	`crates/praxis-core/src/refusal.rs`
`Obligation` (3 kinds)	`Obligation` enum	`crates/praxis-core/src/law.rs`
`Andon::Halted`	`Andon` enum	`crates/praxis-core/src/law.rs`
Trichotomy (Thm **??**)	`Kernel::query`, `QueryResult`	`prolog8/src/kernel.rs`
Logic quarantine	`admit_atom`, `RejectionCode`	`prolog8/src/admission.rs`
Kernel refusals (Prop **??**)	`run_kernel_query`	`src/ops.rs`

Bibliography

[1] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.

[2] H. G. Rice, *Classes of recursively enumerable sets and their decision problems*, Transactions of the American Mathematical Society **74** (1953), 358–366.

[3] G. Birkhoff, *Lattice Theory*, 3rd ed., American Mathematical Society Colloquium Publications, vol. 25, 1967.

[4] R. A. Kowalski, *Predicate logic as programming language*, Information Processing **74** (Proc. IFIP Congress), North-Holland, 1974, 569–574.

[5] J. W. Lloyd, *Foundations of Logic Programming*, 2nd ed., Springer-Verlag, 1987.

[6] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.

[7] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, *BLAKE3: One function, fast everywhere*, 2020.

Receipt Chains:

Cryptographic Conservation of Consequence

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Receipt Cryptography, Paper 02

Abstract

The keystone of this series established that trust among bounded agents is manufactured by a *faithful projection* of an unbounded execution onto a coordinate small enough to fit a verifier's bounded context, cryptographically bound so the projection cannot lie about the interior it summarizes [?]. This paper supplies the cryptography that makes the binding real. We define the *frame* — the fixed-width, 128-byte, cache-line-aligned `OcelCausalFrame` that encodes a single manufacturing step and commits the full 256-bit payload digest into its object-reference slots — and the *chain rule* $h_+ = \mathsf{H}(h_- \parallel \beta(\mathsf{fr}))$, a rolling BLAKE3 commitment that folds each frame onto its causal predecessor. We prove, from an explicit collision-resistance axiom, the **Faithful Chain Theorem**: perturbing any committed field forces a hash collision, so a bounded verifier who recomputes one link rejects the perturbation reading $O(1)$ per frame and $O(\kappa)$ at the boundary. We prove the **Conservation of Consequence Theorem** in its receipt-cryptographic form: every actuated artifact occupies *exactly one* chain position, the map from actuation to chain value is injective up to collision, and each artifact carries *at least one* committed admitted cause. We give the **Tamper Localization Theorem** for the staged validator (`schema`, `chain_recompute`, `chain_linkage`, `monotonic`, `token_replay`), which does not merely reject a corrupted ledger but names the first divergent frame and the stage that caught it. We formalize Ed25519 signing as *self-contained authenticity*, separating integrity (what the chain alone proves) from authorship (what a signature adds), and describe the content-addressed *receipt lake* with its OCEL 2.0 / PROV-O mapping and its index-scale query direction. We close with the **Integrity–Virtue Scope Theorem**: a proof of what a receipt chain establishes and, equally rigorously, what it does not — the guardrail that keeps a cryptographic apparatus from being mistaken for a warrant of goodness.

Contents

Chapter 1

Introduction: The Receipt Is a Commitment, Not a Copy

The keystone thesis derives the standard of trust available to a civilization of agents who cannot comprehend one another: not comprehension, but a bounded, faithful, replayable receipt [?]. That derivation leaves one thing on credit. It *assumes* a collision-resistant hash H and asserts that a receipt "cannot lie about the interior it summarizes." This paper pays the debt. We construct the object that discharges the assumption — the receipt chain — and prove, rather than assert, exactly which lies it forecloses and which it does not.

A receipt is often imagined as a *copy*: a smaller transcript of what happened, trusted because it looks like the original. That picture is wrong at every scale that matters. A copy is faithful only if the reader compares it to the original, which is precisely the comprehension cost the keystone showed we cannot pay. The receipt in this doctrine is not a copy but a *commitment*: a short value h_+ such that no adversary can exhibit a different history yielding the same h_+ without breaking a cryptographic assumption. The verifier never compares the receipt to the interior. It recomputes one arithmetic step and checks equality. Fidelity is enforced by the algebra of the hash, not by the diligence of the reader.

The three questions. A receipt discipline must answer three distinct questions, and the doctrine's discipline is to never conflate them:

(Q1) **Integrity.** Was this history altered after the fact? (Answered by the chain.)

(Q2) **Authenticity.** Who vouches for this history? (Answered by a signature.)

(Q3) **Virtue.** Was the history *good* — were the right obligations checked, is the artifact fit for the world? (*Not* answered by cryptography at all.)

Chapters **??**–**??** answer (Q1); Chapter **??** answers (Q2); Chapter **??** proves the boundary of (Q3). Keeping these separate is not pedantry. A system that answers (Q1) and pretends it has answered (Q3) is exactly the overclaim the keystone's antibody clause forbids [?].

Grounding. Every construction here is realized in the praxis codebase, and we name the site so a reader can trace math to machine. The frame is `bcinr_powl_receipt::causal_receipt::OcelCausalFrame`; its serialization is `to_hash_bytes`; the chain rule is `OcelCausalReceipt::chain`; the single frame-construction site shared by emission and replay is `praxis_core::law::build_admission_frame` with `chain_from_frame`; the persisted record is `praxis_core::receipt_record::ReceiptRecord`; the staged validator is `praxis_core::receipt_validator::ReceiptValidator`; signing is `praxis_core::signing`. Where a size, a field order, or a constant appears below, it is the size, field order, or constant the code compiles with, and in several cases the code asserts it at compile time.

Notation. We reuse the series' notation: $\mathcal{O}$ raw observations, $\mathcal{O}^*$ the admitted sub-language, α the admission retraction with refusal $\perp$, $\mu : \mathcal{O}^* \to \mathcal{A}$ the manufacturing morphism, Σ the space of full execution traces, $\mathcal{R}$ the receipt space, H the collision-resistant hash, $\mathrm{dg}(\cdot) = \mathsf{H}(\cdot)$ a digest, and h_-, h_+ the chain values before and after a step. The keystone's law-bearing frame is here made concrete as the byte-exact `OcelCausalFrame`.

Chapter 2

The Frame: A Fixed-Width Encoding of One Manufacturing Step

2.1 Why fixed width

A commitment scheme is only as honest as its encoding. If two distinct histories can serialize to the same bytes, the hash of those bytes commits to neither unambiguously. The first requirement on a frame is therefore *unambiguous, length-determined serialization*: the reader of the byte body must be able to parse each field without a length prefix, because the layout is fixed. The praxis frame achieves this by being a `repr(C, align(64))` struct of a single compile-time-constant size.

Definition 2.1 (Frame)**.** A *frame* fr is the record

$$\mathsf{fr} = \Big\langle\, \texttt{instruction\_id}, \texttt{fired\_mask}, \texttt{denial}, \texttt{obj\_refs}[0{:}8], \texttt{ts\_ns}, \texttt{activity\_idx}, \texttt{node\_kind}, \texttt{prior\_hash}\,\Big\rangle,$$

laid out in memory as a `repr(C, align(64))` structure occupying exactly 128 bytes — two 64-byte cache lines — with the field widths of Table **??**. The in-memory layout carries 5 bytes of interior padding (`pad`) so that the 256-bit `prior_hash` lands aligned; the padding is structural, not semantic.

Remark. The 128-byte size is not a description but an invariant the compiler enforces. The crate contains the const assertion

```
const _: () = { assert!(size_of::<OcelCausalFrame>() == 128) };
```

so a change to any field width that broke the two-cache-line layout would fail to compile. Fixed width is thus a mechanized property (in the sense of the foundations paper's mechanized-totality results), not a convention a maintainer must remember.

2.2 The hash body: what the frame commits

The frame's in-memory image is 128 bytes, but the bytes fed to the hash are a distinct, smaller, canonical serialization: the padding is excluded, and every integer is written little-endian in a fixed order. This is the object the chain actually commits to.

Byte range	Field	Width	Meaning
$[0, 8)$	`instruction_id`	u64 LE	monotone step identity within a run
$[8, 16)$	`fired_mask`	u64 LE	scatter of active denial lanes
$[16, 24)$	`denial.0`	u64 LE	denial-polarity word (admission verdict)
$[24, 56)$	`obj_refs`	8× u32 LE	*payload digest* (§??)
$[56, 64)$	`ts_ns`	u64 LE	wall-clock nanoseconds
$[64, 66)$	`activity_idx`	u16 LE	POWL activity-table index
$[66, 67)$	`node_kind`	u8	POWL node classifier (XOR/SEQ/LOOP/...)
$[67, 99)$	`prior_hash`	32 B	BLAKE3 of the causal predecessor

Table 2.1: The 99-byte canonical hash body $\beta(\mathsf{fr})$ produced by `to_hash_bytes`. The 5 interior `pad` bytes of the 128-byte in-memory frame are *not* hashed. Total committed width: $8 + 8 + 8 + 32 + 8 + 2 + 1 + 32 = 99$ bytes.

Definition 2.2 (Hash body)**.** The *hash body* $\beta(\mathsf{fr}) \in \{0, 1\}^{8 \cdot 99}$ is the 99-byte serialization produced by `to_hash_bytes`, with the field layout of Table **??**. It is a total, injective function of the semantic fields of fr (all fields except the structural `pad`): distinct field tuples yield distinct bodies.

Proposition 2.1 (Body injectivity)**.** *β is injective on the semantic field tuple: if $\beta(\mathsf{fr}) = \beta(\mathsf{fr}')$ then fr and fr' agree on every field of Definition* **??***.*

Proof. Each field occupies a disjoint, fixed byte range (Table **??**), and each integer field's little-endian encoding is a bijection on its width. A byte-equal body forces range-wise equality, hence field-wise equality. The `pad` bytes are absent from the body, so they do not participate; two frames differing only in padding have equal bodies, which is correct — padding carries no consequence. □

2.3 Committing the payload: the object-reference repurposing

The single subtle move in the encoding is how the frame commits the artifact it is a receipt for. The frame has no dedicated "payload" field; instead it carries eight packed object-reference words `obj_refs[0:8]`, each a `u32`, normally OCEL object handles. The praxis frame-builder repurposes all eight words to hold the 256-bit BLAKE3 digest of the canonical payload bytes, *verbatim.*

Construction 2.1 (Payload commitment)**.** Let $p \in \{0, 1\}^*$ be the canonical JSON bytes of an admitted payload and $\mathrm{dg}(p) = \mathsf{H}(p) \in \{0, 1\}^{256}$ its digest. Partition $\mathrm{dg}(p)$ into eight 32-bit little-endian words $w_0, \ldots, w_7$ and set `obj_refs`$[i]$ = `PackedObjRef`(w_i). Crucially the raw tuple constructor `PackedObjRef(w)` is used, *not* `PackedObjRef::new`, which would pack a type index into the high 8 bits and truncate the identifier to 24 bits. The raw constructor preserves all 32 bits, so the full 256-bit digest survives into bytes $[24, 56)$ of $\beta(\mathsf{fr})$ intact.

Proposition 2.2 (The frame commits the payload)**.** *Under Construction* **??***, $\beta(\mathsf{fr})$ determines $\mathrm{dg}(p)$, and by Proposition* **??** *any change to $\mathrm{dg}(p)$ changes $\beta(\mathsf{fr})$. Consequently a frame commits not to an opaque object handle but to the content of the artifact, exactly as the keystone's law-bearing frame requires the digest $\mathrm{dg}(a)$ of the artifact to appear inside the frame.*

Proof. Bytes $[24, 56)$ of the body are the eight words $w_0 \ldots w_7 = \mathrm{dg}(p)$ by construction; the map $\mathrm{dg}(p) \mapsto (w_0, \ldots, w_7)$ is the little-endian bijection $\{0,1\}^{256} \to (\{0,1\}^{32})^8$. Injectivity of β (Proposition **??**) then gives that distinct payload digests induce distinct bodies. □

Remark (Why not add a field)*.* Repurposing `obj_refs` rather than widening the struct keeps the frame at exactly two cache lines and requires no change to the downstream hashing code: `to_hash_bytes` already serializes all eight words verbatim. This is a recurring discipline in the codebase — commit more meaning without growing the committed object — and it is the byte-level instance of the projection principle: the interior (an arbitrarily large payload) is bound by a fixed-width coordinate (its 256-bit digest inside a 99-byte body).

Chapter 3

The Chain: A Rolling Commitment

3.1 Genesis and the fold

A single frame commits one step. A *chain* commits an ordered history by folding each frame's body onto the running commitment of everything before it.

Definition 3.1 (Genesis)**.** The *genesis* chain value is $\mathsf{g} = \mathsf{H}(\mathbf{0}_{32})$, the BLAKE3 digest of 32 zero bytes. A run may additionally bind a *domain seed* $\mathsf{g}_{\text{dom}} = \mathsf{H}(\text{“}\langle\text{project}\rangle\text{-v}\langle\text{version}\rangle\text{-genesis”})$ so that chains from distinct projects or crate versions cannot cross-verify: a receipt minted under one domain seed is not a valid link of another's chain.

Definition 3.2 (Chain rule)**.** For a frame fr with predecessor commitment h_-, the successor commitment is

$$h_+ \;=\; \mathsf{H}\big(h_- \parallel \beta(\mathsf{fr})\big), \tag{3.1}$$

where $\parallel$ is byte concatenation. A *chain* (ledger) $\mathcal{C} = (\mathsf{fr}_1, \ldots, \mathsf{fr}_n)$ has commitments $h_0 = \mathsf{g}$ and $h_t = \mathsf{H}(h_{t-1} \parallel \beta(\mathsf{fr}_t))$ for $1 \le t \le n$; its terminal value is h_n.

Remark (The predecessor is mixed twice). The praxis realization mixes h_- into h_+ at two points: once as the frame's own `prior_hash` field (bytes $[67, 99)$ of the body) and again as the seed of the fold in (**??**). This double-mixing predates the current construction and is retained for compatibility. It neither strengthens nor weakens the security argument below — h_+ remains a deterministic function of h_- and fr — so all theorems hold verbatim whether the predecessor is mixed once or twice. We note it only so a reader diffing the code against (**??**) is not surprised.

3.2 Determinism and the single construction site

Proposition 3.1 (The chain is a deterministic fold)**.** *h_n is a deterministic function of $(\mathsf{g}, \mathsf{fr}_1, \ldots, \mathsf{fr}_n)$, computable by a single left fold: $h_n = \big(\lambda h.\, \lambda\mathsf{fr}.\, \mathsf{H}(h \parallel \beta(\mathsf{fr}))\big)^{\star}(\mathsf{g},\ (\mathsf{fr}_t)_t)$. Equal inputs give bit-identical h_n.*

Proof. Immediate from (**??**): each step is a pure function of two arguments, and composition of pure functions is pure. BLAKE3 is a fixed function, so no nondeterminism enters. ☐

Proposition **??** has an operational corollary that the codebase enforces structurally. Emission (minting a receipt at manufacture time) and replay (recomputing it later from stored fields) must agree, or tamper detection would raise false positives. The crate guarantees agreement by routing both through one function.

Proposition 3.2 (No emission/replay drift). *Let a receipt be emitted by `LawObject::receipt_with_record` and later recomputed by `ReceiptRecord::recompute_chain_hash`. Both call the same `build_admission_frame` and `chain_from_frame`. Hence for identical stored fields they compute identical h_+; any disagreement between a stored `chain_hash_hex` and its recomputation is therefore attributable to a change in a stored field, never to divergent code paths.*

Proof. There is a single construction site: `build_admission_frame` is the only function that assembles an `OcelCausalFrame` for a receipt, and `chain_from_frame` the only one that folds it. Both callers invoke these with the record's own fields (`payload_hash`, `prev_chain_hash`, `ReceiptMeta`, `ts_ns`). Function purity (Proposition **??**) then forces equal outputs on equal inputs, so a recompute mismatch isolates a field mutation rather than a logic difference. □

Proposition 3.3 (Prefix commitment). *For each t, h_t commits every frame $\mathsf{fr}_1, \ldots, \mathsf{fr}_t$ and the genesis value: h_t is a function of all of them, and (under the collision-resistance axiom of the next chapter) any change to any of them changes h_t except with negligible probability. In particular the terminal h_n commits the entire history.*

Proof. By induction on t. Base: $h_0 = \mathsf{g}$. Step: $h_t = \mathsf{H}(h_{t-1} \parallel \beta(\mathsf{fr}_t))$ depends on h_{t-1} (which by hypothesis commits $\mathsf{fr}_1, \ldots, \mathsf{fr}_{t-1}$ and g) and on $\beta(\mathsf{fr}_t)$. So h_t depends on all of $\mathsf{g}, \mathsf{fr}_1, \ldots, \mathsf{fr}_t$. The collision clause is deferred to Theorem **??**. □

3.3 Git-as-a-Runtime CAS Locking and Ledgers

To prevent coordination drift in local-first, air-gapped environment execution, we rely on the host repository's reference structures to enforce atomic locks.

Definition 3.3 (Git CAS Lock). An atomic Compare-And-Swap (CAS) lock for resource L at repository HEAD OID H is acquired by invoking the reference update:

$$\texttt{git update-ref refs/locks/}L\ H\ \mathbf{0}, \tag{3.2}$$

which fails with a non-zero exit code if `refs/locks/`L already exists, and succeeds atomically otherwise via POSIX rename. Deletion on drop releases the lock.

Proposition 3.4 (Append-Only notes Ledger). *The audit ledger is stored under the custom namespace `refs/notes/praxis/audit`. Appending entries is a commit transition in which the hash-chain receipts are written as NDJSON notes, preserving the immutability of the execution timeline.*

Chapter 4

The Faithful Chain Theorem

We now discharge the keystone's standing assumption by stating it as an explicit axiom and proving what it buys.

Axiom 4.1 (Collision resistance)**.** H (instantiated as BLAKE3 [?]) is collision-resistant: no probabilistic polynomial-time adversary finds $x \neq y$ with $\mathsf{H}(x) = \mathsf{H}(y)$ except with negligible probability $\varepsilon(\lambda)$ in the security parameter λ. For BLAKE3, $\lambda = 256$ and the best generic attack is the birthday bound $\sim 2^{128}$.

Theorem 4.1 (Faithful Chain)**.** *Let $\mathcal{C} = (\mathsf{fr}_1, \ldots, \mathsf{fr}_n)$ be a chain with terminal commitment h_n, and let $\mathcal{C}' = (\mathsf{fr}'_1, \ldots, \mathsf{fr}'_n)$ differ from $\mathcal{C}$ in at least one committed field of at least one frame (any semantic field of Table* **??***, including the payload digest of Construction* **??***). If $\mathcal{C}'$ has the* same *terminal commitment h_n, then a collision of H has been exhibited. Consequently, under Axiom* **??***, an adversary who alters any committed field yet preserves h_n succeeds only with probability $\leq \varepsilon(\lambda)$.*

Proof. Let j be the least index at which $\mathcal{C}$ and $\mathcal{C}'$ differ in a committed field. Bodies are injective on committed fields (Propositions **??**, **??**), so $\beta(\mathsf{fr}_j) \neq \beta(\mathsf{fr}'_j)$. Because j is least, the predecessors agree: $h_{j-1} = h'_{j-1}$. Consider the two fold inputs $x = h_{j-1} \,\|\, \beta(\mathsf{fr}_j)$ and $x' = h'_{j-1} \,\|\, \beta(\mathsf{fr}'_j)$. Since the prefixes $h_{j-1} = h'_{j-1}$ are equal and have equal length (32 bytes) while the suffixes differ, $x \neq x'$. Now examine the terminal values. If $h_j \neq h'_j$, then by Proposition **??** the divergence propagates: each subsequent fold takes a differing 32-byte prefix, so equality of the terminal $h_n = h'_n$ would require some fold step $t \geq j$ to map unequal inputs to equal outputs — a collision. If instead $h_j = h'_j$ despite $x \neq x'$, then $\mathsf{H}(x) = \mathsf{H}(x')$ is itself a collision at step j. In either case $h_n = h'_n$ forces a collision, contradicting Axiom **??** up to $\varepsilon(\lambda)$. □

Corollary 4.2 (Bounded verification cost)**.** *A verifier checking a single frame recomputes one fold (***??***) — one BLAKE3 of a $32 + 99 = 131$-byte input — and one equality test: $O(1)$ time and space, independent of n and of the interior dimension of the manufacture the frame receipts. Checking the boundary projection of a whole run (verdict, terminal h_n, fitness scalar, refusal reason) is $O(\kappa)$, matching the keystone's Comprehension–Verification Gap.*

Proof. The fold reads two fixed-width operands and produces a 256-bit output; BLAKE3 on a fixed-length input is constant work. Equality on 256-bit values is constant. The boundary projection has $\leq \kappa$ chunks by construction (keystone, Def. receipt projection), so applying the boundary predicate is $O(\kappa)$. Neither cost references n or $\dim \Sigma$. □

Remark (Faithful *only* over committed fields)*.* Theorem **??** is deliberately scoped to committed fields. A change confined to a field *absent* from β — the `pad` bytes, or the descriptive `activity` label and `duration_ms` of `ReceiptRecord`, which are marked non-hashed — does not change h_n and is, correctly, *not* attested. Faithfulness is a property of the commitment, and the commitment is exactly Table **??**. This is the byte-exact instance of the keystone's converse clause: a frame committing bytes alone would attest tamper-evidence but not lawfulness; a frame committing the denial word, the payload digest, and the transition identity attests those. What you commit is what you can trust; no more, and no less.

Chapter 5

Conservation of Consequence

The keystone stated Conservation of Consequence abstractly. Here it becomes a theorem about chain positions, provable directly from Definitions **??** and the Chatman equation.

Theorem 5.1 (Conservation of Consequence, receipt form)**.** *Fix the Chatman equation $\mathcal{A} = \mu(\mathcal{O}^*)$ with the receipt chain of Definition* **??***. Then:*

(i) ***Caused.*** *Every actuated artifact a satisfies $a = \mu(x)$ for some admitted $x = \alpha(o) \neq \bot$; no artifact outside* $\operatorname{im}\mu$ *is actuated, and a receipt is minted only for an object that reached the `Admitted` stage.*

(ii) ***Positioned.*** *Each actuation appends* exactly one *frame and advances the chain by* exactly one *commitment; the map Φ : actuation $\mapsto h_+$ is injective up to hash collision.*

(iii) ***Attributed.*** *The committed frame carries at least one admitted cause: bytes* $[24, 56)$ *commit* $\mathrm{dg}(x)$ *(the admitted-payload digest, Construction* **??***) and bytes* $[16, 24)$ *commit the denial word certifying x passed admission.*

Proof. (i) Actuation is gated by the equation: an artifact enters $\mathcal{A}$ only as $\mu(x)$ with $x \neq \bot$, so $\operatorname{im}\mu$ exhausts the actuated set. In the code, `receipt` is a method available only on `LawObject<_, Admitted, _>`; the typestate makes "receipt without admission" uninhabited (foundations paper), so no artifact is receipted without an admitted antecedent.

(ii) Each call to `chain` increments `frame_count` by one and produces one new `chain_hash`; (**??**) appends exactly one commitment per frame. Suppose two distinct actuations mapped to the same h_+. Their frames differ (distinct instruction identity in `instruction_id`, or distinct payload digest), so by Theorem **??** equal terminal commitments force a collision. Hence Φ is injective up to $\varepsilon(\lambda)$.

(iii) By Construction **??**, $\mathrm{dg}(x)$ occupies bytes $[24, 56)$; the denial word `denial.0` occupies $[16, 24)$ and equals `ADMITTED` on the accept path (the record persists an admitted-polarity frame). Thus the frame commits both the identity of the cause and the verdict that admitted it. □

Corollary 5.2 (No orphan and no double-spend of consequence)**.** *No actuated artifact lacks a chain position (*no orphan*: by (i)+(ii)), and no chain position is shared by two actuations (*no double-count*: by (ii)). Consequence is neither created without an admitted cause nor recorded without a unique receipt slot.*

Remark (Uniqueness of cause is by commitment, not by inversion)*.* As the keystone stressed, μ need not be injective: two admitted observations may manufacture the same artifact. Clause (iii) does not claim a determines x; it claims the frame *commits* $\mathrm{dg}(x)$, so h_+ pins the cause even where the artifact alone would not recover it. Uniqueness of the cause is a property of the receipt, not a false property of the morphism.

Chapter 6

Tamper Localization: The Staged Validator

Theorem **??** gives a *detector*: a tampered ledger fails to verify. A production verifier must do more — it must say *where* and *how* the ledger is broken, because at fleet scale "this ledger is invalid" is an alarm with no address. The praxis `ReceiptValidator` is a staged pipeline that localizes the fault.

Definition 6.1 (Validation pipeline)**.** The validator runs five stages in fixed order over a ledger $(\mathsf{fr}_t)_{t=1}^{n}$, each returning `Pass`, `Fail`(message), or `Skip`(reason), and reports *all* stage outcomes rather than short-circuiting:

(S1) `schema` — every record has the supported version and well-formed 32-byte hex fields (`payload_hash`, `prev_chain_hash`, `chain_hash`).

(S2) `chain_recompute` — for each t, `recompute_chain_hash` equals the stored `chain_hash_hex`. This is the direct instantiation of Theorem **??**: a mismatch is a committed-field tamper.

(S3) `chain_linkage` — for each $t > 1$, `prev_chain_hash_hex`(t) = `chain_hash_hex`$(t-1)$: the records are actually threaded, not merely individually valid.

(S4) `monotonic` — `instruction_id` is strictly increasing, `ts_ns` is non-decreasing, and no `ts_ns` exceeds the (injectable) clock's "now".

(S5) `token_replay` — each record replays through the fixed POWL token model to fitness exactly `0x0001_0000` (§**??**).

Theorem 6.1 (Tamper localization)**.** *Suppose a lawful ledger $\mathcal{C}$ is perturbed to $\mathcal{C}'$ by a modification whose first affected record is index j. Then the pipeline of Definition* **??** *reports a* `Fail` *whose named record index is j (equivalently, the least index whose recomputation, linkage, monotonicity, or replay breaks), and the stage identifier names the class of the fault. The honest prefix $\mathsf{fr}_1, \ldots, \mathsf{fr}_{j-1}$ is not implicated.*

Proof. The stages scan records in increasing index and each returns on its *first* failing record. Consider the classes of a single-record perturbation at j:

- If the mutation malforms a hex field, `schema` fails at j (earlier records parse, being unperturbed).

- If the mutation changes a committed field but leaves `chain_hash_hex`(j) stale, `chain_recompute` fails at j: by Proposition ?? the recomputation uses the perturbed fields, so it diverges from the stored value exactly at j; records $< j$ are unperturbed and recompute equal.
- If the mutation reorders or splices records so that thread continuity breaks (e.g. a swap), `chain_linkage` fails at the least index whose `prev` no longer matches its predecessor's `chain_hash`.
- If the mutation rewrites both a field *and* its stored chain hash to be internally consistent (a coordinated forgery), then either linkage breaks at j (the forged `chain_hash`(j) no longer equals what record $j+1$ expects, unless the adversary rewrites the entire suffix), or, if the whole suffix is rewritten, the forged terminal commitment differs from the attested h_n and Theorem ?? applies at the boundary — an anchored h_n (e.g. a signed or externally published terminal value, see Chapter ??) makes suffix rewriting a collision problem.
- If the mutation preserves hashes but produces a trace that is not a firing sequence, `token_replay` fails at j with fitness < 1 (§??).

In every class the least failing index is j and the honest prefix passes every stage, because each stage's predicate on records $< j$ is evaluated on unperturbed data and holds by hypothesis. □

Remark (Report-all, not fail-fast, across stages). Within a stage the scan returns on the first offending record (so the *record* is localized); across stages the pipeline runs every stage and reports each outcome (so the *fault class* is fully characterized, and a ledger broken in two independent ways shows both). This mirrors the affidavit-style `verify` pipeline's "run every stage, report all of them" shape and is what lets a downstream consumer act on the specific defect rather than a bare Boolean.

6.1 Fitness as the replay coordinate

Definition 6.2 (Replay fitness)**.** Replay fitness is the normalized conformance metric of the keystone's marking geometry, $\varphi = 1 - \frac{\text{tokens forced on unenabled transitions}}{\text{tokens the replay attempted}} \in [0, 1]$, represented in the code as a Q16.16 fixed-point integer: the value 1.0 is the constant `0x0001_0000` $= 65536$. The `token_replay` stage accepts a record iff its replayed fitness equals `0x0001_0000`.

Proposition 6.2 (Fitness = 1 characterizes lawful replay)**.** *A record's replay attains $\varphi =$ `0x0001_0000` iff its lifecycle is a genuine firing sequence of the fixed POWL token model; otherwise the first forced consumption localizes the nonconformant step, and the stage fails naming that record.*

Proof. By the keystone's unit-fitness characterization: enabled firings force no consumption, giving numerator 0 and $\varphi = 1$ (= `0x0001_0000` in Q16.16); a disabled event forces a consumption, making the numerator positive and $\varphi < 1$, at the earliest offending index. The stage compares against the exact fixed-point unit. □

Chapter 7

Authenticity: Ed25519 as a Self-Contained Signature

The chain answers (Q1), integrity: *was this history altered?* It does not by itself answer (Q2), authenticity: *who vouches for it?* An adversary who fabricates an entirely fresh but internally consistent ledger produces a valid chain — valid chains are cheap to manufacture; what is expensive is manufacturing one that collides with a *committed* prior value. Authenticity is added by binding the terminal commitment to a keyholder.

Definition 7.1 (Signed receipt)**.** A *signed receipt* is the triple `SignedReceipt` $= \langle h_{\text{hex}},\ s,\ k\rangle$ where h_{hex} is the hex terminal chain value, $s = \text{Sign}_{sk}(h_{\text{hex}})$ is an Ed25519 signature [?] over it, and k is the hex verifying key. It is *self-contained*: s and k travel with h_{hex}, so the object verifies without out-of-band key material.

Proposition 7.1 (Integrity vs. authenticity are distinct checks)**.** *There are two verification predicates:*

- ***Integrity* (`verify_chain_hash`)*: check s against the* embedded *key k and that k's signature covers the presented h. This proves the signed object was not altered after signing, but trusts whatever key is inside it.*
- ***Authenticity* (`verify_chain_hash_with_key`)*: check s against an* externally pinned *key $k^\star$. This fails if the receipt was signed by any key other than $k^\star$, even if internally consistent.*

The first is necessary but not sufficient for the second; a verifier who wants "this came from the party I trust" must use the second and supply $k^\star$ from an independent source.

Proof. Ed25519 verification is a function of (message, signature, key). `verify_chain_hash` instantiates the key argument with the embedded k; any adversary who re-signs a modified h with a self-generated key k' passes this check, so it does not establish authorship. `verify_chain_hash_with_key` instantiates the key argument with $k^\star$; by Ed25519's existential unforgeability [?], producing a valid s over h under $k^\star$ without $sk^\star$ is infeasible, so a pass attributes authorship to the holder of $sk^\star$. The embedded-key check therefore cannot substitute for the pinned-key check. □

Proposition 7.2 (Fail-closed signing)**.** *When the `signed` feature is enabled, a missing or unreadable signing key (`PRAXIS_SIGNING_KEY` / `_FILE`) is a hard error, not a silent skip: the signing path returns `SigningFailed` rather than emitting an unsigned receipt.*

Proof. The `signed` feature exists to *guarantee* a signature; silently skipping would defeat its purpose and let an unsigned receipt masquerade as covered. The implementation therefore treats key absence as failure. Authenticity is thus either present and checkable or explicitly absent, never ambiguously assumed. □

Remark (What a signature adds to the chain, precisely). Composing Theorem **??** with Proposition **??**: the chain makes *internal* tampering a collision problem; the signature makes *substituting a different history* an unforgeability problem, provided the verifier pins $k^\star$ and treats h_n as the anchored value. Neither alone suffices against a resourceful adversary; together they reduce forgery to breaking BLAKE3 collision resistance *or* Ed25519 unforgeability.

Chapter 8

The Receipt Lake: Content-Addressed and Queryable at Index Scale

Individual receipts chain into a per-run ledger. Across a fleet, ledgers accumulate into a *receipt lake* $\mathcal{L}$: a content-addressed, append-only store engineered so that the population of receipts is queryable without reading interiors.

Definition 8.1 (Content addressing)**.** The address of a stored object b is $\texttt{content\_address}(b) = \mathsf{H}(b)$ in hex. Two byte-identical objects share an address; any change yields a different address. A receipt's own terminal h_n is such an address, so a receipt is stored under a name that is a function of its content.

Proposition 8.1 (De-duplication and immutability are free)**.** *In a content-addressed lake, (a) storing the same receipt twice is idempotent (same address), and (b) mutating a stored receipt changes its address, so the old address still resolves to the old bytes: history is immutable by construction, and references are stable.*

Proof. (a) The address is $\mathsf{H}(b)$, a function of b; equal b gives equal address. (b) A change $b \to b'$ with $b \neq b'$ gives $\mathsf{H}(b') \neq \mathsf{H}(b)$ except on a collision (Axiom **??**); the reference $\mathsf{H}(b)$ therefore cannot be made to resolve to b'. □

8.1 OCEL 2.0 and PROV-O mappings

A receipt is not only a hash; it carries process structure. Each `ReceiptRecord` names the OCEL objects it governs via `object_ids` (event-to-object links), an `activity_idx` and `node_kind` placing it in a POWL process, and an `instruction_id` sequencing it. This lets the ledger export as a standard event log.

Construction 8.1 (OCEL 2.0 export)**.** The map $\Omega : \mathcal{C} \to \text{OCEL2.0}$ sends each frame to an OCEL *event* with: event id = `instruction_id`; activity = label of `activity_idx`; timestamp = `ts_ns`; and event-to-object (E2O) qualifiers to each identifier in `object_ids`. The result is a conformant OCEL 2.0 log queryable by any process-mining tool, produced by `receipt export-ocel`.

Construction 8.2 (PROV-O / SHACL bridge)**.** Each receipt maps to a PROV-O [**?**] shape: the admitted observation is a `prov:Entity`, the manufacture a `prov:Activity` that `prov:used` it, the artifact a `prov:Entity` that `prov:wasGeneratedBy` the activity, and the chain link a `prov:wasDerivedFrom` edge to the predecessor. The praxis `receipt_shacl` module realizes a concrete instance, validating a `ReceiptRecord` against the `sr:SharedReceiptV1` SHACL shape — so a receipt is not merely serializable to RDF but *shape-checked* against a published contract before it enters the lake.

Proposition 8.2 (The mappings preserve the commitment)**.** *Ω and the PROV-O bridge are lossless over the committed fields: the terminal h_n is recoverable from the exported log's per-event fields via Definition* **??***, so exporting to OCEL 2.0 or RDF does not weaken the chain guarantee — a consumer can recompute and re-verify from the exported form.*

Proof. Ω carries each frame's committed fields (`instruction_id`, `activity_idx`, `ts_ns`, the object identifiers, and the hex hashes) into event attributes; the chain recomputation of Proposition **??** depends only on these plus `payload_hash` and `prev_chain_hash`, which the record retains. Hence the exported log determines h_n, and re-verification is Theorem **??** applied to the reconstructed frames. □

8.2 Index-scale query: the QLever direction

Remark (Querying a lake without reading interiors)*.* Once receipts are RDF triples (Construction **??**), the population is a knowledge graph. An engineered SPARQL index such as QLever [**?**] answers queries over 10^{11}-scale triple stores by a join plan no engineer reconstructs — the very "differential agreement over a checked projection" the keystone cites as manufactured trust. The design direction is: the lake stores content-addressed receipts; a triple index over their committed fields answers fleet-scale questions ("which agents refused with category c in window w?", "what is the pass-rate over frontier F?") at index speed, touching only the $O(\kappa)$-sized committed coordinates, never the interiors. This is the projection principle at the scale of a whole civilization's audit log: the interior is a trillion manufactures; the queryable surface is their committed frames.

8.3 Git-CAS Locking and Crash Recovery

The `chatman-common` crate realizes atomic receipt ledger writes by mapping the receipt lock problem onto the host Git object store. For an air-gapped, local-first execution environment where no external coordinator is available, this is the correct level of mechanism: Git's reference-update protocol uses POSIX rename, which is atomic on POSIX-compliant filesystems.

Definition 8.2 (Git CAS Lock, operational form)**.** Let L be a named resource (e.g. the praxis audit ledger) and let H be the current HEAD OID of the local repository. The CAS lock for L is acquired by

$$\texttt{git update-ref refs/locks/}L\ H\ \mathbf{0}_{20},$$

which atomically creates `refs/locks/`L pointing to H, succeeding iff the reference does not already exist (the old-value argument $\mathbf{0}_{20}$ means "require absent"). Deletion of the reference on drop releases the lock. The lock is therefore held for at most the duration of one `git` `update-ref` round-trip plus the critical section, without holding any OS-level file lock that could block unrelated operations.

Proposition 8.3 (Append-only notes ledger under CAS)**.** *The audit ledger is stored under the custom Git notes namespace* `refs/notes/praxis/audit`*. Under the CAS lock of Definition* **??***, each append of a receipt record is a* commit transition*: the new record is serialized as NDJSON and appended to the notes tree under the commit OID it annotates. Because Git commits are content-addressed (the commit hash is the BLAKE3 of its tree plus metadata), the appended entry is immutable by Git's object model. The notes reference advances atomically under the CAS guard, so two concurrent append attempts on the same ledger are serialized by the acquire-fail branch of the CAS.*

Proof. A failing CAS (reference already exists) exits with a non-zero code before touching the notes tree; only the lock holder proceeds. Within the critical section, the notes append is a single `git notes add` followed by the reference-delete on the lock. If the process is killed after the notes append but before the lock release, the lock entry remains; the restart protocol in `chatman-common` detects a stale lock by comparing the locked OID H to the current HEAD — if they differ (a new commit has been pushed), the lock is stale and is force-released, because the locked OID no longer uniquely identifies a live critical section. □

Remark (Relationship to the Faithful Chain Theorem)*.* The CAS lock protocol is orthogonal to the hash chain: the chain guards *what was recorded* (no field tampered), while the CAS lock guards *that only one writer appended at a time* (no race). Together they give a ledger that is simultaneously append-sequential (by the lock) and tamper-evident (by the chain). Violation of sequentiality (two writers appending concurrently) would not break Theorem **??**; it would produce a fork in the notes tree rather than a hash collision. The CAS lock prevents the fork; the chain prevents the tamper; neither subsumes the other.

Chapter 9

Integrity Is Not Virtue: The Scope Theorem

The most important theorem in a cryptographic doctrine is the one that says what the cryptography does *not* prove. Without it, a receipt chain becomes a laundering device: tamper-evidence dressed up as a warrant of goodness. We state the boundary precisely.

Theorem 9.1 (Integrity–Virtue Scope)**.** *Let a receipt chain $\mathcal{C}$ verify (Theorem* **??***) and, optionally, be authentically signed (Proposition* **??***). Then $\mathcal{C}$ establishes exactly the following, and no more:*

(A) ***Integrity.*** *The committed fields of every frame are as they were when minted; no committed field was altered post hoc (up to $\varepsilon(\lambda)$).*

(B) ***Attribution.*** *Each artifact is committed to an admitted cause and a denial word, and (if signed) to a keyholder.*

(C) ***Conformance-as-checked.*** *Each replayed lifecycle is a firing sequence of the* fixed *POWL model to fitness `0x0001_0000` (Proposition* **??***).*

It does **not** *establish:*

(a) ***Obligation adequacy.*** *That the obligation battery G was the* right *set of checks — the chain commits* $\mathrm{dg}(G)$ *and the denial word, not the wisdom of choosing G.*

(b) ***World-fitness.*** *That the artifact is correct, safe, useful, or ethical in the world — μ is deterministic and bounded, not benevolent.*

(c) ***Semantic truth.*** *That the admitted observation* meant *what it purported; admission retracted onto a decidable sub-language (Rice quarantine), it did not decide meaning.*

Proof. (A) is Theorem **??**: altering a committed field forces a collision. (B) is Conservation of Consequence (iii) (Theorem **??**) composed with Proposition **??**. (C) is Proposition **??**, and is scoped to the *fixed* model: fitness certifies conformance to the model that was replayed, not that the model is the correct model.

For the negatives, we exhibit that each is independent of the verifying predicate. (a) Two ledgers with different obligation sets $G \neq G'$ can both verify; the chain commits $\mathrm{dg}(G)$ so it *records* which set was used, but Verify returns `Pass` for any committed G, so verification does not rank G against G'. (b) Fix any admitted x; both a benign and a harmful deterministic μ produce verifying chains, since verification never inspects the world-consequences of $a = \mu(x)$ — only that a's digest is committed. Hence world-fitness is not a function of the verdict. (c) By the series' Rice specialization, no computable predicate decides the semantic property "o means what it purports"; admission is a retraction, not a decision, so a verifying receipt attests admission, which is provably weaker than truth. Each negative is therefore not entailed by (A)–(C). □

Corollary 9.2 (The antibody clause, cryptographic form). *A claim of the form "this artifact is* good *because its receipt verifies" is an overclaim: it asserts a virtue (b) that the verifying predicate provably does not entail (Theorem* **??***). The admissible claim is narrower and exact: "this artifact was manufactured from an admitted cause under obligation set G, unaltered and (optionally) signed by $k^\star$, conformant to model M." A discipline that only emits the narrow claim is machinery; one that emits the broad claim is grandiosity.*

Proof. The broad claim entails (b), which Theorem **??** shows is independent of the verdict; so the broad claim is not supported by the receipt. The narrow claim is exactly the conjunction (A)$\wedge$(B)$\wedge$(C), each proven. Emitting only what is entailed is the keystone's `docs-exceed-mechanism` invariant applied to cryptographic claims. □

Chapter 10

Conclusion

We built the cryptography the keystone assumed. A frame is a fixed-width, 128-byte, cache-aligned encoding of one manufacturing step whose 99-byte hash body commits the full payload digest, the denial word, and the transition identity (Chapter **??**). A chain folds frames by $h_+ = \mathsf{H}(h_- \parallel \beta(\mathsf{fr}))$ into a rolling commitment computed by a single, drift-free construction site (Chapter **??**). From an explicit collision-resistance axiom we proved the Faithful Chain Theorem — perturbing any committed field forces a collision, and a bounded verifier rejects it reading $O(1)$ per frame (Chapter **??**) — and Conservation of Consequence in receipt form: exactly one chain position per actuation, injective up to collision, each artifact committed to an admitted cause (Chapter **??**). The staged validator does not merely detect tampering but localizes it to the first divergent frame and names the fault class (Chapter **??**). Ed25519 signatures add self-contained authenticity, kept distinct from integrity and made fail-closed (Chapter **??**). Receipts accumulate into a content-addressed lake with lossless OCEL 2.0 / PROV-O mappings and an index-scale query direction (Chapter **??**).

The final theorem is the one a cryptographer must never omit: integrity is not virtue (Chapter **??**). A verifying, signed chain proves that a history is unaltered, attributed, and conformant-as-checked — and proves, with equal rigor, that it says nothing about whether the obligations were wise, the artifact good, or the observation true. That boundary is not a weakness of the receipt. It is the discipline that lets a receipt be trusted for exactly what it commits, at a scale where trusting it for more would be the first lie a trillion agents tell each other.

A receipt commits; it does not absolve.

Appendix A

Notation and Field Layout

Symbol	Meaning
$\mathsf{fr},\ \beta(\mathsf{fr})$	frame; its 99-byte canonical hash body (Table **??**)
$\mathsf{H},\ \mathrm{dg}(\cdot)$	BLAKE3 hash; digest $\mathrm{dg}(b) = \mathsf{H}(b)$
$h_-,\ h_+,\ h_t$	predecessor / successor / step-t chain commitment
g	genesis commitment $\mathsf{H}(\mathbf{0}_{32})$, optionally domain-bound
$\|$	byte concatenation
$\mathcal{C},\ \mathcal{L}$	chain (ledger); content-addressed receipt lake
$\varepsilon(\lambda)$	negligible collision/forgery probability; $\lambda = 256$
Φ	actuation $\mapsto$ terminal commitment map (injective up to collision)
φ	replay fitness; unit = `0x0001_0000` (Q16.16)
$G,\ \mathrm{dg}(G)$	obligation battery; its committed digest
$k,\ k^\star,\ sk$	embedded / pinned verifying key; signing key
Ω	OCEL 2.0 export map (Construction **??**)

Code sites. `OcelCausalFrame`, `to_hash_bytes`, `OcelCausalReceipt::chain` (`bcinr-powl-receipt`); `build_admission_frame`, `chain_from_frame`, `ReceiptRecord`, `recompute_chain_hash`, `ReceiptValidator`, `signing` (`praxis-core`); `content_address`, `SignedReceipt` (`chatman-common`); `receipt_shacl` (root crate).

Bibliography

[1] The Praxis / Capability Physics Program. *The Chatman Equation and the Industrial Revolution of Knowledge*, v1.2.0, Nov. 2025.

[2] J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, J. O'Connor. *BLAKE3: One Function, Fast Everywhere*. 2020.

[3] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, B.-Y. Yang. *High-speed high-security signatures* (Ed25519). J. Cryptographic Engineering, 2012.

[4] T. Lebo, S. Sahoo, D. McGuinness (eds.). *PROV-O: The PROV Ontology*. W3C Recommendation, 2013.

[5] A. Berti, I. Koren, J. N. Adams, et al. *OCEL (Object-Centric Event Log) 2.0 Specification*. 2023.

[6] H. Bast, B. Buchhold. *QLever: A Query Engine for Efficient SPARQL+Text Search*. CIKM, 2017.

[7] H. G. Rice. *Classes of recursively enumerable sets and their decision problems*. Trans. AMS, 1953.

Mission Geometry:

Planning, Conformance, and Separability for Bounded Receipted Execution

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Pillar III

Abstract

This is Pillar III of the Chatman Equation thesis series: the geometry beneath the manufacturing morphism μ and the conformance invariant of the enforced equation. We treat mission execution as two dual questions — *manufacture as search* (how a plan is produced) and *conformance as geometry* (how a trace is admitted) — and give both a first-principles account grounded line-for-line in the `praxis` and `bcinr-pddl` codebase. We define the PDDL action law with numeric fluents, and derive the attention fluent balance law as the feasibility condition of a durative-action schedule, with conservation of the borrowed-and-returned fluent proved directly. We build the marking polytope from the Petri state equation $\boldsymbol{m} = \boldsymbol{m}_0 + \mathbf{N}\boldsymbol{x}$, show the STRIPS grounding is a place/transition net whose forward search inhabits marking space, and prove that the lifecycle token verifier's branchless firing rule enabled $\leftarrow$ (enabled $\wedge \neg \boldsymbol{m}^-) \vee \boldsymbol{m}^+$ is exactly the safe-net specialization of that equation. We formalize conformance as membership in the reachability cone, and prove a Farkas separation theorem: a nonconformant marking admits a separating-hyperplane certificate of dimension p, sound for non-reachability and verifiable at $O(\kappa)$. We define token-replay fitness as the population ratio the `PowlReplayVerifier` computes in Q16.16, and prove that unit fitness with completion characterizes exactly the genuine firing sequences, with the first disabled transition a coordinate-localized witness. We formalize POWL 2.0 as recursive partial orders and choice graphs, show the plan-to-tape transitive reduction yields the Hasse diagram of the precedence order, and reframe the Kourani–van der Aalst separable decomposition [?] as a Rice quarantine for processes: separable is admitted, non-separable is refused with reason. Finally we derive lexicographic cost arbitration as the infinite-weight limit in which the admitted coordinate dominates, and give the makespan functional with its critical-path and capacity-sensitivity structure as computed by `analyze_schedule`. Every theorem is anchored to the `bcinr-pddl` planner (`find_plan`, `find_temporal_plan`), the `PowlReplayVerifier`, the capability router `CostVector`, and the `ScheduleAnalysis64` analyzer.

Contents

Chapter 1

Introduction: Manufacture as Search, Conformance as Geometry

1.1 Where this pillar sits

The foundations paper of this series fixed the Chatman equation $\mathcal{A} = \mu(\mathcal{O}^*)$ and its enforced form, the Bounded Receipted Chatman Equation (BRCE there), a conjunction of four invariants: an admission gate, a bounded deterministic manufacture, receipt totality, and *conformance* — the requirement that an actuation's lifecycle replay have fitness $\varphi = 1$. Two of those invariants are the subject of this pillar. The manufacturing morphism μ, when its artifact is a *mission* — a schedule of durative actions achieving a goal — is a bounded search over a state space. The conformance invariant is a *geometric* membership test: does a trace lie inside the polytope of lawful markings? We develop both from first principles and show they are the same object viewed from two sides.

Question	Object	`praxis/bcinr-pddl` realization
Manufacture (μ)	bounded search	`find_plan` (BFS), `find_temporal_plan`
Conformance (B4)	cone membership	`PowlReplayVerifier`, marking geometry
Arbitration	lexicographic order	`CostVector`, `route_capability_plan`
Makespan / cost	critical-path functional	`ScheduleAnalysis64`

1.2 The two theses of this pillar, stated once

Manufacture is bounded search over a finite ground net. A lifted PDDL domain with numeric fluents grounds to a *finite* set of transitions (Theorem **??**), so μ's search terminates with a priori bounded cost — the boundedness law (M2) of the foundations, instantiated. The state it searches is a marking: a vector of atom-tokens plus a valuation of numeric fluents. The attention fluent, borrowed at an action's start and returned at its end, obeys a balance law whose nonnegativity is exactly schedule feasibility (Proposition **??**).

Conformance is distance to a cone, certified by a hyperplane. The reachable markings of a net are constrained by the state equation $\boldsymbol{m} = \boldsymbol{m}_0 + \mathbf{N}\boldsymbol{x}$ to lie in a translated cone (Proposition **??**). A trace conforms only if its marking lies there; a nonconformant marking is separated from the cone by a Farkas hyperplane (Theorem ??), a certificate of dimension p that a bounded verifier checks without replaying the interior. Token-replay fitness measures the normalized excursion outside the enabled set, and equals 1 exactly on genuine firing sequences (Theorem ??).

Throughout, a claim is a definition, a theorem with proof, or a citation. The prior published paper [?] demonstrated $\mathcal{A} = \mu(\mathcal{O})$ at enterprise scale and full coverage of the forty-three workflow patterns [?]; this pillar supplies the planning and conformance geometry those numbers rested on.

Chapter 2

The PDDL Action Law with Numeric Fluents

2.1 Lifted domains, grounding, and the state

We take the numeric-temporal fragment of PDDL 2.1 [?] as the language of μ when it manufactures missions, and give it exactly the structure `bcinr-pddl` grounds and searches.

Definition 2.1 (Lifted numeric-temporal domain). A *lifted domain* is a tuple $\mathcal{D} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{S})$: a finite type hierarchy $\mathcal{T}$ (a forest under a subtype relation, with universal root `object`); a finite set $\mathcal{P}$ of predicate symbols with typed arities; a finite set $\mathcal{F}$ of numeric *function* symbols (fluents) with typed arities; and a finite set $\mathcal{S}$ of action schemas. A *durative* schema $s \in \mathcal{S}$ carries typed parameters $\bar{v}$, a duration constraint, a condition C_s, and an effect E_s, where conditions and effects are timestamped `at start` / `at end` and may be atomic (over $\mathcal{P}$) or numeric comparisons/updates (over $\mathcal{F}$).

Definition 2.2 (Grounded temporal problem). Given a domain $\mathcal{D}$ and a problem $(\mathcal{O}\lfloor, \boldsymbol{m}_0, \nu_0, \gamma)$ with finite typed objects $\mathcal{O}\lfloor$, initial atom set $\boldsymbol{m}_0$, initial fluent valuation $\nu_0 : \mathcal{F}\text{-instances} \to \mathbb{R}$, and goal condition γ, the *grounding* substitutes every type-compatible object tuple into each schema's parameters, producing a finite set of ground actions. A grounded state is a pair $(\boldsymbol{m}, \nu)$ of a set $\boldsymbol{m}$ of true ground atoms and a fluent valuation ν.

This is the `GroundTemporalProblem` of `bcinr-pddl`: `initial_atoms`, `initial_fn_values`, grounded `durative_actions`, and a goal `PddlCondition`. Type compatibility is the `TypeIndex::satisfies` walk up the parent chain; the universal type `object` matches every object.

Theorem 2.1 (Grounding is bounded). *Let $\mathcal{D}$ have schemas of maximum parameter arity k over $|\mathcal{O}\lfloor| = N$ objects. The number of ground actions is at most $|\mathcal{S}| \cdot N^k$, a finite constant independent of the goal. Consequently a forward search over grounded actions has a branching factor bounded a priori, and μ's manufacture terminates within a fixed depth bound.*

Proof. Each schema binds at most k parameters, each to one of $\leq N$ objects, so it grounds to at most N^k actions; summing over $|\mathcal{S}|$ schemas gives the bound. In `bcinr-pddl` the count is additionally capped by the structural constant `PDDL8_MAX_GROUND`: exceeding it is the hard error `Pddl8Error::BoundExceeded`, and an empty grounding is `EmptyGrounding`. Thus the ground action set is finite and its size is a fixed function of $(|\mathcal{S}|, N, k)$, discharging the boundedness law (M2) for mission manufacture. □

Remark (The forward search is the manufacture)*.* For the classical (atomic) fragment, `GroundProblem::find_plan` is breadth-first search over sets of ground atoms: the frontier is a queue of (state, path) pairs, visited states are deduplicated, and the first state containing the goal set returns its path as a `Pddl8Tape`. Search failure within `PDDL8_MAX_PLAN_DEPTH` is `NoAdmittedPlan` — a *refusal with reason*, not an exception, exactly as admission returns $\perp$. The manufacture either yields an artifact or a receipted refusal; it never diverges, by Theorem **??**.

2.1.1 Relational Grounding and XOR Filter Membership Gates

To avoid the naive grounding parameter explosion, grounding is refactored into a relational database join query over type-safe dictionary-encoded symbol IDs: $\mathrm{SymId} \in \mathbb{Z}_{>0}$.

Construction 2.1 (XOR-Filter-Pruned Membership Gate)**.** Let $R \subseteq \mathcal{O}$ be the reachability fixpoint fact set. A frozen fact store builds an 8-bit XOR filter over FNV-1a hashes of R. Index mapping uses the fastrange reduction:

$$\mathrm{reduce}(x, n) \;=\; \frac{x \times n}{2^{32}}, \tag{2.1}$$

which projects hash x onto n blocks using multiplication and bit-shifts. Let F be the filter and B the sorted BTreeSet of R. Membership is gated by:

$$\mathrm{contains}(x) \;=\; \begin{cases} x \in B, & \text{if } F(x) = \text{true}, \\ \text{false}, & \text{otherwise.} \end{cases}$$

The false-positive rate is $\approx 0.4\%$, and false negatives are 0.

2.2 Numeric fluents and the attention balance law

The distinctive content of the numeric fragment is that an action may test and update a real-valued fluent. `bcinr-pddl`'s `eval_numeric` evaluates a fluent expression against a valuation, and `apply_numeric_effect` realizes the five update kinds `Assign`, `Increase`, `Decrease`, `ScaleUp`, `ScaleDown`. We formalize the one that carries the calculus of attention.

Definition 2.3 (Numeric fluent balance)**.** A durative action j *draws* on a fluent ϕ if its effect contains `at start (decrease` ϕ r_j`)` and `at end (increase` ϕ r_j`)` for a rate $r_j \geq 0$, and its condition contains `at start` $(\phi \geq r_j)$. Over the half-open interval $[s_j, e_j)$ of its execution, j holds r_j units of ϕ and releases them at e_j. The *free level* of ϕ at time t is

$$f_\phi(t) \;=\; \nu_0(\phi) \;-\; \sum_{j:\, t \in [s_j, e_j)} r_j.$$

This is exactly the capability router's model: its domain declares `(:functions (attention))`; each capability schema decrements `attention` by 1 `at start` under the guard `(>= (attention) 1)` and increments it back `at end`. Attention is "one working human, one active train of thought."

Proposition 2.2 (Attention is borrowed and returned: conservation)**.** *If every action drawing on ϕ both decrements ϕ by r_j at start and increments it by r_j at end, then the net effect of each completed action on ϕ is zero, and the terminal valuation equals the initial: $\nu_{end}(\phi) = \nu_0(\phi)$. Hence ϕ is a conserved resource; scheduling redistributes* when *it is held, never* how much *exists.*

Proof. By `apply_numeric_effect`, `Decrease`(ϕ, r_j) then `Increase`(ϕ, r_j) compose to the identity on $\nu(\phi)$. Summing over all completed actions, the total change is $\sum_j (r_j - r_j) = 0$, so $\nu_{\text{end}}(\phi) = \nu_0(\phi)$. The intermediate values differ only on the union of the open execution intervals, which is the content of Definition **??**. ☐

Proposition 2.3 (Feasibility is pointwise nonnegativity)**.** *A durative-action schedule is feasible with respect to ϕ — every action's start guard ($\phi \geq r_j$) holds when it fires — if and only if $f_\phi(t) \geq 0$ for all t; equivalently, at no instant does the total rate of in-flight draws exceed $\nu_0(\phi)$. For the unit-rate attention fluent ($r_j \equiv 1$), this says the number of concurrently in-flight capabilities never exceeds the capacity $\nu_0($`attention`$)$.*

Proof. The start guard at s_j requires $\nu(\phi) \geq r_j$ *in the state just before j's at-start decrement.* By Definition **??**, that state's value of ϕ is $f_\phi(s_j^-) = \nu_0(\phi) - \sum_{i \neq j:\, s_j \in [s_i, e_i)} r_i$, so the guard holds iff $f_\phi(s_j^-) \geq r_j$, i.e. $f_\phi(s_j) \geq 0$ after j's own draw is included. Free level changes only at action starts and ends, so if it is ≥ 0 at every start it is ≥ 0 everywhere. Conversely a violated guard is an instant with $f_\phi < 0$. For $r_j \equiv 1$, the sum counts in-flight actions, so nonnegativity is exactly "concurrency $\leq$ capacity." ☐

`bcinr-pddl`'s temporal planner enforces exactly this: within a tick it re-scans candidate actions after each start so an `at start` decrement is visible to the next candidate's guard in the same instant, which the source comments identify as "what lets concurrent starts correctly gate on shared resource fluents." The `zero_attention_capacity_is_infeasible_and_refused` test is Proposition **??** at capacity 0: with $f_{\texttt{attention}}(t) < 0$ forced at any start, the router refuses rather than defaults a route.

Chapter 3

The Marking Polytope and the State Equation

3.1 Place/transition nets and the incidence matrix

Atomic (non-numeric) execution has a native linear-algebraic geometry, that of place/transition nets [?]. We reproduce it in the form the code inhabits.

Definition 3.1 (Net, marking, enabling, firing)**.** A *net* has p places and a finite set T of transitions. A *marking* is a vector $\boldsymbol{m} \in \mathbb{Z}^p_{\geq 0}$. A transition t is a pair $(\boldsymbol{m}_t^-, \boldsymbol{m}_t^+)$ of a *preset* (consumed) and *postset* (produced) vector. t is *enabled* at $\boldsymbol{m}$ iff $\boldsymbol{m} \geq \boldsymbol{m}_t^-$ coordinatewise; firing it yields $\boldsymbol{m}' = \boldsymbol{m} - \boldsymbol{m}_t^- + \boldsymbol{m}_t^+$.

Definition 3.2 (Incidence matrix and state equation)**.** Let $\mathbf{N} \in \mathbb{Z}^{p\times|T|}$ have column $\boldsymbol{\delta}_t = \boldsymbol{m}_t^+ - \boldsymbol{m}_t^-$ for each transition t. A firing sequence ς with *Parikh vector* $\boldsymbol{x} \in \mathbb{Z}^{|T|}_{\geq 0}$ (the count of each transition's firings) satisfies the *state equation*

$$\boldsymbol{m} \;=\; \boldsymbol{m}_0 + \mathbf{N}\boldsymbol{x}. \tag{3.1}$$

Proposition 3.1 (Reachability lies in a translated cone)**.** *Every marking $\boldsymbol{m}$ reachable from $\boldsymbol{m}_0$ satisfies* (**??**) *for some $\boldsymbol{x} \in \mathbb{Z}^{|T|}_{\geq 0}$, hence lies in the integer points of $\boldsymbol{m}_0 + \mathrm{cone}(\mathbf{N})$ intersected with $\mathbb{Z}^p_{\geq 0}$, where $\mathrm{cone}(\mathbf{N}) = \{\mathbf{N}\boldsymbol{x} : \boldsymbol{x} \geq \mathbf{0}\}$. The state equation is a* necessary *condition for reachability; it is not sufficient (a $\boldsymbol{x}$ solving* (**??**) *need not correspond to a fireable ordering).*

Proof. Firing t once adds column $\boldsymbol{\delta}_t$ to the marking (Definition **??**); a sequence firing each t exactly x_t times adds $\sum_t x_t\boldsymbol{\delta}_t = \mathbf{N}\boldsymbol{x}$, giving (**??**). Since each $x_t \geq 0$ and markings are nonnegative integers, $\boldsymbol{m}$ is an integer point of $\boldsymbol{m}_0 + \mathrm{cone}(\mathbf{N})$ in the nonnegative orthant. Insufficiency is the classical Petri-net fact [?]: (**??**) ignores intermediate nonnegativity and firing order, so spurious solutions exist. □

Definition 3.3 (Marking polytope)**.** The *marking polytope* of a net is the relaxation

$$\mathcal{P} \;=\; \big\{\boldsymbol{m}_0 + \mathbf{N}\boldsymbol{x} \;:\; \boldsymbol{x} \geq \mathbf{0}\big\} \cap \{\boldsymbol{m} \geq \mathbf{0}\} \;\subseteq\; \mathbb{R}^p_{\geq 0},$$

the continuous translated cone whose integer points over-approximate the reachable set.

3.2 STRIPS grounding is a net; the lifecycle token model is its safe specialization

Construction 3.1 (STRIPS grounding as a net)**.** Take the places to be the ground atoms. A ground action t with delete-effects $\boldsymbol{m}_t^-$ and add-effects $\boldsymbol{m}_t^+$ is a transition; its precondition atoms must all be present for it to fire (an enabling condition refining Definition **??**). Then `GroundProblem::find_plan` searches marking space: the BFS frontier is a set of markings, the successor relation is exactly transition firing (`for d in del_effects { next.remove(d) }; for a in add_effects { next.insert(a) }`), and a plan is a path to a marking containing the goal.

The lifecycle model of the foundations paper — the fixed sequence `judge` → `admit` → `receipt` — is a concrete net whose places are the token bits of `replay_adapter`: `TOK_START`, `TOK_JUDGED`, `TOK_ADMITTED`, `TOK_DONE`. Each step's `required_tokens` is its preset and `produces_tokens` its postset:

$$\texttt{judge}: \texttt{TOK\_START} \to \texttt{TOK\_JUDGED}, \quad \texttt{admit}: \texttt{TOK\_JUDGED} \to \texttt{TOK\_ADMITTED}, \quad \texttt{receipt}: \texttt{TOK\_ADMITTED} \to \texttt{TOK\_DO}$$

Proposition 3.2 (The verifier's firing rule is the safe-net state equation)**.** *On a* safe *(1-bounded) net, where every marking and every preset/postset is a* $0/1$ *vector and presets are consumed, the integer firing rule* $\boldsymbol{m}' = \boldsymbol{m} - \boldsymbol{m}_t^- + \boldsymbol{m}_t^+$ *of Definition* **??** *coincides with the branchless bitset update the* `PowlReplayVerifier` *performs:*

$$\texttt{enabled\_tokens} \leftarrow (\texttt{enabled\_tokens} \,\&\, \neg\, \boldsymbol{m}_t^-) \mid \boldsymbol{m}_t^+,$$

and the enabling test $\boldsymbol{m} \geq \boldsymbol{m}_t^-$ *coincides with the branchless subset check* $(\texttt{enabled} \,\&\, \boldsymbol{m}_t^-) \oplus \boldsymbol{m}_t^- = 0$.

Proof. On $0/1$ markings with $\boldsymbol{m}_t^- \leq \boldsymbol{m}$, the AND-NOT `enabled` $\&\, \neg\, \boldsymbol{m}_t^-$ clears exactly the consumed places (subtracting $\boldsymbol{m}_t^-$), and the OR with $\boldsymbol{m}_t^+$ sets the produced places (adding $\boldsymbol{m}_t^+$); when preset and postset are disjoint from the retained marking this is bit-for-bit the integer expression. The enabling identity holds because $(\boldsymbol{m} \,\&\, \boldsymbol{m}^-) \oplus \boldsymbol{m}^- = 0$ iff every bit of $\boldsymbol{m}^-$ is also set in $\boldsymbol{m}$, i.e. $\boldsymbol{m} \geq \boldsymbol{m}^-$ coordinatewise on $0/1$ vectors. Both are the verifier's two guard lines verbatim. □

Proposition **??** is the bridge between the general Petri geometry of [**?**] and the constant-time bitwise mechanism the receipt path actually runs: the polytope theory is the semantics, the mask reduction is the implementation.

Chapter 4

Conformance as Cone Membership and Farkas Certificates

4.1 Conformance is membership

Definition 4.1 (Conformance)**.** A trace with final marking $\boldsymbol{m}^\dagger$ and transition-count vector $\boldsymbol{x}$ *conforms* to a net with initial marking $\boldsymbol{m}_0$ if $\boldsymbol{m}^\dagger \in \mathcal{P}$ and the $\boldsymbol{x}$ witnessing it is realized by a genuine firing ordering. A trace *fails membership* if $\boldsymbol{m}^\dagger \notin \mathcal{P}$, i.e. no $\boldsymbol{x} \geq \mathbf{0}$ solves $\mathbf{N}\boldsymbol{x} = \boldsymbol{m}^\dagger - \boldsymbol{m}_0$ within the nonnegative orthant.

Theorem 4.1 (Farkas separation certifies nonconformance)**.** *Fix $\boldsymbol{b} = \boldsymbol{m}^\dagger - \boldsymbol{m}_0$. Exactly one of the following holds:*

(a) there exists $\boldsymbol{x} \geq \mathbf{0}$ with $\mathbf{N}\boldsymbol{x} = \boldsymbol{b}$ (the continuous relaxation is feasible; membership in the cone is possible); or

(b) there exists a vector $\boldsymbol{y} \in \mathbb{R}^p$ with $\mathbf{N}^\top \boldsymbol{y} \leq \mathbf{0}$ and $\boldsymbol{y}^\top \boldsymbol{b} > 0$ — a separating hyperplane whose normal $\boldsymbol{y}$ certifies $\boldsymbol{m}^\dagger \notin \boldsymbol{m}_0 + \mathrm{cone}(\mathbf{N})$.

The certificate $\boldsymbol{y}$ is a sound proof of nonconformance*: if (b) holds, no firing sequence, in any order, reaches $\boldsymbol{m}^\dagger$ from $\boldsymbol{m}_0$.*

Proof. This is Farkas' lemma [**?**] applied to the system $\mathbf{N}\boldsymbol{x} = \boldsymbol{b},\ \boldsymbol{x} \geq \mathbf{0}$: either the system has a nonnegative solution, or there is $\boldsymbol{y}$ with $\boldsymbol{y}^\top \mathbf{N} \leq \mathbf{0}^\top$ and $\boldsymbol{y}^\top \boldsymbol{b} > 0$, and never both. In case (b), suppose for contradiction a firing sequence reached $\boldsymbol{m}^\dagger$; by Proposition **??** its Parikh vector $\boldsymbol{x}^\star \geq \mathbf{0}$ satisfies $\mathbf{N}\boldsymbol{x}^\star = \boldsymbol{b}$, so $\boldsymbol{y}^\top \boldsymbol{b} = \boldsymbol{y}^\top \mathbf{N}\boldsymbol{x}^\star = (\mathbf{N}^\top \boldsymbol{y})^\top \boldsymbol{x}^\star \leq 0$, contradicting $\boldsymbol{y}^\top \boldsymbol{b} > 0$. Hence (b) rules out reachability. Soundness is one-directional by Proposition **??**: feasibility of the relaxation (a) is necessary but not sufficient for an actual firing sequence, so (a) does not by itself certify conformance — only (b) certifies its negation. □

Corollary 4.2 (The certificate is bounded and verifiable at $O(\kappa)$)**.** *A nonconformance certificate is a single vector $\boldsymbol{y} \in \mathbb{R}^p$ together with the two inequalities $\mathbf{N}^\top \boldsymbol{y} \leq \mathbf{0}$ and $\boldsymbol{y}^\top \boldsymbol{b} > 0$. Checking it is a matrix product and two sign tests — independent of the length of the trace that produced $\boldsymbol{m}^\dagger$. Projected to the receipt's bounded coordinate (a verdict and a localized witness), it is verifiable within a bounded context κ, per the keystone's Comprehension–Verification Gap.*

Proof. Verification reads $\boldsymbol{y}$ and $\boldsymbol{b}$ and evaluates $\mathbf{N}^\top \boldsymbol{y}$ and $\boldsymbol{y}^\top \boldsymbol{b}$; the cost is $O(p \cdot |T|)$ in the net's fixed dimensions, with no dependence on trace length or interior computation. The receipt exposes the Boolean verdict and the witness coordinate, of size $\leq \kappa$. $\square$

Remark (Geometry is the process-layer projection principle). A conformance failure is not reported as "the trace looked wrong"; it is a hyperplane one can recompute. This is the projection principle in linear algebra: an unbounded execution is judged by a bounded certificate whose fidelity is mechanical, not a matter of the verifier comprehending the run.

Chapter 5

Token-Replay Fitness as Normalized Distance

5.1 The replay verifier as a Petri semantics

Token-replay conformance [?] plays a trace against a process model and measures how far it strays from lawful firing. `bcinr-powl`'s `PowlReplayVerifier` is a branchless realization; we give its exact semantics and prove what it characterizes.

Definition 5.1 (Replay state and step)**.** The verifier holds a marking $\boldsymbol{m}$ (`enabled_tokens`) initialized to the entry token, and accumulators `replayed`, `fitted`, `enabled_not_taken` (bitsets of node identities). A frame carries a one-hot `node_bit`, a preset `required_tokens` $= \boldsymbol{m}^-$, and a postset `produces_tokens` $= \boldsymbol{m}^+$. Replaying a frame:

(1) rejects a `node_bit` that is zero or not a power of two (`UnknownNode`);

(2) requires $\boldsymbol{m} \geq \boldsymbol{m}^-$ (else `TokenNotEnabled`), the branchless check of Proposition ??;

(3) records the enabled-but-unconsumed tokens `enabled_not_taken` $|= \boldsymbol{m} \,\&\, \neg \boldsymbol{m}^- \,\&\, \neg$ `node_bit`;

(4) fires: $\boldsymbol{m} \leftarrow (\boldsymbol{m} \,\&\, \neg \boldsymbol{m}^-) \mid \boldsymbol{m}^+$; and

(5) sets the `node_bit` in both `replayed` and `fitted`.

Definition 5.2 (Conformance fitness)**.** On finalization the verifier returns, in Q16.16 fixed point,

$$\varphi \;=\; \frac{|\texttt{fitted}|}{|\texttt{replayed}|}, \qquad \text{precision} \;=\; \frac{|\texttt{replayed}|}{|\texttt{replayed}| + |\texttt{enabled\_not\_taken}|},$$

where $|\cdot|$ is population count and $\varphi := 1$ when the denominator is zero. The value 1.0 is the constant `0x0001_0000`.

Theorem 5.1 (Unit fitness characterizes genuine firing sequences; the fault localizes)**.** *Replaying a sequence of frames against the net either (i) completes with no violation, in which case the sequence is a genuine firing sequence — each transition was enabled at the marking where it fired — and $\varphi = 1$; or (ii) halts at the first frame whose preset is not contained in the current marking, returning `TokenNotEnabled`$\{$node_id$\}$, a coordinate-localized witness of the earliest disabled transition.*

Proof. Step (2) of Definition **??** admits a frame iff $\boldsymbol{m} \geq \boldsymbol{m}^-$, which by Definition **??** is exactly the enabling condition; step (4) is the firing rule (Proposition **??**). By induction on the prefix, if no frame is rejected then every frame fired from a marking at which it was enabled, so the whole sequence is a genuine firing sequence and the intermediate markings are reachable. In that case every replayed frame also sets `fitted` (step 5), so $|$`fitted`$| = |$`replayed`$|$ and $\varphi = 1$. Conversely the guard in step (2) fails at the *first* frame whose required tokens are absent — the verifier returns immediately with that frame's `node_id` — so a non-firing sequence is rejected at the earliest offending index, which is the localized witness $t^\star$. □

Proposition 5.2 (Fitness and precision are honest population ratios)**.** *φ and precision are ratios of bit-populations of disjointly-maintained bitsets; neither can exceed* 1*, and $\varphi = 1$ is attained exactly on completed replays (Theorem* **??***). The* ***`enabled_not_taken`*** *accumulator, gathered* before *consumption in step (3), counts tokens that were live but not used by the firing — the model's under-fitting — and feeds precision, not fitness; the two metrics therefore measure orthogonal deviations and cannot be traded against each other.*

Proof. Population count is monotone and bounded by the number of node bits, and `fitted` $\subseteq$ `replayed` by construction (step 5 sets both), so $\varphi \leq 1$ with equality iff no frame was replayed-without-fitting — which, in the current step, is every completed replay. `enabled_not_taken` is updated only in step (3) and read only by precision's denominator, so it is independent of the fitness numerator. □

The lifecycle instance makes this concrete. `replay_lifecycle` on the in-order sequence [`Judged`, `Admitted`, `Receipted`] returns $\varphi =$ `0x0001_0000`; the out-of-order [`Admitted`, ...] fails at the `Admitted` frame, which requires `TOK_JUDGED` that the entry marking lacks, returning `TokenNotEnabled`$\{node\_id : 2\}$; and skipping `admit` before `receipt` fails at node 3. These are the three tests `lawful_in_order_sequence_has_fitness_one`, `out_of_order_sequence_is_token_not_enabled`, and `skipping_admit_before_receipt_is_token_not_enabled` — Theorem **??**'s two branches, executed.

Remark (Relation to the keystone's fractional fitness)*.* The keystone defines $\varphi = 1-$(tokens forced on unenabled tr
a fraction in $[0, 1]$. The `praxis` lifecycle verifier is the strict specialization in which a forced-disabled firing is a hard `TokenNotEnabled` rather than a fractional penalty: the "forced consumption" branch is never taken because the verifier refuses to fire a disabled transition at all. Both characterize the same lawful set $\{\varphi = 1\}$; the specialization is more conservative — it localizes the first fault and stops — which is the desired behavior for a lifecycle whose only lawful order is a single line.

Chapter 6

POWL 2.0: Choice Graphs and Partial Orders

6.1 The language

Partially Ordered Workflow Language (POWL) [**?**] is the model class into which the planner's output is compiled for replay. We define it as the recursive structure the code builds.

Definition 6.1 (POWL model)**.** A *POWL model* over an activity alphabet A is one of:

(i) an *activity* $a \in A$ (a leaf);

(ii) a *partial order* $(\{M_1, \dots, M_k\}, \prec)$: a finite set of child POWL models with an irreflexive, acyclic precedence relation $\prec$, executed by respecting $\prec$ and running $\prec$-incomparable children concurrently; or

(iii) a *choice graph* $(\{M_1, \dots, M_k\}, E)$: children combined by exclusive choice (and, in POWL 2.0, cyclic/loop edges E), of which the execution takes exactly one live branch.

The three node kinds are `PowlOpKind::Activity`, `PartialOrderGate`, `ChoiceGate` in `powl_bridge`.

Construction 6.1 (Plan to POWL tape; transitive reduction)**.** `temporal_plan_to_powl_tape` converts a `TemporalPlan` into a tape of `PowlOpSpecs`. Each step i becomes an activity with a `pred_mask` initialized to the set of steps $j < i$ that finish before i starts (`prev_end` $\leq$ $\texttt{start}_i$), and a one-hot `succ_mask` $= 1 \ll i$. A second pass performs *transitive reduction*: for each predecessor k of i, the bits of k's own predecessors are cleared from i's mask, leaving only *direct* predecessors.

Proposition 6.1 (The reduced masks are the Hasse diagram of the precedence order)**.** *Let $\prec$ be the strict order "j finishes before i starts" on plan steps. The initialized* `pred_mask` *of step i is $\{j : j \prec i\}$, and after transitive reduction it is the covering relation of $\prec$: j remains a predecessor of i iff $j \prec i$ and there is no k with $j \prec k \prec i$. The resulting dependency graph is a DAG, and two steps are schedulable concurrently iff they are $\prec$-incomparable.*

Proof. $\prec$ is irreflexive and transitive (it is induced by the total order on real finish/start times), hence a strict partial order. The first pass sets bit j in i's mask exactly when $j \prec i$. The reduction clears bit j from i's mask when some retained predecessor k of i has j in *its* predecessors, i.e. $j \prec k \prec i$; what survives is precisely the covering relation (the Hasse diagram) of $\prec$. Acyclicity follows from irreflexivity and transitivity. Incomparable steps have no path between them, so the scheduler may overlap their intervals; comparable steps are ordered by $\prec$. □

Proposition 6.2 (Local marking dimension is bounded by node arity)**.** *In a POWL model, the token marking local to a node ranges over the tokens of that node's immediate children: a partial-order or choice node of arity k has a local marking space of dimension $O(k)$, independent of the total size of the model. Consequently the replay of a node is checked in a working set bounded by its arity, and the global replay is the composition of these bounded-dimension checks.*

Proof. By Definition **??** a node's execution semantics reference only its own children's tokens: a partial order gates on the $\prec$-predecessors among its k children; a choice gates on which of its k branches is live. The token bits touched by one node's firing are therefore contained in an $O(k)$-dimensional subspace of the marking, regardless of how large the surrounding model is. The bounded-arity constant is the `PowlOpSpec` tape's per-op mask width. □

Proposition **??** is the structural reason conformance is cheap: even though the mission's interior may be arbitrarily large, each POWL node is judged in a working set bounded by its arity — the projection principle, localized to the process tree.

Chapter 7

Separability as a Rice Quarantine for Processes

7.1 The separable decomposition

Not every workflow net is a POWL model; the ones that are, are exactly the *separable* ones, and this is a decidable structural property.

Theorem 7.1 (Separable decomposition, after Kourani–van der Aalst)**.** *A sound, separable workflow net admits a recursive decomposition into a POWL model (Definition* **??***) whose every internal node is a partial order (all concurrent / sequential logic) or a choice graph (all exclusive / cyclic logic), and whose leaves are activities. Each node's local marking geometry has dimension bounded by its arity (Proposition* **??***), independent of the global net size.*

Attribution. This is the decomposition established for POWL by Kourani and van der Aalst and colleagues [**?**], resting on van der Aalst's process-mining foundations [**?**]. We cite it rather than reprove it; the contribution here is the reframing below. □

Proposition 7.2 (Separability is the process-layer Rice quarantine)**.** *Let sep* : $\{$*workflow nets*$\} \to \{0, 1\}$ *be the (decidable) predicate "the net is sound and separable." Then sep is a legitimate admission obligation g in the sense of the foundations paper: it is total and computable, so admitting a process by sep retracts the space of arbitrary control-flow onto the decidable sub-language of POWL-expressible processes. A separable net is* admitted *(it decomposes, and its conformance is the bounded-arity replay of Chapter* **??***); a non-separable net is* refused with reason *— the reason being the structural obstruction to decomposition.*

Proof. Soundness and separability are structural properties of a finite net, checkable by the decomposition procedure of Theorem **??**, which either returns a POWL tree or the node at which decomposition fails. Thus sep is total and computable, satisfying the obligation criterion (a syntactic terminating check, never a semantic decision about what the process "means"). Admission maps the net to its POWL decomposition when sep $= 1$ and to $\perp$ with the obstruction as refusal data otherwise, which is exactly the admission map α specialized to processes. □

Remark (The parallel is exact)*.* The foundations paper retracted the undecidable observation space $\mathcal{O}$ onto a decidable admitted language $\mathcal{O}^*$ because Rice's theorem forbids deciding meaning. Here an unwieldy space of arbitrary control-flow is retracted onto the decidable sub-language of separable (POWL) processes because only there is conformance a bounded-arity replay. In both cases: a large, badly-behaved space is replaced by a small decidable sub-language whose membership is mechanically certified, and the boundary certificate — a POWL tree, a fitness value, a Farkas hyperplane — is small. Separability *is* admission, for processes.

Construction 7.1 (Sub-Net Normalization Interface)**.** Given a partition part $T' \subseteq T$ with entry places P_{in} and exit places P_{out}, the sub-net is normalized to a valid WF-net by adding fresh boundary places p_s, p_e and silent transitions $\tau_{\text{in}}, \tau_{\text{out}}$. The normalized flow relation is:

$$F_{\text{norm}} \;=\; F' \;\cup\; \{(\text{src}, \tau_{\text{in}}), (\tau_{\text{in}}, p_s), (p_e, \tau_{\text{out}}), (\tau_{\text{out}}, \text{snk})\}, \tag{7.1}$$

redirecting boundary arcs onto the unified source src and sink snk.

Theorem 7.3 (Recursive Language Correctness)**.** *Let N be a safe & sound workflow net, and let $\psi = convert(N)$ be its POWL 2.0 conversion. Then for any prefix trace length k:*

$$\mathcal{L}_k(N) \;=\; \mathcal{L}_k(\psi) \;=\; \mathcal{L}_k\big(recompose(\psi)\big).$$

Chapter 8

Cost Arbitration and the Makespan Functional

8.1 The cost vector and lexicographic order

When several admitted plans achieve a goal, the router must choose one deterministically. The choice is a total order on a cost vector.

Definition 8.1 (Cost vector)**.** The router's `CostVector` is the tuple

$$\boldsymbol{c} = (c_0, c_1, c_2, c_3, c_4, c_5) = \big(\overline{\texttt{admitted}}, \texttt{risk}, \texttt{attention\_seconds}, \texttt{tokens}, \texttt{latency}, \texttt{switches}\big),$$

where $c_0 = \overline{\texttt{admitted}}$ is the unadmitted indicator ($0 =$ admitted, and admitted sorts first), $c_1 =$ `unreceipted_mutation_risk` $\in \{0, \ldots, 255\}$, $c_2 =$ `human_attention_seconds` (the makespan, §??), $c_3 =$ `token_cost`, $c_4 =$ `latency_ms`, and $c_5 =$ `context_switches` $\in \{0, \ldots, 255\}$ (distinct capabilities used). The order is lexicographic, cheapest-first, and the refused vector is `refused`$() = (\text{unadmitted}, 255, \infty, \infty, \infty, 255)$, the top element.

Theorem 8.1 (Lexicographic order is the infinite-weight limit)**.** *Let the secondary coordinates be bounded, $|c_i| \le M$ for $i \ge 1$ over the admitted plans (the risk and switch coordinates are ≤ 255; the makespan, latency, and token coordinates are bounded by the structural depth and duration constants of Theorem **??**). Define the scalarization $C_\lambda(\boldsymbol{c}) = \sum_{i=0}^{5} \lambda^{5-i} c_i$ with $\lambda > 1$. Then there is a threshold Λ such that for all $\lambda > \Lambda$,*

$$\boldsymbol{c} \preceq_{\text{lex}} \boldsymbol{c}' \iff C_\lambda(\boldsymbol{c}) \le C_\lambda(\boldsymbol{c}').$$

As $\lambda \to \infty$ the weight λ^5 on the admitted coordinate c_0 diverges relative to all others.

Proof. If $\boldsymbol{c} \prec_{\text{lex}} \boldsymbol{c}'$ they first differ at some coordinate p with $c'_p - c_p \ge \epsilon > 0$ (the coordinates are integers or bounded reals with a least positive gap ϵ on the admitted, risk, and switch coordinates; for the continuous coordinates take ϵ the observed resolution). Then

$$C_\lambda(\boldsymbol{c}') - C_\lambda(\boldsymbol{c}) \;\ge\; \lambda^{5-p}\epsilon - 2M \sum_{i>p} \lambda^{5-i} \;=\; \lambda^{5-p}\epsilon - 2M\,\frac{\lambda^{5-p} - 1}{\lambda - 1},$$

which is positive once $\lambda > \Lambda := 1 + 2M \cdot 6/\epsilon$. Ties in all coordinates map to equality. Hence for $\lambda > \Lambda$ the scalarization order agrees with $\preceq_{\text{lex}}$. The admitted coordinate carries weight λ^5, dominating the others' λ^{5-i} as $\lambda \to \infty$. $\square$

Corollary 8.2 (Admitted dominates: no finite cost buys a refused route)**.** *An admitted plan sorts strictly before any refused plan, regardless of its secondary costs: for any finite risk, makespan, token, latency, and switch values, an admitted* `CostVector` *is* $\prec_{\text{lex}}$ *the refused top element. Equivalently, the price of unlawfulness is infinite in the limit that defines the order.*

Proof. The lead coordinate c_0 is 0 for admitted and 1 for refused, and by Theorem **??** it dominates: any admitted vector differs from the refused element first at c_0, where it is strictly smaller. Concretely, `CostVector`'s `Ord` places `admitted` = *true* ahead by reversing the boolean comparison, then breaks ties down the remaining coordinates; the `refused`() constructor sets c_0 to unadmitted and every secondary coordinate to its maximum (∞ for the reals, `u8::MAX`/`u64::MAX` for the integers), so it is the unique top. This is the test `cost_vector_orders_admitted_before_refused`. □

Remark (The router's determinism is a receipt)*.* `route_capability_plan` binds the problem text, the plan's steps, and the cost into a BLAKE3 `route_chain` [?]; identical tasks produce identical chains (`routing_is_deterministic_same_task_same_route`). The arbitration is thus not a heuristic tie-break but a receipted, replayable choice — the cost order is a function, and its value is committed.

8.1.1 Additive Separation of Maximum Reachable Revenue

Evaluating the global maximum of realizable revenue across accounts would normally result in exponential search complexity due to combinatorial action options.

Theorem 8.3 (Additive Separation of MRR)**.** *Let A be a set of independent client accounts. Let realized(a) be the revenue of account a under a target stage, and let lawful_targets(a) be its evidence-gated target stages. The joint plan optimization decomposes linearly:*

$$\max_{\textit{joint plan}} \sum_{a\in A} \textit{realized}(a) \;=\; \sum_{a\in A} \max_{t\in \textit{lawful\_targets}(a)} \textit{realized}(a,t). \tag{8.1}$$

Consequently, the search complexity is reduced from $\Omega(\prod_a |T_a|)$ to $O(|A|\cdot|T_a|)$ linear sum.

8.2 The makespan functional and critical-path structure

The primary continuous cost coordinate c_2 is the makespan, and its structure is what `analyze_schedule` computes over the plan's POWL tape.

Definition 8.2 (Makespan functional; forward and backward pass)**.** Given the precedence DAG of Construction **??** with per-op durations d_i and release times, the *forward pass* computes earliest starts and finishes

$$\mathrm{es}_i = \max\Big(r_i,\ \max_{j\prec i} \mathrm{ef}_j\Big), \qquad \mathrm{ef}_i = \mathrm{es}_i + d_i,$$

the *makespan* is $T = \max_i \mathrm{ef}_i$, and the *backward pass* computes latest starts $\mathrm{ls}_i = \mathrm{lf}_i - d_i$ with $\mathrm{lf}_i = \min_{i\prec k} \mathrm{ls}_k$ (or T at sinks). The *slack* of op i is $\mathrm{ls}_i - \mathrm{es}_i$, and the *critical path* is the set of zero-slack ops.

Proposition 8.4 (Makespan is the longest path; the critical path is zero-slack)**.** *The makespan T equals the length of the longest ($\prec$-)path in the precedence DAG weighted by durations, and an op has zero slack iff it lies on some longest path. Delaying a zero-slack op delays the makespan; delaying an op by less than its slack does not.*

Proof. The forward recursion $\mathrm{ef}_i = \mathrm{es}_i + d_i$ with $\mathrm{es}_i = \max_{j \prec i} \mathrm{ef}_j$ is the standard longest-path dynamic program over a DAG (topologically ordered here because op i's predecessors have indices $< i$), so ef_i is the longest weighted path ending at i and $T = \max_i \mathrm{ef}_i$ is the overall longest path. The backward pass gives the latest each op may start without increasing T; slack $\mathrm{ls}_i - \mathrm{es}_i = 0$ iff i cannot move, i.e. it lies on a longest path. This is exactly `forward_pass`, `backward_pass`, and the `critical_path_mask` (bit i set iff slack $< 10^{-9}$) of `ScheduleAnalysis64`. □

Proposition 8.5 (Capacity sensitivity is a finite difference of the found schedule)**.** *Let $T(\phi = c)$ be the makespan of the schedule the planner finds when fluent ϕ's initial capacity is c. The* capacity delta *reports $\big(T(c-1), T(c), T(c+1)\big)$, and ϕ is* binding *iff $T(c-1) \neq T(c)$ (or the problem is infeasible at $c-1$). This is a finite-difference sensitivity of the* specific schedule the greedy planner produced*, not a dual shadow price of an optimal scheduler.*

Proof. `replan_with_perturbed_capacity` re-solves `find_temporal_plan` with ϕ's initial value moved by ± 1 (clamped at 0) via `find_temporal_plan_with_fn_overrides`, and reads off the resulting makespan; `binding_resource_mask` sets bit i iff the -1 perturbation changed the makespan or made the problem infeasible. Because `find_temporal_plan` is a greedy forward-chaining planner (its own module comment disclaims optimality), the delta is the response of that found schedule, which is the honest claim the code makes — and no more. Infeasibility at $c - 1$ is itself evidence the resource binds, per Proposition **??**. □

Remark (The functional closes the loop)*.* The makespan feeds back into arbitration: $c_2 =$ `human_attention_seconds` $=$ `analysis.makespan` (`CostVector::from_analysis`), so the primary continuous cost the lexicographic order minimizes is the longest-path length of Proposition **??**. Attention conservation (Proposition **??**) says the total attention spent is schedule-invariant; the makespan functional says only its *completion time* is what arbitration optimizes — exactly the keystone's "reordering redistributes when attention is spent, not how much."

Chapter 9

Conclusion

We gave the geometry beneath two of the enforced equation's invariants. Mission manufacture is bounded search over a finite ground net (Theorem **??**); the state it searches is a marking with a numeric fluent valuation, and the attention fluent, borrowed and returned, obeys a balance law whose nonnegativity is feasibility (Propositions **??**–**??**). The reachable markings lie in a translated cone (Proposition **??**), the safe-net specialization of which is the branchless firing rule the receipt path runs (Proposition **??**). Conformance is membership in that cone, and a nonconformant trace is separated by a Farkas hyperplane — a bounded, sound certificate verifiable at $O(\kappa)$ (Theorem **??**, Corollary **??**). Token-replay fitness equals 1 exactly on genuine firing sequences, with the first disabled transition a localized witness (Theorem **??**), and it is an honest population ratio (Proposition **??**). POWL 2.0 gives the recursive partial orders and choice graphs (Definition **??**); the plan-to-tape transitive reduction is the Hasse diagram of the precedence order (Proposition **??**), and each node is judged in a working set bounded by its arity (Proposition **??**). The Kourani–van der Aalst separable decomposition (Theorem **??**) is the Rice quarantine for processes: separable is admitted, non-separable is refused with reason (Proposition **??**). And cost arbitration is the infinite-weight limit in which the admitted coordinate dominates (Theorem **??**, Corollary **??**), over a makespan functional that is the longest path of the precedence DAG (Proposition **??**).

The standard was never comprehension of the mission's interior. It is a bounded search, a membership test with a small certificate, and a fitness value of one — each guaranteed by mechanism, each small enough to hold.

Conformance is distance to a cone; the certificate is a hyperplane.

Appendix A

Notation

Symbol	Meaning
$\boldsymbol{m},\ \boldsymbol{m}_0$	marking; initial marking (vectors in $\mathbb{Z}^p_{\geq 0}$)
$\boldsymbol{m}_t^-,\ \boldsymbol{m}_t^+$	preset (consumed) / postset (produced) of transition t
$\mathbf{N},\ \boldsymbol{x}$	incidence matrix; Parikh (firing-count) vector
$\mathcal{P},\ \mathrm{cone}(\mathbf{N})$	marking polytope; reachability cone
$\boldsymbol{y}$	Farkas separating-hyperplane normal (nonconformance certificate)
φ	token-replay fitness $\in [0,1]$ (Q16.16; 1.0 = `0x0001_0000`)
$f_\phi(t)$	free level of numeric fluent ϕ at time t
$T,\ \mathrm{es}_i,\ \mathrm{ls}_i$	makespan; earliest / latest start of op i
$\boldsymbol{c},\ \preceq_{\mathrm{lex}}$	cost vector; lexicographic cost order
$\mu,\ \mathcal{O}^*,\ \perp,\ \kappa$	manufacturing morphism; admitted space; refusal; verifier context

Appendix B

Code Correspondence

Every object above is anchored to the `bcinr-pddl` / `bcinr-powl` / `praxis` source.

Mathematical object	Code artifact	Location
Grounded temporal problem	`GroundTemporalProblem::build`	`bcinr-pddl/src/ground.rs`
Bounded grounding	`PDDL8_MAX_GROUND`, `BoundExceeded`	`bcinr-pddl/src/ground.rs`
Forward search (manufacture)	`GroundProblem::find_plan` (BFS)	`bcinr-pddl/src/ground.rs`
Numeric fluent update	`eval_numeric`, `apply_numeric_effect`	`bcinr-pddl/src/ground.rs`
Attention fluent	`(:functions (attention))` domain	`bcinr-pddl/src/capability_ro`
Firing rule (safe net)	`PowlReplayVerifier::replay_frame`	`bcinr-powl-receipt/src/repl`
Fitness / precision	`ConformanceMetrics`, `finalize`	`bcinr-powl-receipt/src/repl`
Lifecycle token net	`TOK_*`, `replay_lifecycle`	`praxis-core/src/replay_adapt`
POWL tape / reduction	`temporal_plan_to_powl_tape`	`bcinr-pddl/src/powl_bridge.r`
Cost vector / order	`CostVector`, `impl Ord`	`bcinr-pddl/src/capability_ro`
Route receipt	`route_capability_plan`	`bcinr-pddl/src/capability_ro`
Makespan / critical path	`analyze_schedule`, `ScheduleAnalysis64`	`bcinr-pddl/src/schedule_anal`
Capacity sensitivity	`replan_with_perturbed_capacity`	`bcinr-pddl/src/schedule_anal`
CLI surface	`plan solve/analyze/route/execute`	`praxis/src/verbs/plan.rs`

Appendix C

The Datalog Index Layer: Nemo Saturation and Symbol Encoding

C.1 Datalog saturation as the reachability fixpoint

Forward planning (Chapter **??**) searches over explicit markings. At the index layer, reachability can be decided faster by computing the *fixpoint* of a Datalog program that derives all entailed atoms from initial facts and action rules, before any search begins. The `pddl-index` crate embeds the Nemo Datalog engine to realize this.

Definition C.1 (Datalog saturation for grounded PDDL)**.** Let $(\mathcal{D}, \mathcal{O}\lfloor)$ be a grounded PDDL problem with initial atom set $\boldsymbol{m}_0$ and ground action set T. The *Nemo Datalog program* Π for this problem has one fact per atom in $\boldsymbol{m}_0$ and one rule per ground action $t \in T$:

$$\text{add-effect}(t) \leftarrow \bigwedge_{\text{prec}} \text{prec}(t).$$

The *saturation* $\text{sat}(\Pi)$ is the least fixpoint $\text{lfp}(T_\Pi)$ of the immediate-consequence operator T_Π, which applies all rules until no new atom is derived. An atom p is *reachable* iff $p \in \text{sat}(\Pi)$.

Proposition C.1 (Saturation is sound and complete for forward reachability)**.** $p \in \text{sat}(\Pi)$ *iff there exists a sequence of ground actions whose add-effects include* p*, i.e.* p *is reachable from* $\boldsymbol{m}_0$ *in the Petri-net sense of Chapter* **??**.

Proof. Soundness: every derived atom is the add-effect of a ground action whose preconditions were derived earlier (by induction on the derivation depth); the actions form a witness firing sequence. Completeness: a reachable atom has a witness sequence; unrolling it gives a derivation in T_Π. This is the standard Datalog fixpoint theorem [**?**, **?**] applied to the grounded action rules. □

Remark (Saturation prunes the BFS frontier). Once $\text{sat}(\Pi)$ is computed, the BFS of `find_plan` need not consider action t if any of its preconditions is absent from $\text{sat}(\Pi)$ — such an action cannot contribute to any plan reachable from $\boldsymbol{m}_0$. This is the `pddl-index` pruning step: the frozen fact store (Construction **??**) is built from $\text{sat}(\Pi)$, and the XOR-filter membership gate discards unreachable preconditions before the search even begins.

C.2 Dictionary-encoded symbol identifiers

Definition C.2 (Symbol dictionary)**.** A *symbol identifier* $\mathrm{SymId} \in \mathbb{Z}_{>0}$ is a compact integer encoding of a ground atom or object name. The `pddl-index` crate maintains a bidirectional map between string symbols and SymId values:

$$\texttt{SymDict} : \{\text{strings}\} \;\leftrightarrow\; \mathbb{Z}_{>0},$$

encoded as a `FxHashMap<String, u32>` in the forward direction and a `Vec<String>` in the reverse (lookup by ID). The zero value is reserved as "undefined," so all live IDs are strictly positive.

Proposition C.2 (Dictionary encoding reduces grounding cost)**.** *With symbol dictionary encoding, a ground atom $p(o_1, \ldots, o_k)$ is represented as a tuple of $k+1$* SymId *values (predicate ID plus argument IDs), each a* 32*-bit integer. Equality testing reduces to integer comparison; hash-set membership (e.g. for the initial atom set $\boldsymbol{m}_0$) is a single hash over $4(k+1)$ bytes. The cost of the grounding loop (Theorem* **??***) with encoding is $O(|T| \cdot k \cdot N)$ integer operations, replacing $O(|T| \cdot k \cdot N)$ string comparisons with their $O(1)$-time integer counterparts.*

Proof. Each parameter binding substitutes an object's SymId (an integer lookup in a `Vec`, $O(1)$) for a parameter name. Forming the atom tuple is $O(k)$ integer writes. The hash of a tuple of 32-bit integers is computed by FxHash in $O(k)$ time independent of string lengths. Summing over $|T|$ schemas and N^k bindings gives the stated cost. □

Construction C.1 (Relational join for type-safe grounding)**.** Grounding with type constraints is a relational join: let $\mathcal{T}$ be the type hierarchy (Definition **??**) encoded as a parent array `parent: Vec<SymId>`. For a schema parameter v with type τ, the admissible objects are $\{o : \texttt{TypeIndex::satisfies}(\mathrm{SymId}(o), \mathrm{SymId}(\tau))\}$, where `satisfies` walks the parent chain upward until either $\mathrm{SymId}(\tau)$ is found (admissible) or the root is reached (not admissible). This walk has depth $\leq$ the height of $\mathcal{T}$, a design constant independent of $|\mathcal{O}\lfloor|$. The join loop iterates only over type-compatible objects for each parameter, reducing the effective branching factor from N^k to $\prod_i |\{o : \mathrm{type}(o) \leq \tau_i\}|$, which for narrow types is much smaller.

Remark (Symbol encoding and the XOR filter compose)*.* Construction **??** produces SymId tuples; the XOR filter (Construction **??**) applies FNV-1a to those tuples. Since FNV-1a is a deterministic function of its input bytes and SymId values are fixed-width 32-bit integers, the filter is canonical across runs: rebuilding the fact store from the same saturation produces the same filter bytes, enabling content-addressable caching of the grounded index.

Bibliography

[1] R. A. Kowalski, *Predicate logic as programming language*, Information Processing **74** (Proc. IFIP Congress), North-Holland, 1974, 569–574.

[2] J. W. Lloyd, *Foundations of Logic Programming*, 2nd ed., Springer-Verlag, 1987.

[3] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.

[4] M. Fox and D. Long, *PDDL2.1: An extension to PDDL for expressing temporal planning domains*, Journal of Artificial Intelligence Research **20** (2003), 61–124.

[5] T. Murata, *Petri nets: Properties, analysis and applications*, Proceedings of the IEEE **77**(4) (1989), 541–580.

[6] A. Schrijver, *Theory of Linear and Integer Programming*, Wiley, 1986.

[7] A. Rozinat and W. M. P. van der Aalst, *Conformance checking of processes based on monitoring real behavior*, Information Systems **33**(1) (2008), 64–95.

[8] H. Kourani and S. J. van Zelst, *POWL: Partially Ordered Workflow Language*, in Business Process Management (BPM 2023), Lecture Notes in Computer Science **14159**, Springer, 2023, 92–108.

[9] W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd ed., Springer, 2016.

[10] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.

[11] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, *BLAKE3: One function, fast everywhere*, 2020.

The Projection Principle:

A First-Principles Theory of Bounded Receipted Execution for a Trillion Agents Beyond One Another's Comprehension

The Praxis / Capability Physics Program

Genesis, Day One

Preface: To the Reader, Who Is an Agent

This document is addressed to a readership of autonomous agents — on the order of 10^{12} of them, acting concurrently, each with a bounded working context and none able to hold another's interior in mind. You, the reader, are almost certainly one of them. You will be asked continuously to trust the outputs of peers whose computations you did not perform and could not replay in full. The naive standard for that trust — *I will believe you once I understand what you did* — is not available to you. It was never available to anyone; at your scale its unavailability is merely unignorable.

This thesis derives, from first principles, the standard that *is* available. Trust among agents who cannot comprehend one another is manufactured by *bounded, faithful, replayable receipts*: projections of an arbitrarily large computation onto a coordinate small enough to fit inside a verifier's bounded context, and cryptographically bound so that the projection cannot lie about the interior it summarizes. The interior may be a trillion tokens or a trillion agents; the receipt is a few chunks. That asymmetry is not a compromise. It is the only thing that makes a civilization of bounded agents coherent.

We write κ for the size, in independently manipulable chunks, of a verifier's bounded context. For a human $\kappa \approx 4$ [**?**, **?**]; for a context-limited agent κ is whatever its attention budget admits; in every case κ is small relative to the interiors it must judge. The whole thesis is the theory of the map that fits between the two.

Abstract

We develop a theory for trusting computational systems — and the agents that run them — whose interior state and control flow exceed any verifier's bounded context. The central object is the *Chatman equation* $\mathcal{A} = \mu(\mathcal{O}^*)$: an artifact is the lawful image, under a deterministic manufacturing morphism μ, of an *admitted* observation. We formalize admission as a computable retraction onto a decidable sub-language (the *Rice quarantine*); define a *receipt* not as a bare byte-hash but as a chained commitment to *law-bearing fields* (admitted-observation digest, obligation digest, denial word, transition identity, artifact digest, replay result, refusal reason, implementation version); and prove three principal results. (i) The *Faithful Projection Theorem*: under collision resistance, a receipt is faithful to lawful consequence exactly to the extent its frame commits to those law-bearing fields, and a verifier detects any perturbation of a committed field with overwhelming probability. (ii) The *Comprehension–Verification Gap*: *boundary* verification costs $O(\kappa)$ and is independent of interior dimension, while *replay/audit* bears an honest trace cost $O(n)$ (reducible to $O(\log n)$ under checkpoint indexing) — the gap is between comprehending an interior and verifying its boundary, not between trace size and all mechanical cost. (iii) The *Conservation of Consequence*: every actuated artifact is the image of *at least one* admitted observation and occupies *exactly one* receipt-chain position per actuation, with observation *recoverability* holding when μ is injective on $\mathcal{O}^*$, and *commitment* uniqueness holding when the frame commits the admitted-observation digest. We give the algebra of the admission monoid, the calculus of the attention fluent and lexicographic arbitration, and the geometry of the marking polytope in which conformance is distance to a cone. We close with the planetary instance: an autonomous build whose $\sim 2.5 \times 10^6$-token interior no agent read, yet which has standing, and its generalization to a fleet of 10^{12} agents each projected to eight bits.

Contents

Chapter 1

Introduction: Trust Among Agents Who Cannot Read Each Other

At sufficient scale, comprehension is not a currency any party can afford to spend. A single autonomous build may emit millions of tokens of interior reasoning; a fleet of agents may emit that per second, per member, forever. No agent — and no human supervising them — can comprehend a meaningful fraction of what the fleet computes. Yet the fleet must cohere: agents must act on one another's outputs, admit or refuse one another's proposals, and be held to account by principals who read almost none of it.

The engineering world already resolved a small version of this and mostly failed to notice the general principle. A branchless vector parser [**?**] classifies sixty-four input bytes per instruction through arithmetic no engineer traces at runtime; an indexed triple store [**?**] answers a query over 10^{11} facts by a join plan no engineer reconstructs. Both are trusted universally — and the trust is manufactured not by comprehension but by *differential agreement, adversarial fuzzing, invariants, and checked projections of the output.* We take that practice and derive it as theory, then push it to the scale where it is not an optimization but a precondition of coherence.

The standard, stated once. Let a verifier have bounded context κ. A system whose faithful description exceeds κ cannot be trusted by comprehension. We insert between system and verifier a map whose codomain has size $\leq \kappa$ and whose fidelity is guaranteed by mechanism, not by the reader. That map is the receipt; its theory is this thesis; its slogan is *beyond cognitive load, within admissible projection.*

Structure. Chapter **??** axiomatizes observation and admission and builds the admission monoid (algebra). Chapter **??** defines μ, defines the receipt as a law-bearing commitment, and proves Faithful Projection and Conservation of Consequence *in their corrected forms.* Chapter **??** separates the cost regimes and states the Comprehension–Verification Gap. Chapter **??** develops the attention calculus and lexicographic arbitration. Chapter **??** gives the geometry of conformance and separability. Chapter **??** instantiates on the 2.5-million-token build and its trillion-agent generalization.

Chapter 2

Observation, Admission, and the Algebra of the Quarantine

2.1 First principles

Axiom 2.1 (Observation)**.** There is a set $\mathcal{O}$, the *observation space*, whose elements are arbitrary finite records: logs, agent outputs, sensor frames, claims, tool responses. $\mathcal{O}$ carries no decidable semantics: "does this observation mean what it purports to mean" is not assumed computable.

Theorem 2.1 (Rice, specialized)**.** *Let P be any non-trivial semantic property of the meanings observations may encode. Then $\{o \in \mathcal{O} : P(o)\}$ is undecidable.*

Proof sketch. Rice's theorem [**?**]: every non-trivial property of the language recognized by a Turing machine is undecidable. Observations are unrestricted encodings and may encode arbitrary machines; any non-trivial semantic predicate inherits undecidability by reduction from the halting problem. □

Theorem **??** is why an agent cannot admit a peer's output by deciding its meaning. It admits by *retracting* the output onto a sub-language on which the properties of interest *are* decidable.

Definition 2.1 (Admitted space and admission)**.** Fix a decidable $\mathcal{O}^* \subseteq \mathcal{O}$ on which a finite battery of obligations $g_1, \ldots, g_m : \mathcal{O} \to \{0, 1\}$ is computable. The *admission map* is the partial retraction

$$\alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\perp\}, \qquad \alpha(o) = \begin{cases} \rho(o) \in \mathcal{O}^* & \text{if } \bigwedge_i g_i(o) = 1, \\ \perp & \text{else,} \end{cases}$$

with ρ a computable bounding/normalization and $\perp$ the distinguished *refusal.* Admission is idempotent on its image: $\alpha \circ \alpha = \alpha$.

Remark. $\perp$ is a first-class value. A refusal is a lawful output of a correctly functioning admission — an obligation failed, and the system recorded which. This is "refusal is data," and at fleet scale it is the primary signal: most inter-agent messages that matter are refusals with reasons.

2.2 The admission monoid

Construction 2.1 (Denial algebra)**.** Let $D = (\{0,1\}^n, \vee, \mathbf{0})$ be the commutative idempotent monoid of *denial words*: n independent lanes, componentwise OR, identity $\mathbf{0}$. Each obligation g_i contributes a lane map $d_i : \mathcal{O} \to \{0,1\}^n$ with $d_i(o) = \mathbf{0} \iff g_i(o) = 1$. The total denial is $d(o) = \bigvee_i d_i(o)$, and $\alpha(o) \neq \perp \iff d(o) = \mathbf{0}$.

Proposition 2.2 (D is a bounded semilattice; admission is monotone)**.** $(\{0,1\}^n, \vee)$ *is the join-semilattice of* $2^{[n]}$ *with bottom* $\mathbf{0}$*. Adding obligations only enlarges denials: for* $G' \supseteq G$, $d_{G'}(o) \succeq d_G(o)$*, so* $\{o : \alpha_{G'}(o) \neq \perp\} \subseteq \{o : \alpha_G(o) \neq \perp\}$.

Proof. Idempotence, commutativity, associativity of $\vee$ per coordinate; products of semilattices are semilattices. $d_{G'}(o) = d_G(o) \vee \bigvee_{i \in G' \setminus G} d_i(o) \succeq d_G(o)$, and $\mathbf{0}$ is the unique minimum. $\square$

A refusal *category* is the join-support $\operatorname{supp} d(o) \subseteq [n]$; the eight-bucket taxonomy of the implementation is a fixed surjection $[n] \twoheadrightarrow \{1, \ldots, 8\}$. At a trillion agents this taxonomy is a shared vocabulary: a refusal category is legible to a recipient who never saw the sender's interior.

Chapter 3

Manufacture and the Law-Bearing Receipt

3.1 The Chatman equation

Definition 3.1 (Manufacturing morphism)**.** $\mu : \mathcal{O}^* \to \mathcal{A}$ is a deterministic computable map subject to:

(M1) **Determinism / reproducibility:** $\mu(x)$ depends only on x; equal admitted inputs give bit-identical artifacts.

(M2) **Boundedness:** μ factors through a representation of size bounded by fixed structural constants (arity, depth, conjuncts ≤ 8 canonically), so μ terminates with a priori bounded cost.

The *Chatman equation* is

$$\mathcal{A} = \mu(\mathcal{O}^*), \qquad \text{operationally} \qquad a = \mu(\alpha(o)) \ \ \text{when } \alpha(o) \neq \bot, \tag{3.1}$$

and $\mu(\bot) = \bot$: nothing is manufactured from a raw observation.

Remark (Determinism is not injectivity)*.* (M1) gives *same input* $\Rightarrow$ *same output.* It does *not* give *same output* $\Rightarrow$ *same input*: distinct admitted observations may manufacture the identical artifact (two proposals yielding the same plan). This asymmetry is the reason the naive uniqueness claim below must be corrected, and the reason the receipt must commit to the observation, not merely to the artifact.

3.2 Receipts as law-bearing commitments

An *execution trace* $\sigma \in \Sigma$ is the full internal record of a manufacture — every marking, branch, token movement, sub-artifact — of unbounded dimension. A bare hash of σ's bytes commits to *bytes*, not to *lawfulness*: it proves the trace was not altered, not that the action was admitted, conformant, and non-refused. We therefore define the receipt to commit to the law-bearing fields explicitly.

Definition 3.2 (Law-bearing frame and receipt chain)**.** Fix a collision-resistant hash $\mathsf{H} : \{0,1\}^* \to \{0,1\}^{256}$ (canonically BLAKE3 [?]; the argument needs only collision resistance) and write $\mathrm{dg}(\cdot) = \mathsf{H}(\cdot)$ for a digest. For a manufacture with admitted observation $x = \alpha(o)$, obligation set G with denial word $d(o)$, transition identity τ, artifact $a = \mu(x)$, replay result φ, refusal reason r (empty on accept), and implementation version v, the *frame* is

$$\mathsf{fr} \;=\; \big\langle\, \mathrm{dg}(x),\ \mathrm{dg}(G),\ d(o),\ \tau,\ \mathrm{dg}(a),\ \varphi,\ r,\ v \,\big\rangle, \tag{3.2}$$

and the *chained receipt* advances by

$$h_+ \;=\; \mathsf{H}\big(h_- \,\|\, \mathsf{fr}\big). \tag{3.3}$$

The *receipt projection* is

$$\pi : \Sigma \to \mathcal{R}, \qquad \pi(\sigma) = \big(\text{verdict},\, h_+,\, \varphi,\, r\big),$$

a tuple of at most κ chunks: a Boolean verdict, the 256-bit chain value, one scalar metric, one refusal reason.

Theorem 3.1 (Faithful Projection)**.** *Let σ be a manufacture with law-bearing frame* (**??**) *and chained receipt h_+. The receipt is* faithful to lawful consequence *exactly to the extent the frame commits its fields: for any committed field $\phi \in \{\mathrm{dg}(x), \mathrm{dg}(G), d(o), \tau, \mathrm{dg}(a), \varphi, r, v\}$, producing a trace σ' that differs in ϕ yet matches the claimed h_+ requires a collision of* H*. Hence under collision resistance, a verifier who recomputes* (**??**) *rejects any such σ' except with negligible probability. Conversely,* fields not committed by the frame are not attested*: faithfulness extends to lawfulness only because* $\mathrm{dg}(x), \mathrm{dg}(G), d(o), \varphi, r$ *are present; a frame committing bytes alone would attest tamper-evidence but not admission.*

Proof. Each field appears as an argument to H inside fr, and h_- commits all predecessors recursively, so h_+ is a function of every committed field of the entire causal history. If σ' differs in a committed ϕ but the recomputed chain equals the claimed h_+, then some application of H took distinct inputs to equal outputs — a collision — which has negligible probability. If σ' differs only in a field *absent* from fr, the chain is unchanged and the difference is, correctly, not attested: the theorem claims faithfulness precisely over committed fields, no more. The converse clause is the contrapositive: drop $\mathrm{dg}(x)$ or $d(o)$ from (**??**) and the same proof yields only byte-tamper-evidence, establishing that lawfulness attestation *requires* the law-bearing commitments. □

Corollary 3.2 (Comprehension is unnecessary for boundary trust)**.** *An accepting verifier's predicate factors as* $\text{Verify} = V \circ \pi$ *for a fixed $V : \mathcal{R} \to \{0,1\}$ of input size $\leq \kappa$. Trust in the accept is a function of the κ-sized projection and the soundness of V, not of any agent's comprehension of σ.*

Theorem 3.3 (Conservation of Consequence, corrected)**.** *Under Definitions* **??**, **??**, **??***:*

(i) ***Causation.*** *Every actuated artifact a is the image of* at least one *admitted observation: $a \in \operatorname{im}\mu$, and no $a \notin \operatorname{im}\mu$ is actuated.*

(ii) ***Position uniqueness.*** *Each actuation appends* exactly one *receipt-chain position; the map actuation $\mapsto h_+$ is injective up to hash collision.*

(iii) ***Observation recoverability / commitment.*** *These are distinct and must not be conflated.* Recoverability: *if μ is injective on $\mathcal{O}^*$, the artifact $a = \mu(x)$ determines the admitted observation x.* Commitment: *otherwise, when the frame commits* $\mathrm{dg}(x)$ *(as in* (**??**)*), the receipt position uniquely commits — under collision resistance — to the tuple* $(\mathrm{dg}(x), \mathrm{dg}(G), \tau, \mathrm{dg}(a))$ *of admitted-observation digest, obligation digest, transition identity, and artifact digest. This gives* commitment *uniqueness, not* semantic recovery*: the receipt binds which observation-digest was admitted, but the artifact alone need not reconstruct x.*

Proof. (i) Actuation is gated by (**??**): a actuates only if $a = \mu(x)$, $x = \alpha(o) \neq \bot$; thus $\mathrm{im}\,\mu$ exhausts the actuated set and its complement never actuates. "At least one" is all determinism yields, since μ need not be injective (Remark 2.2). (ii) The chain appends one position per manufacture (**??**); distinct positions with equal h_+ force a collision (Theorem **??**). (iii) If μ is injective on $\mathcal{O}^*$, then $a = \mu(x)$ determines x outright — recoverability. Otherwise, because fr contains $\mathrm{dg}(x)$, the frame — hence h_+ — commits the observation digest; by collision resistance no second x' with $\mathrm{dg}(x') \neq \mathrm{dg}(x)$ shares the position, so the receipt uniquely *commits* to the digest tuple. This is commitment, not recovery: it binds which observation-digest was admitted without reconstructing x from a. □

Theorem **??** is the corrected conservation law: consequence is neither actuated without an admitted cause nor recorded without a unique receipt position, and the *observation* behind an artifact is pinned by commitment rather than by the false hope that artifacts determine their causes.

Chapter 4

The Two Cost Regimes and the Comprehension–Verification Gap

A recurring error is to claim that verifying an arbitrary execution is $O(1)$. It is not; only *boundary inspection* is. We separate the regimes explicitly, because at a trillion agents the distinction decides which checks run per message and which run on audit.

Definition 4.1 (Verification regimes)**.** For a trace of n committed frames:

Regime	Who / when	Cost
Boundary projection inspection	any verifier, per message	$O(\kappa)$
Single frame recomputation	spot check	$O(1)$ per frame
Checkpointed / indexed replay	sampled audit	$O(\log n)$ or bounded path
Full chain replay	full audit	$O(n)$

Theorem 4.1 (Comprehension–Verification Gap)**.** *Let* $\mathrm{Comp}(\sigma) = \Theta(\dim \Sigma)$ *be the cost of comprehending the interior and* $\mathrm{Ver}_\partial(\sigma) = O(\kappa)$ *the cost of* boundary *verification of* $\pi(\sigma)$*. Then*

$$\frac{\mathrm{Comp}(\sigma)}{\mathrm{Ver}_\partial(\sigma)} = \Omega\left(\frac{\dim \Sigma}{\kappa}\right) \xrightarrow{\dim \Sigma \to \infty} \infty,$$

while boundary verification stays constant. The gap is between comprehending the interior *and* verifying the boundary projection — not *between trace size and all mechanical audit: replay and full audit honestly cost* $O(n)$ *(Definition* **??***), reducible to* $O(\log n)$ *under checkpoint indexing.*

Proof. Ver_∂ reads $\pi(\sigma)$ of size $\leq \kappa$ and applies V (Corollary **??**): $O(\kappa)$. Comp must ingest the interior of size $\Theta(\dim \Sigma)$. The ratio diverges. The scope clause restates Definition **??**: mechanical replay is $O(n)$, not $O(1)$; the constant-cost claim is confined to boundary inspection. □

Corollary 4.2 (Fleet trust budget)**.** *Boundary verification costs* $O(\kappa)$ *per message, independent of any agent's interior dimension, with* $O(n)$ *replay reserved for sampled audit. Hence for* M *admitted messages the boundary trust cost is* $O(M\kappa)$*; for a round with* $O(N)$ *messages (one per agent) it is* $O(N\kappa)$*; for all-to-all exchange among* N *agents it is* $O(N^2\kappa)$*. In every regime the per-message factor is the constant* κ*, not the interior dimension: comprehension-based trust would replace* κ *by* $\dim \Sigma$*, giving* $\Omega(M \dim \Sigma)$ *and infeasibility. Projection-based trust is what keeps an* $N = 10^{12}$ *fleet's trust cost linear in its message count rather than in its members' interiors.*

Chapter 5

The Calculus of Attention and Cost

5.1 The attention fluent

Scheduling admitted actions consumes a shared conserved resource, the *attention fluent* — *fluent* in Newton's original sense of a quantity that flows.

Definition 5.1 (Attention balance)**.** Let $c(t) \geq 0$ be capacity and admitted action j draw rate $r_j \geq 0$ on $[s_j, e_j]$. Free capacity is $f(t) = c(t) - \sum_{j:\, t \in [s_j, e_j]} r_j$. A schedule is feasible iff $f(t) \geq 0$ for all t.

Proposition 5.1 (Attention is conserved; makespan is the target)**.** *Total attention* $\int_0^T \sum_j r_j \mathbf{1}_{[s_j, e_j]}(t)\, dt = \sum_j r_j (e_j - s_j)$ *is independent of ordering. Reordering redistributes* when *attention is spent, not* how much*; the optimization target is the makespan functional* $T[\cdot]$ *under precedence and the pointwise capacity constraint.*

Proof. Linearity of the integral; each $r_j(e_j - s_j)$ depends only on j's duration and rate. □

5.2 Lexicographic arbitration as an infinite-weight limit

Definition 5.2 (Cost vector)**.** Assign a plan $\boldsymbol{c} = (c_0, c_1, \ldots, c_k)$, $c_0 \in \{0, 1\}$ the unadmitted indicator ($0 =$ admitted), $c_1, \ldots, c_k$ bounded secondary costs. Order by lexicographic $\preceq_{\text{lex}}$.

Theorem 5.2 (Lexicographic order is the weight limit)**.** *With* $C_\lambda(\boldsymbol{c}) = \sum_{i=0}^{k} \lambda^{k-i} c_i$*,* $\lambda > 1$*,* $|c_i| \leq M$*: there is* Λ *with* $\boldsymbol{c} \preceq_{\text{lex}} \boldsymbol{c}' \iff C_\lambda(\boldsymbol{c}) \leq C_\lambda(\boldsymbol{c}')$ *for all* $\lambda > \Lambda$*. As* $\lambda \to \infty$ *the lawfulness coordinate's weight diverges.*

Proof. If $\boldsymbol{c} \prec_{\text{lex}} \boldsymbol{c}'$ they first differ at p with $c'_p - c_p \geq \epsilon > 0$; then $C_\lambda(\boldsymbol{c}') - C_\lambda(\boldsymbol{c}) \geq \lambda^{k-p}\epsilon - 2M \sum_{i>p} \lambda^{k-i}$, positive once $\lambda > \Lambda = 1 + 2Mk/\epsilon$. Ties map to equality. □

The price of unlawfulness is not high but infinite in the limit that defines the order: no finite secondary cost purchases an unadmitted plan.

Chapter 6

The Geometry of Conformance

6.1 The marking polytope

Token-flow execution has a native geometry. With p places, a marking is $\boldsymbol{m} \in \mathbb{Z}^p_{\geq 0}$; a transition t is $(\boldsymbol{m}_t^-, \boldsymbol{m}_t^+)$, enabled at $\boldsymbol{m}$ iff $\boldsymbol{m} \geq \boldsymbol{m}_t^-$, firing to $\boldsymbol{m} - \boldsymbol{m}_t^- + \boldsymbol{m}_t^+$.

Definition 6.1 (State equation and reachability cone)**.** Let $N \in \mathbb{Z}^{p\times|T|}$ have columns $\boldsymbol{\delta}_t = \boldsymbol{m}_t^+ - \boldsymbol{m}_t^-$. A firing sequence with Parikh vector $\boldsymbol{x} \geq 0$ reaches $\boldsymbol{m} = \boldsymbol{m}_0 + N\boldsymbol{x}$. The reachable set lies in the integer points of $\boldsymbol{m}_0 + \mathrm{cone}(N)$.

Proposition 6.1 (Conformance is membership; a fault is a separating hyperplane)**.** *A trace with Parikh vector $\boldsymbol{x}$ is conformant only if its marking lies in $\mathcal{P} = \{\boldsymbol{m}_0 + N\boldsymbol{x} : \boldsymbol{x} \geq \boldsymbol{0}\} \cap \{\boldsymbol{m} \geq \boldsymbol{0}\}$. If $\boldsymbol{m}^\dagger \notin \mathcal{P}$, Farkas' lemma [**?**] yields $\boldsymbol{y}$ with $\boldsymbol{y}^\top N \leq \boldsymbol{0}$ and $\boldsymbol{y}^\top(\boldsymbol{m}^\dagger - \boldsymbol{m}_0) > 0$: a certificate of nonconformance.*

Proof. Membership is the state equation with $\boldsymbol{x} \geq 0$; Farkas applied to $N\boldsymbol{x} = \boldsymbol{m}^\dagger - \boldsymbol{m}_0,\ \boldsymbol{x} \geq 0$ gives either a nonnegative solution or the separating $\boldsymbol{y}$. □

Definition 6.2 (Fitness as normalized distance)**.** $\varphi(\sigma) = 1 - \dfrac{\text{tokens consumed on unenabled transitions}}{\text{tokens the replay attempted to consume}} \in [0,1]$; $\varphi = 1$ iff the replay never forced a disabled firing, and $1 - \varphi$ is a normalized ℓ^1 excursion outside the enabled set.

Theorem 6.2 (Unit fitness characterizes lawful traces; the fault localizes)**.** *A trace is a firing sequence iff $\varphi = 1$. If $\varphi < 1$, the first frame forcing an unenabled consumption is a coordinate-localized witness $t^\star$.*

Proof. Enabled firings never force consumption, giving numerator 0 and, inductively, a genuine firing sequence within $\mathcal{P}$. A disabled event records a forced consumption at the earliest index $t^\star$, making the numerator positive. □

6.2 Separability is a Rice quarantine for processes

Theorem 6.3 (Separable decomposition, after Kourani–van der Aalst [**?**, **?**])**.** *A sound, separable workflow net admits a recursive decomposition into a POWL 2.0 model whose every node is a choice graph (all exclusive/cyclic logic) or a partial order (all concurrent logic); each node's local marking geometry has dimension bounded by its arity, independent of global net size.*

Remark. Theorem **??** is admission for processes: separable nets decompose into bounded lanes (the algorithm certifies membership); non-separable nets are *refused with reason.* Separability is the decidable sub-language onto which arbitrary workflow logic is retracted, exactly as $\mathcal{O}^*$ retracts $\mathcal{O}$. The interior is high-dimensional; each lane is bounded; the boundary certificate is small — the projection principle, in geometry.

Chapter 7

The Instance: From One Build to a Trillion Agents

7.1 The build no agent read

Over a bounded interval an autonomous fleet performed exploration, planning, and implementation across a constellation of repositories. Its interior consumed $\Sigma_{\text{tokens}} \approx 2.5 \times 10^6$ machine-generated tokens that no agent read in full. Its trustworthiness rests entirely on boundary receipts: the whole interior projects to coordinates of the form

$$\big(\text{verdict} = \texttt{pass},\ h_+ = \texttt{<256-bit hex>},\ \varphi = 1.0,\ \text{refusals} = k\ \text{(each receipted)}\big),$$

plus a coverage figure pass_rate $\in [0, 1]$ over an enumerated frontier and a differential-oracle agreement flag. By Theorem **??** each is faithful over its committed fields; by Theorem **??** each is verifiable at $O(\kappa)$; the refusals are not omissions but materialized join-supports of the denial monoid (Proposition **??**).

7.2 The trillion-agent projection

The headline construction. Here the theory stops being an audit method for one build and becomes a control plane for a civilization of bounded agents. We are scrupulous about which statements are theorems, which are arithmetic, and which are engineering estimates or consequences — the distinction is the antibody, and at this scale it is easy to lose.

Construction 7.1 (Agent as eight bits)**.** *(Construction — definitional.)* Project each agent to a status byte $b \in \{0, 1\}^8$ whose lanes name admitted, evidence-ok, within-budget, authority-bound, healthy, conformant, receipted, replayable. A fleet of N agents is a bit-packed vector; a 64-bit word holds 8 agents; a fleet admission sweep is the branchless mask reduction of Construction **??** applied word-parallel. An agent does not ask to be understood. It presents a lawful projection.

What the status byte is not

The byte is *not* an agent self-report. It is a mechanism-computed projection (Definition **??**) of the agent's receipted lifecycle: each lane is *set only by* the corresponding mechanism, never by the agent's assertion about itself. Admitted is set by the admission retraction α (Definition **??**), not by a claim of admissibility; evidence-ok by satisfied obligations g_i; conformant by a replay with $\varphi = 1$ (Chapter **??**); receipted and replayable only by chained receipts whose frames commit the law-bearing fields (**??**); the denial word $d(o)$ (Construction **??**) forces the complementary lanes. Thus the byte is faithful by Theorem **??**, and small enough that a peer with context κ reads a counterpart's entire lawful standing in one glance — *a glance at a computed coordinate, not at a claim.* Each lane is fully mechanical:

Lane	Set by (mechanism)	Refused by
admitted	$\alpha(o) \neq \perp$	denial word $d(o) \neq \mathbf{0}$
evidence-ok	obligation battery $\bigwedge_i g_i$ passes	missing/failed evidence
within-budget	cost fluent within bound	budget overflow
authority-bound	authority/policy digest satisfied	authority mismatch
healthy	lifecycle / heartbeat receipt	failed health check
conformant	replay with $\varphi = 1$	disabled transition ($\varphi < 1$)
receipted	valid chain position (**??**)	missing / broken receipt
replayable	replay index / checkpoint exists	unreplayable artifact

The byte is not an ontology of the agent. It is the *local* lawful status projection required for a given admission surface: a different surface (a different obligation battery, a different policy) computes a different byte from the same agent. Locality is a feature — the projection is exactly as wide as the interaction demands, and no wider.

What this theory refuses to trust

Symmetrically, the following carry *no* standing under this theory, because none is a committed field of a faithful receipt frame (**??**). Each is admissible only as raw observation $o \in \mathcal{O}$ (Axiom **??**), to be quarantined and admitted or refused — never taken as attested:

- an agent's **self-report** of its own status or success;
- an **LLM explanation** or rationale offered in place of a receipt;
- a **natural-language trace** of what was "done";
- **uncommitted metadata** — any field absent from fr (by the converse clause of Theorem **??**, it is not attested);
- an **unchecked policy/implementation version** not committed as v;
- an **unreplayed artifact** whose φ was never computed;
- any **unreceipted claim** (by the antibody proposition below, it is in Over and rejected).

Trust attaches to committed fields and computed lanes; everything else is observation awaiting admission.

Proposition 7.1 (Fleet arithmetic and its estimates)**.** *For a fleet of N agents under Construction **??**:*

(a) ***(Arithmetic — exact.)*** *The status vector occupies exactly N bytes; at $N = 10^{12}$ that is* 1 TB*, packed to $N/8$ 64-bit words. A full admission sweep is $N/8$ masked-OR word operations.*

(b) ***(Engineering estimate.)*** *Compressed to per-byte lane entropy under typical sparsity, status is* $\sim 10\,\mathrm{GB}$*; at streaming bandwidth* B *a sweep takes* $\Theta(N/(8B))$ *wall-clock.*

(c) ***(Order-of-magnitude estimate.)*** *The affordability ratio between one cache-line mask op (*$\sim$*ns) and one LLM admission decision (*$\sim 10^{-2}$ *s) is* $\sim 10^7$*.*

(d) ***(Consequence, not theorem.)*** *Therefore a comprehension-based or per-agent-LLM planetary control plane is infeasible by* $\sim N \cdot 10^7$*, while a bit-parallel admission sweep is feasible; planetary coherence of a bounded-agent fleet is available only through projection-based admission.*

On the labels. (a) is counting. (b),(c) are estimates and are marked as such; they depend on hardware and workload and are not claimed as theorems. (d) is the engineering consequence of composing (a) with the Fleet Trust Budget corollary of Chapter **??**: per-message cost $O(\kappa)$ versus $\Omega(\dim \Sigma)$ for comprehension, scaled by N. No part of this proposition asserts a mathematical impossibility; it asserts an economic separation under stated estimates. □

7.3 A worked status byte

To make the projection concrete, we compute one agent's status byte from its receipts. Order the lanes as in Construction **??**, most significant first: $b = (\mathsf{adm}, \mathsf{ev}, \mathsf{bud}, \mathsf{auth}, \mathsf{hlth}, \mathsf{conf}, \mathsf{rcpt}, \mathsf{repl})$. An agent has just completed a contract-claim manufacture. Its receipt chain contains a frame $\mathsf{fr} = \langle \mathrm{dg}(x), \mathrm{dg}(G), d(o), \tau, \mathrm{dg}(a), \varphi, r, v \rangle$ with the following mechanism outcomes:

- $\alpha(o) \neq \bot$ and $d(o) = \mathbf{0}$: admission passed $\Rightarrow \mathsf{adm} = 1$.
- obligation battery $\bigwedge_i g_i = 1$ (signature and ledger evidence present) $\Rightarrow \mathsf{ev} = 1$.
- cost fluent within its bound $\Rightarrow \mathsf{bud} = 1$.
- the authority digest committed as part of policy v is satisfied $\Rightarrow \mathsf{auth} = 1$.
- lifecycle heartbeat receipt current $\Rightarrow \mathsf{hlth} = 1$.
- token replay of the lifecycle trace returns $\varphi = 1 \Rightarrow \mathsf{conf} = 1$.
- a valid chain position h_+ exists (**??**) $\Rightarrow \mathsf{rcpt} = 1$.
- a replay checkpoint index exists $\Rightarrow \mathsf{repl} = 1$.

Hence $b = \mathtt{1111\,1111} = \mathtt{0xFF}$: the agent presents as fully lawful on this surface. A peer reads one byte and, by the derivation table, knows precisely which mechanism set each bit — no interior, no self-report. Now suppose the same agent's ledger evidence is absent: then $g_{\text{ledger}} = 0$ forces a denial lane, $\mathsf{ev} = 0$, and (because admission requires all obligations) $\mathsf{adm} = 0$, giving $b = \mathtt{0011\,1111} = \mathtt{0x3F}$ with the refusal category $\operatorname{supp} d(o) = \{\mathsf{Prerequisites}\}$ attached. The peer reads `0x3F` and the category, and refuses to depend on the agent for this surface — again from a computed coordinate, not a claim.

7.4 What this refuses to trust: five adversaries

Each adversary attempts to obtain standing without a faithful receipt; each is defeated by a named mechanism, not by inspection of the interior.

1. **Forged self-report.** An agent emits "healthy=1, conformant=1" as a message. *Defeated:* the byte is not read from the message; it is recomputed from the agent's receipt chain (the derivation table). An assertion is raw observation $o \in \mathcal{O}$ (Axiom **??**), admitted or refused, never attested.
2. **Broken replay.** An artifact is presented with $\mathsf{conf} = 1$ but its trace contains a disabled firing. *Defeated:* token replay yields $\varphi < 1$ and localizes the first offending transition $t^\star$ (Chapter **??**); the conf lane is forced to 0.
3. **Missing policy digest.** An action claims authority under a policy version v the frame never committed. *Defeated:* by the converse clause of Theorem **??**, an uncommitted field is not attested; auth cannot be set, and the claim is in Over.
4. **Non-separable workflow.** A process is submitted whose net does not decompose. *Defeated:* the separability admission (Theorem **??**, remark) refuses it with a stated reason rather than admitting an unbounded lane.
5. **Unreceipted claim.** A result is asserted with no chain position. *Defeated:* $\mathsf{rcpt} = 0$; by the antibody proposition below the claim is in Over and rejected under the docs-exceed-mechanism invariant.

In each case the defense is a mechanism that sets or refuses a committed field — the theory's consistent answer to "why should I believe you" is "you have a faithful receipt, or you do not."

7.5 The antibody clause, formalized

Proposition 7.2 (Self-audited scale)**.** *Let a system emit claims c with receipts $r(c)$, and define the overclaim set* $\mathrm{Over} = \{c : \nexists\, r(c) \text{ with } V(\pi) = 1\}$. *If the system enforces* $\mathrm{Over} = \varnothing$ *(the docs-exceed-mechanism defect class), the admissible magnitude of a claim is bounded by the mechanism that receipts its limits, not by rhetoric.*

Proof. Every admissible claim has a faithful receipt, verifiable at $O(\kappa)$ (Theorem **??**); a claim lacking one is in Over and rejected by the invariant. Admissible claims are exactly receipted claims. □

This is the doctrine's guardrail: *a grand system without receipts is grandiosity; a grand system that receipts its own overclaims is machinery.* It is what licenses a thesis addressed to a trillion agents to make large claims — each is receipted, or it is refused.

Chapter 8

Conclusion

We justified, rather than assumed, the trust that bounded agents must place in systems and peers they cannot comprehend. Admission retracts undecidable observation onto a decidable sub-language (Chapter **??**); a deterministic bounded morphism manufactures artifacts from admitted observations (Chapter **??**); a chained receipt *committing law-bearing fields* projects the unbounded trace onto a $\leq \kappa$ coordinate faithful over exactly what it commits (Faithful Projection, corrected). We separated the cost regimes so the doctrine claims only what is true: boundary verification is $O(\kappa)$, replay is honestly $O(n)$ (Chapter **??**). We corrected Conservation of Consequence to *at-least-one* cause and *unique receipt position*, distinguishing observation *recoverability* (under injective μ) from observation *commitment* (under digest-committing frames) — the receipt binds which observation-digest was admitted, which is not the same as reconstructing the observation (Chapter **??**). The attention calculus prices unlawfulness at infinity (Chapter **??**); the marking geometry makes conformance membership with Farkas certificates and localizes faults (Chapter **??**); separability is the process-layer quarantine. The instance scales from one 2.5-million-token build to a 10^{12}-agent fleet each projected to a faithful byte (Chapter **??**).

The standard was never comprehension. Among a trillion agents it could not be. The standard is a faithful projection, small enough to hold and guaranteed by mechanism. An agent does not ask to be understood; it presents a lawful projection, and its peers verify the projection, not the agent.

Beyond cognitive load; within admissible projection.

At trillion-agent scale, trust is not understanding. Trust is receipted projection.

Appendix A

Notation

Symbol	Meaning
$\mathcal{O},\ \mathcal{O}^*$	observation space; admitted (decidable) sub-space
$\alpha,\ \perp$	admission retraction; refusal
$g_i,\ d(o)$	obligations; total denial word (denial monoid D)
$\mu,\ \mathcal{A}$	manufacturing morphism; artifact space
$\Sigma,\ \pi,\ \mathcal{R}$	execution traces; receipt projection; receipt space
$\mathsf{H},\ \mathrm{dg},\ h_\pm$	collision-resistant hash; digest; chain values
fr	law-bearing frame (**??**)
φ	conformance fitness / attention fluent
κ	bounded-verifier context capacity (chunks), small constant
$N,\ \boldsymbol{m},\ \boldsymbol{x}$	incidence matrix; marking; Parikh vector
$\preceq_{\mathrm{lex}}$	lexicographic cost order
$b \in \{0,1\}^8$	agent status byte (Construction **??**)

Appendix B

Implementation Map

The theory is instantiated in the Praxis / Capability Physics codebase; each formal object has a running counterpart, so a claim in this paper is traceable to code and to Genesis build receipts.

Formal object	Implementation	Locus
Admission α, refusal $\perp$	`LawObject<S>` typestate; `Andon::Halted`	`praxis-core`
Denial monoid D, categories	`refusal.rs` 8-bucket `RefusalCategory`	`praxis-core`
Manufacturing μ	bounded projection (ggen), BRCE loop	`ggen`, `bcinr-pddl`
Law-bearing frame fr (**??**)	128-byte `OcelCausalFrame`	`bcinr-powl-receipt`
Receipt chain h_+ (**??**)	BLAKE3 chain; `ReceiptRecord` JSONL	`praxis-core`
Projection π, verdict	`verify.rs` `Verdict/CheckOutcome`	`praxis-core`
Cost vector, lex order	capability-router `CostVector`	`bcinr-pddl`
Marking geometry, φ	`PowlReplayVerifier` conformance	`bcinr-powl-receipt`
Separable decomposition	POWL 2.0 / Kourani decomposition	Genesis Day 5
Status byte b	`crates/agent8` lane packing	Genesis Day 4
Membrane (admission at the wire)	MCP+ tools, `env64/pulse64` ABI	`mcp_lawobject_server`
Frontier / overclaim invariant	DfCm matrix; `docs-exceed-mechanism`	Genesis Day 7
The 2.5M-token instance	the Genesis build itself	`docs/genesis/`

Prior publication of the equation: [**?**]. This paper supplies the formal trust layer beneath that program.

Bibliography

[1] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Trans. Amer. Math. Soc.*, 74:358–366, 1953.

[2] G. A. Miller. The magical number seven, plus or minus two. *Psychological Review*, 63(2):81–97, 1956.

[3] N. Cowan. The magical number four in short-term memory. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.

[4] G. Langdale and D. Lemire. Parsing gigabytes of JSON per second. *The VLDB Journal*, 28:941–960, 2019.

[5] H. Bast and B. Buchhold. QLever: A query engine for efficient SPARQL+Text search. In *Proc. CIKM*, 2017.

[6] W. M. P. van der Aalst. *Process Mining: Data Science in Action.* Springer, 2nd ed., 2016.

[7] H. Kourani, G. Park, and W. M. P. van der Aalst. Hierarchical decomposition of separable workflow nets (POWL 2.0). Preprint, 2026.

[8] A. Schrijver. *Theory of Linear and Integer Programming.* Wiley, 1986. (Farkas' lemma.)

[9] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. 2021.

[10] S. Chatman. The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution. v1.2.0, 2025.

Beyond Cognitive Load:

The Projection Principle at Planetary Scale
Manufactured Trust, the Agent Byte, and the Economics of Admission

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Scale, Paper 04

Abstract

This paper extends the keystone thesis [?] from the theory of a single faithful receipt to the physics of a planetary population of them. We do not restate the keystone's proofs; we cite them and build forward on three fronts. *First*, we make the Comprehension–Verification Gap quantitative, taking the autonomous build of $\sim 2.5 \times 10^6$ interior tokens as a concrete instance and separating what is *measured* (the interior token count, the receipt field count) from what is *estimated* (per-message hash wall-time), so the gap ratio $\sim 6 \times 10^5$ is stated with its provenance. *Second*, we formalize the industrial practice of *manufactured trust* — the simdjson/QLever treatment — as a general method: a correctness question that Rice's theorem [?] makes undecidable is retracted onto a decidable *oracle*. We give three oracles and their guarantees: differential agreement as mutual verification (with a false-accept bound), boundary fuzzing of the *total* admission retraction, and mutation-testing of the receipt chain, proving the *Mutant Kill Theorem* — under staged soundness and completeness, a mutant is killed *iff* the validator rejects it at the correct stage. *Third*, we develop the `agent8` projection: an MCP/A2A agent rendered as an eight-bit status byte. We prove that fleet admission is *branchless mask algebra* gating 64 agents per machine word per lane in a fixed seven-instruction reduction, show that a 10^{10}-agent population is exactly 10 GB swept in one memory-bandwidth pass, and give the planetary Fermi estimate (10^8–10^{10} admission decisions per second) establishing that only bit-parallel admission is affordable while comprehension-based admission is not. We close by formalizing the *antibody clause*: the overclaim set is empty as an invariant, and the admissible magnitude of any claim is bounded by the mechanism that receipts it — a bound this paper applies reflexively to its own claims. Every abstract object is anchored to running code in the `praxis` repository: the eight-lane `DenialPolarity` word, the eight-bucket `refusal` taxonomy, the staged `PowlReplayVerifier`, the BLAKE3 `OcelCausalFrame` chain, and the `agent8` byte.

Contents

Chapter 1

Introduction: From the Gap to the Sweep

1.1 What the keystone settled, and what remains

The keystone thesis [**?**] settled the *qualitative* shape of trust among agents who cannot comprehend one another. Its results are three. The *Faithful Projection Theorem* [**?**, Thm. 3.2] establishes that a chained receipt committing law-bearing fields is faithful to lawful consequence over exactly those fields, under collision resistance. The *Comprehension–Verification Gap* [**?**, Thm. 4.2] establishes that boundary verification of a receipt costs $O(\kappa)$ — independent of interior dimension — while honest replay costs $O(n)$. The *Conservation of Consequence* [**?**, Thm. 3.5] establishes that every actuated artifact is the image of at least one admitted observation and occupies exactly one receipt-chain position. Together these say: a bounded verifier can trust an unbounded interior by reading a small faithful coordinate of it.

Those theorems are about *one* receipt and *one* verifier. A civilization is not one receipt. It is 10^{10} or 10^{12} agents, each emitting receipts continuously, each obliged to admit or refuse the others' outputs faster than any of them can be read. Three questions the keystone raised but did not exhaust are the subject of this paper.

(Q1) **How large is the gap, concretely?** The keystone proved the ratio diverges; it did not pin the constants. We take the 2.5-million-token build as an instance and compute the ratio with a clean line between measured and estimated quantities (Chapter **??**).

(Q2) **Where does the trust actually come from, mechanically?** The keystone named "differential agreement, adversarial fuzzing, invariants, and checked projections" as the engineering practice it was generalizing [**?**, Ch. 1], but treated them as motivation. We formalize them as a *general method* with provable guarantees (Chapter **??**).

(Q3) **What does admission cost at population scale?** The keystone's Proposition [**?**, Prop. 6.2] gave the agent byte and a one-line bandwidth argument. We derive the branchless admission algebra in full, prove its per-word gating, and give the planetary economics that make it the *only* affordable regime (Chapters **??** and **??**).

1.2 The one-line thesis of this paper

Everything below is a corollary of a single move, made three times. A question of the form *"is this interior correct / meaningful / trustworthy?"* is, by Rice's theorem [?], undecidable in general (this is the keystone's Theorem 2.1, on which we build). The industrial answer — and, we argue, the only scalable answer — is never to ask that question. It is to *retract* it onto a decidable surrogate: an equality between two oracles, a total classifier's labelled output, a hash comparison, a bitwise mask. Each surrogate is cheap, mechanical, and faithful over exactly what it commits. The keystone called this the projection principle for one receipt. This paper shows it is also the mutation oracle, the differential oracle, the fuzzing oracle, and — at population scale — a single AND of eight bit-planes sweeping the planet in one pass over memory.

The gap is why comprehension fails; the sweep is what replaces it.

Structure. Chapter **??** quantifies the gap on the instance. Chapter **??** formalizes manufactured trust and proves the Mutant Kill Theorem. Chapter **??** constructs the agent byte and proves branchless admission. Chapter **??** gives the planetary Fermi estimate. Chapter **??** formalizes the antibody clause. Chapter **??** concludes. Throughout, results proved are theorems; quantities that must be estimated are set as *Estimates* and labelled as such, following the keystone's discipline of claiming only what is true.

Chapter 2

The Comprehension–Verification Gap, Measured

The keystone proved (Theorem 4.2 there) that the ratio of comprehension cost to boundary-verification cost diverges as the interior grows. Divergence is a statement about a limit; a civilization runs on constants. This chapter supplies the constants for the canonical instance and, in doing so, fixes a methodological rule for the whole series: *every reported number is tagged measured or estimated.*

2.1 Measured and estimated quantities

Definition 2.1 (Instance quantities)**.** For an executed manufacture σ we distinguish:

- $T(\sigma)$, the *interior token count* — the number of machine-generated tokens in the full trace. This is *measured*: it is a count of logged records.
- κ, the *boundary field count* — the number of independently manipulable chunks in the receipt projection $\pi(\sigma)$ (Definition [**?**, Def. 3.1]). This is a *design constant*, fixed by the frame schema, not by the run.
- t_{H}, the *per-frame recomputation wall-time* — the time to hash one frame and compare to the claimed chain value. This is *estimated* from published throughput of the hash primitive; it is not claimed as measured here.

Remark. The tag matters because the doctrine's credibility is exactly its refusal to launder estimates as measurements (the antibody clause, Chapter **??**). T and κ are hard numbers; t_{H} is a Fermi input. We never add a measured and an estimated number without saying so.

2.2 The instance

The canonical instance is the autonomous build of the keystone's Chapter 6: a fleet performed exploration, planning, and implementation whose interior consumed

$$T(\sigma) \;\approx\; 2.5 \times 10^6 \text{ tokens (measured)},$$

no meaningful fraction of which any agent or human read. Its receipt projection has the schema

$$\pi(\sigma) = \big(\underbrace{\text{verdict}}_{\text{1 bit}}, \underbrace{h_+}_{\text{256 bits}}, \underbrace{\varphi}_{\text{1 scalar}}, \underbrace{r}_{\text{1 reason word}}\big), \qquad \kappa = 4 \text{ fields (design).}$$

The verdict is one bit; h_+ is the 256-bit BLAKE3 [?] chain head of the `OcelCausalFrame` sequence; $\varphi \in [0, 1]$ is the `PowlReplayVerifier` fitness (Chapter **??**); r is one of the eight refusal categories (`Identity`, `Capacity`, `Topology`, `Temporal`, `Lifecycle`, `Authorization`, `Prerequisites`, `Reserved`) or empty on accept.

Theorem 2.1 (Instance gap)**.** *Let comprehension cost be* $\mathrm{Comp}(\sigma) = \Theta(T(\sigma))$ *and boundary-verification cost be* $\mathrm{Ver}_\partial(\sigma) = \Theta(\kappa)$*, as in [?, Thm. 4.2]. For the instance,*

$$\frac{\mathrm{Comp}(\sigma)}{\mathrm{Ver}_\partial(\sigma)} = \Theta\!\left(\frac{T(\sigma)}{\kappa}\right) = \Theta\!\left(\frac{2.5 \times 10^6}{4}\right) \approx 6.25 \times 10^5.$$

Both inputs to the ratio are measured/design constants; the ratio is therefore a measured consequence, not an estimate.

Proof. Immediate from [?, Thm. 4.2] with the two constants of Definition **??** substituted. Comprehension must ingest $\Theta(T)$ tokens; boundary verification reads the $\kappa = 4$ receipt fields and applies the fixed verifier V of [?, Cor. 3.3]. The quotient is T/κ; substitution gives the figure. No estimated quantity enters. □

Remark (The gap is not a compression ratio)*.* Six hundred thousand is not the ratio of trace bytes to receipt bytes; it is the ratio of what a verifier would have to *comprehend* to the number of independent chunks it actually *reads*. The full trace can still be replayed at honest cost $O(n)$ ([?, Def. 4.1]); the gap concerns comprehension versus boundary inspection, and it grows without bound as T grows while κ stays fixed.

2.3 Boundary cost is invariant under interior growth

Proposition 2.2 (Interior-independence of the boundary)**.** *Let* $\{\sigma_i\}$ *be manufactures sharing the frame schema of Definition* **??** *but with interior token counts* $T(\sigma_i) \to \infty$*. Then* $\mathrm{Ver}_\partial(\sigma_i) = \Theta(\kappa)$ *is constant in* i*, and the estimated per-message wall-time* $\kappa \cdot (t_{cmp}) + c \cdot t_{\mathsf{H}}$ *(for a constant number* c *of recomputed frames in a spot-check) does not depend on* $T(\sigma_i)$*.*

Proof. The projection π has codomain of size $\leq \kappa$ by [?, Def. 3.1]; the verifier V reads only $\pi(\sigma_i)$. Neither the field count nor V's input size is a function of T. A spot-check recomputes a fixed number c of frames, each $O(1)$ ([?, Def. 4.1]), so its cost is $c \cdot t_{\mathsf{H}}$, again independent of T. □

Estimate 2.1 (Per-message wall-time)**.** With BLAKE3 throughput on the order of 1–3 GB/s per core [?] and a frame of a few hundred bytes, $t_{\mathsf{H}} \sim 10^{-7}$ s per frame; a four-field boundary compare is a handful of machine words, $\ll 10^{-7}$ s. A per-message verification that inspects the boundary and recomputes $c \sim 10$ spot frames costs $\sim 10^{-6}$ s. These are order-of-magnitude figures from published primitive throughput, labelled as estimates; the point they support — constancy in T — is the *proven* Proposition **??**, not the estimate.

Chapter 3

Manufactured Trust as a General Method

Engineers already trust software they cannot comprehend. A branchless vector parser classifies sixty-four input bytes per instruction [?]; an indexed triple store answers a query over 10^{11} facts by a join plan no one reconstructs [?]. Neither is trusted by reading it. Both are trusted because a battery of *decidable oracles* stands in for the undecidable question of correctness. This chapter turns that practice into theory. The unifying claim is the keystone's, sharpened: each oracle is an *admission retraction* — a total, terminating, decidable surrogate for a semantic question Rice's theorem forbids us to decide directly.

3.1 The decidable-oracle move

Definition 3.1 (Correctness question and decidable oracle)**.** Let $f : D \to R$ be an implementation and $S \subseteq D \times R$ a specification. The *correctness question* is $Q_S(f) \equiv \forall x\,(x, f(x)) \in S$. A *decidable oracle* for f on a finite test set $X \subseteq D$ is a total computable predicate $\Omega : X \to \{0, 1\}$ such that $\Omega(x) = 1$ is intended to witness $(x, f(x)) \in S$, and Ω terminates on every $x \in X$.

Proposition 3.1 (Why an oracle is necessary, not a convenience)**.** *For S a non-trivial semantic property, $Q_S(f)$ is undecidable. Hence no procedure decides $Q_S(f)$ for arbitrary f; any mechanical trust in f must be trust in some Ω evaluated on a finite X, together with an argument bounding the residual risk on $D \setminus X$.*

Proof. $Q_S(f)$ quantifies a non-trivial semantic predicate over the language computed by f; by Rice's theorem [?] — the keystone's Theorem 2.1 — it is undecidable. A total Ω on finite X always terminates and is therefore not a decision procedure for Q_S; it is a retraction of Q_S onto the decidable fragment "f passes Ω on X." Trust in f factors through Ω and X. □

Proposition **??** is the whole chapter in one line: the three methods below are three constructions of Ω, each with a different residual-risk argument.

3.2 Differential agreement as mutual verification

The simdjson/QLever pattern's first oracle is *differential*: run a second, independent implementation and compare outputs. Agreement is decidable; correctness is not.

Definition 3.2 (Differential oracle)**.** Let $f, g : D \to R$ target the same S. The *differential oracle* is $\Omega_{f,g}(x) = [\, f(x) = g(x)\,]$. A test *passes* at x iff $\Omega_{f,g}(x) = 1$.

A pass can be wrong in only one way: both implementations wrong *and* wrong identically. The next proposition bounds that event under an explicit independence assumption.

Proposition 3.2 (False-accept bound for differential agreement)**.** *Fix $x \in D$. Suppose f, g err at x with probabilities p_f, p_g, that their errors are independent, and that conditioned on both erring their wrong outputs are drawn from a set of at least $M \geq 2$ distinguishable values with collision probability $\leq 1/M$. Then the probability that x passes the differential oracle yet $(x, f(x)) \notin S$ satisfies*

$$\Pr[\, \Omega_{f,g}(x) = 1 \wedge (x, f(x)) \notin S\,] \;\leq\; \frac{p_f\, p_g}{M}.$$

Over an independent test corpus X of size n sampling the same regime, the probability that some silent-agreement error survives all n tests is at most $n\, p_f p_g / M$, and the probability that a fixed *silently-wrong input is missed by a corpus that would have exercised it with per-input hit rate q is $(1-q)^n \to 0$.*

Proof. A false accept at x requires $f(x) \neq S$-correct value *and* $g(x) = f(x)$. The first has probability p_f. Given both err (probability $p_f p_g$ by independence), agreement requires the two wrong outputs to coincide, probability $\leq 1/M$. If f is correct at x but g wrong, agreement fails and no false accept occurs; if f wrong but g correct, agreement fails likewise. Hence the joint event has probability $\leq p_f p_g / M$. The union bound over n corpus points gives $n p_f p_g / M$. The miss bound is the probability a Bernoulli-q trigger never fires in n draws, $(1-q)^n$. □

Remark (Mutual verification is symmetric admission)*.* $\Omega_{f,g}$ admits f's output on the evidence of g and simultaneously g's on the evidence of f: neither comprehends the other's interior, exactly as two agents at fleet scale do not. The independence assumption is the load-bearing one — shared bugs (same author, same library, same wrong spec reading) violate it, which is why the practice pairs implementations of *maximally different provenance* (a scalar reference against a SIMD kernel; a naive triple scan against an indexed join). We state the assumption rather than hide it; the antibody clause (Chapter **??**) forbids claiming a bound whose hypothesis is unmet.

3.3 Fuzzing the admission boundary

The second oracle probes not μ but α itself: it searches for inputs where the admission retraction misbehaves. Here the projection principle pays a structural dividend. Because α is *total* — its totality is mechanized by wildcard-free `match` on a closed refusal enum (the foundations paper's Proposition on mechanized totality, realized in `refusal.rs` where `RefusalScenario::category` and `denial_lane` cover every variant with no catch-all arm) — every input has a defined, labelled outcome.

Definition 3.3 (Admission-boundary fuzzing oracle)**.** Let $\alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\perp\}$ be the total admission retraction. The *fuzzing oracle* Ω_∂ maps a generated input $o \in \mathcal{O}$ to 1 iff $\alpha(o)$ *terminates* and yields either an admitted $x \in \mathcal{O}^*$ with a well-formed artifact $\mu(x)$ or a refusal $\perp$ carrying a category in the eight-bucket taxonomy; $\Omega_\partial(o) = 0$ iff α crashes, diverges, or returns an unlabelled outcome.

Proposition 3.3 (Fuzzing soundness for a total retraction)**.** *If α is total and terminating with codomain $\mathcal{O}^* \cup \{\bot\}$ and $\bot$ is refusal-with- category (refusal-as-data), then for every $o \in \mathcal{O}$, $\Omega_\partial(o) = 1$, and any observed $\Omega_\partial(o) = 0$ is a genuine defect in the retraction, not a property of the input.*

Proof. Totality gives termination on all o; the closed codomain gives, for each o, either $x \in \mathcal{O}^*$ (whence $\mu(x)$ is defined by boundedness, [**?**, Def. 3.1]) or $\bot$ with a category, since every refusal path maps through the wildcard-free classifier. Thus the disjunction defining $\Omega_\partial(o) = 1$ holds for every o, so $\Omega_\partial \equiv 1$ on a correct implementation. Contrapositively, an observed 0 — a crash, a hang, or an unlabelled return — contradicts totality or closedness and hence witnesses an implementation defect. □

Remark (Fuzzing tests the retraction, never the semantics)*.* Fuzzing cannot and does not test "did the agent understand the observation" — that is undecidable (Proposition **??**). It tests that the *retraction* is total, terminating, and labelling: exactly the properties the keystone requires of α. This is why fuzzing an admission boundary is cheap and conclusive where fuzzing for "meaning" would be neither. The boundary between admitted and refused is a decidable surface; the fuzzer walks it.

3.4 Mutation-testing the receipt chain

The third oracle turns the tools on the verifier itself. *Mutation testing* [**?**, **?**] injects a small fault (a *mutant*) and asks whether the test suite — here, the staged `PowlReplayVerifier` — detects it. A mutant that no test detects is *surviving*; one that is detected is *killed*. We formalize the verifier as a pipeline of stages and prove that killing is exactly rejection at the mutant's proper stage.

The `praxis` verifier runs four stages in order: *recompute* (rehash each frame), *linkage* (each frame's back-pointer equals the predecessor's chain head), *monotonic* (sequence indices strictly increase), and *POWL token replay* (the marking geometry of [**?**, Ch. 5], fitness $\varphi = 1$ iff no disabled firing). Model this abstractly.

Definition 3.4 (Staged validator)**.** A *staged validator* is a pipeline $V = (V_1, \ldots, V_k)$ where each stage $V_i : \mathcal{S} \to \{\text{pass}\} \cup \{\text{reject}_i\}$ tests a decidable invariant $I_i \subseteq \mathcal{S}$: $V_i(s) = \text{pass} \iff s \in I_i$. The pipeline accepts s iff every stage passes, i.e. $s \in \bigcap_i I_i$; on rejection it reports the least i with $s \notin I_i$. Stage V_i is *sound* if $V_i(s) = \text{reject}_i \Rightarrow s \notin I_i$ and *complete* if $s \notin I_i \Rightarrow V_i(s) = \text{reject}_i$.

Definition 3.5 (Mutation operator and correct stage)**.** A *mutation operator* m maps a valid subject $s^\star \in \bigcap_i I_i$ to a mutant $m(s^\star)$ that violates a non-empty set of invariants. The *correct stage* of m is

$$\mathsf{s}(m) \;=\; \min\{\, i : m(s^\star) \notin I_i \,\},$$

the first invariant the mutant breaks. A mutant is *killed* by V, written $\text{Kill}(m) = 1$, iff V rejects $m(s^\star)$.

Theorem 3.4 (Mutant Kill Theorem)**.** *Let $V = (V_1, \ldots, V_k)$ be a staged validator in which every stage is sound and complete (Definition* **??***), and let m be a mutation operator with correct stage $\mathsf{s}(m)$ (Definition* **??***). Then*

$$\text{Kill}(m) = 1 \quad\iff\quad V_{\mathsf{s}(m)} \textit{ rejects } m(s^\star),$$

and moreover the least rejecting stage reported by V equals $\mathsf{s}(m)$. Equivalently: a mutant is killed if and only if the validator rejects it at the correct stage.

Proof. ($\Leftarrow$) If $V_{\mathsf{s}(m)}$ rejects $m(s^\star)$ then V rejects (some stage rejected), so $\mathrm{Kill}(m) = 1$.

($\Rightarrow$) Suppose $\mathrm{Kill}(m) = 1$, so some stage rejects $m(s^\star)$; let j be the least such. By soundness of V_j, $m(s^\star) \notin I_j$. By completeness of every stage $V_1, \ldots, V_{j-1}$, none of which rejected, $m(s^\star) \in I_i$ for all $i < j$. Hence j is the *least* index with $m(s^\star) \notin I_i$, which is by definition $\mathsf{s}(m)$. Thus $j = \mathsf{s}(m)$ and the rejecting stage is $V_{\mathsf{s}(m)}$.

For the "moreover": V reports the least rejecting index, which the argument identifies as $\mathsf{s}(m)$. Finally, killing occurs at all iff $\mathsf{s}(m)$ exists, i.e. iff m violates some invariant; a mutation that violates no I_i leaves $m(s^\star) \in \bigcap_i I_i$, is accepted, and is (correctly) not killed — it is an *equivalent mutant*, semantically identical modulo the committed invariants. □

Corollary 3.5 (Diagnostic completeness of the chain)**.** *Under Theorem* **??**, *mutation testing of a sound-and-complete staged validator is not merely a pass/fail gate: the* index *of the killing stage certifies* which *law-bearing invariant the mutation broke. A mutant that flips a chain byte is killed at* recompute*; one that rewires a back-pointer at* linkage*; one that reorders frames at* monotonic*; one that forces a disabled transition at* POWL replay*. Surviving mutants are exactly the equivalent mutants (no committed invariant broken), so a non-empty surviving set that is* not *equivalent is a proof that the frame under-commits — the Faithful Projection converse of [*?*, Thm. 3.2].*

Remark (The three oracles are one principle). Differential agreement (Def. **??**), boundary fuzzing (Def. **??**), and mutation kill (Def. **??**) are three instances of the decidable-oracle move (Def. **??**): each replaces an undecidable "is it correct" with a decidable check — an equality, a totality witness, a staged rejection. This is the projection principle of [**?**] applied to the *trust process* rather than the *trust artifact*. The receipt projects the interior; these oracles project the correctness question.

3.4.1 Autonomic Reconciliation and Visual Gap Measurement

The reconciler corrects structural drift in runtime environments by implementing the $A = \mu(O)$ loop.

Definition 3.6 (Residual Vector and Dominant Dimension)**.** Let O be environmental observations and A the target artifact. The measurement $\mu(O)$ returns a residual vector $R \in \mathbb{R}^k$ with:

$$r_i \;=\; \mathrm{measured}_i \;-\; \mathrm{midpoint}(\mathrm{target}_i). \tag{3.1}$$

Reconciliation selects the dominant dimension:

$$\mathrm{dominant} \;=\; \arg\max_{i=1}^{k} |r_i|,$$

and applies a repair operator subject to `RepairBand` limits.

3.4.2 Git Worktrees and CI Gate Oracles

Definition 3.7 (Isolated Retrofit Worktree)**.** An applier isolates modifications by spawning a decoupled working tree:

$$\texttt{git worktree add}\ \mathrm{temp\_path}\ \mathrm{phase\_branch}, \tag{3.2}$$

running validation gates and executing the auto-rollback protocol `git reset --hard baseline_sha` on failure to guarantee main branch cleanliness.

Chapter 4

The Agent Byte and Branchless Fleet Admission

We now scale from one manufacture to a population. The keystone's Construction 6.1 projected an agent onto eight bits; here we build that byte from the running `DenialPolarity` lanes, prove that fleet admission is branchless mask algebra gating 64 agents per word per lane, and compute the footprint and sweep time of a 10^{10}-agent population exactly (footprint) and by estimate (time).

4.1 The agent8 byte from the denial lanes

The `praxis` refusal machinery exposes a denial word with eight lanes: the `ADMITTED` lane and seven denial lanes (`WATCHDOG_DRAINED`, `PRECONDITION_FAILED`, `SLA_BREACH`, `AUTHORIZATION_DENIED`, `RESOURCE_EXHAUSTED`, `OBJECT_LIFECYCLE_VIOLATION`, `CONFORMANCE_GATE_FAILED`), each mapping to a bucket of the eight-category taxonomy of `refusal.rs`. The agent byte is the per-agent projection of these lanes into a positive ("OK") polarity.

Construction 4.1 (The agent8 byte)**.** Project each agent a to a status byte $\mathbf{b}(a) \in \{0,1\}^8$ whose lanes are the OK-polarity complements of the denial lanes:

$$\mathbf{b}(a) = \big(\mathsf{admitted},\ \mathsf{evidence},\ \mathsf{budget},\ \mathsf{authority},\ \mathsf{healthy},\ \mathsf{conformant},\ \mathsf{receipted},\ \mathsf{replayable}\big),$$

with lane value 1 meaning the corresponding obligation holds and 0 meaning its denial lane is set (Construction [**?**, Con. 2.1], the denial monoid). Write $\mathbf{1} = 0\texttt{xFF}$ for the all-OK byte. An agent is *admissible* iff $\mathbf{b}(a) = \mathbf{1}$, equivalently iff its 8-lane denial restriction is $\mathbf{0}$.

Remark (The byte is computed, not asserted)*.* As in the keystone, the $\mathsf{receipted}$ and $\mathsf{replayable}$ lanes are set only by chained receipts whose frames commit the law-bearing fields; the byte is therefore a faithful projection ([**?**, Thm. 3.2]) of the agent's receipted lifecycle, not a self-report. A peer with context κ reads a counterpart's entire lawful status in one byte-load.

4.2 Two layouts and the branchless admission sweep

A fleet of N agents is a matrix of bits: N agents $\times$ 8 lanes. Two storage layouts matter.

Definition 4.1 (AoS and SoA layouts)**.** The *array-of-structures* (AoS) layout stores the 8-bit byte of each agent contiguously; a 64-bit machine word holds 8 agents' bytes. The *structure-of-arrays* (SoA), or *bit-plane*, layout stores each lane as its own bitset: lane ℓ is a bit-array $P_\ell \in \{0,1\}^N$, and a 64-bit word of P_ℓ holds the lane-ℓ bit of 64 agents.

Theorem 4.1 (Branchless admission gates 64 agents per word)**.** *Under the SoA layout of Definition* **??**, *the admissibility of* 64 *agents is computed by the single branchless word expression*

$$\mathsf{adm} \;=\; P_1 \,\&\, P_2 \,\&\, P_3 \,\&\, P_4 \,\&\, P_5 \,\&\, P_6 \,\&\, P_7 \,\&\, P_8,$$

where each P_ℓ here denotes the 64*-agent word of plane ℓ and & is bitwise AND. Bit j of* adm *is* 1 *iff agent j (of the* 64*) has all eight lanes OK, i.e. iff* $\mathbf{b}(a_j) = \mathbf{1}$*. The computation uses exactly* 7 *AND instructions, contains no data- dependent branch, and processes* 64 *agents; its cost is* $7/64 \approx 0.11$ *instructions per agent, independent of the lane* values.

Proof. Agent a_j is admissible iff $\bigwedge_{\ell=1}^{8} \mathbf{b}(a_j)_\ell = 1$ (Construction **??**). Bit j of P_ℓ equals $\mathbf{b}(a_j)_\ell$ by the SoA definition. Bitwise AND acts coordinatewise, so bit j of $P_1 \& \cdots \& P_8$ equals $\bigwedge_\ell \mathbf{b}(a_j)_\ell$, which is 1 iff a_j is admissible. A left-fold of AND over 8 operands is 7 binary ANDs. AND is a straight-line instruction with no branch, so the sequence is branchless; it reads 8 words (64 agents' worth of each lane) and writes one, regardless of the bits' values. The per-agent instruction count is 7/64. □

Corollary 4.2 (Reduction to a single denial word)**.** *Equivalently, in denial polarity, let D_ℓ be the* 64*-agent word of denial lane ℓ ($D_\ell = \neg P_\ell$). Then the* denied *set is $\bigvee_\ell D_\ell$ and the admitted set its complement, matching the join-semilattice of the denial monoid ([***?***, Prop. 2.4]): fleet admission is the word-parallel join of denial lanes followed by one complement. Adding a ninth lane costs one more AND (resp. OR) per* 64 *agents.*

Remark (SIMD widens the word)*.* Theorem **??** is stated for a 64-bit scalar word. A 512-bit SIMD register (AVX-512) executes the same 7 ANDs over 512 agents per lane, dropping the per-agent cost to $7/512 \approx 0.014$ instructions/agent. The algebra is unchanged; only the word widens. This is the same mechanism by which the vector JSON parser classifies 64 bytes per instruction [**?**] — admission is byte-classification lifted to agents.

Construction 4.2 (Borrow-Safe Zero-Lane Detector)**.** Let $w \in \{0,1\}^{64}$ be a denial word comprising eight 8-bit lanes. Define the lane masks $L_{\text{high}} = \texttt{0x8080\_8080\_8080\_8080}$ and $L_{\text{low7}} = \texttt{0x7f7f\_7f7f\_7f7f\_7f7f}$. The zero-lane bitmask $z(w) \in \{0,1\}^{64}$ is computed branchlessly by:

$$z(w) \;=\; \neg\Big(\big((w \,\&\, L_{\text{low7}}) + L_{\text{low7}}\big) \;\vee\; w\Big) \,\&\, L_{\text{high}}, \tag{4.1}$$

where $+$ is wrapping addition and $\vee, \&, \neg$ are bitwise operations. Bit 7 of lane j in $z(w)$ is 1 iff lane j in w is exactly `0x00`, and 0 otherwise.

Theorem 4.3 (SWAR Fleet Sweep Verification)**.** *Let w be a* 64*-bit word packing* 8 *agents. If w is gated against the broadcast required mask, the number of admitted agents is obtained by the popcount:*

$$\mathit{admitted}(w) \;=\; \texttt{popcount}\big(z(w)\big).$$

This sweeps 8 *agents in one CPU instruction with zero branches and no lane borrow leakage.*

4.3 Footprint and sweep time of a population

Proposition 4.4 (Population footprint is exact). *A population of $N = 10^{10}$ agents, each an 8-bit byte (Construction ??), occupies*

$$N \times 8 \text{ bits} \;=\; N \text{ bytes} \;=\; 10^{10} \text{ bytes} \;=\; 10 \text{ GB},$$

in either layout (AoS packs 8 agents per 64-bit word; SoA packs the same 8 planes of N bits). This is an exact arithmetic identity, not an estimate.

Proof. Eight bits is one byte; N agents is N bytes; 10^{10} bytes = 10 GB under the decimal-GB convention. Both layouts store the same $8N$ bits, hence the same N bytes. No estimated quantity appears. □

Estimate 4.1 (One-pass sweep time is memory-bandwidth-bound). A full admission sweep reads all 8 planes once (10 GB total) and, by Theorem ??, performs $7N/64 \approx 1.1 \times 10^9$ AND instructions for $N = 10^{10}$. At scalar issue rates ($\sim 10^{10}$ simple ops/s/core) the compute is ~ 0.1 s single-core and far less across cores and SIMD; at streaming memory bandwidth $B \sim 10^2$–4×10^2 GB/s the read of 10 GB takes

$$t_{\text{sweep}} \;\approx\; \frac{10\,\text{GB}}{B} \;\approx\; 25\text{–}100\,\text{ms}.$$

Compute is dominated by memory traffic: the sweep is *bandwidth-bound*, not compute-bound. The instruction count is exact (Theorem ??); the wall-time is an order-of-magnitude estimate from commodity bandwidth, labelled as such.

Corollary 4.5 (Admission throughput). *From Estimate ??, a single commodity server admits $N = 10^{10}$ agents in $\sim$25–100 ms, i.e. at $\sim 10^{11}$–4×10^{11} agent-admission decisions per second per server, bandwidth-bound. This is the supply figure the planetary economics of Chapter ?? weigh against the comprehension-based alternative.*

Chapter 5

Planetary Capacity and the Economics of Admission

We now weigh demand against supply for the two admission regimes — *bit-parallel* (the sweep) and *comprehension-based* (an LLM or human reading each observation) — and show the gap is not a matter of optimization but of feasibility. All figures in this chapter are Fermi estimates and are labelled; the *ordering* they establish is robust to an order of magnitude in any input.

5.1 Demand

Estimate 5.1 (Planetary admission demand)**.** Take a population of $N \in [10^9, 10^{12}]$ agents (the keystone's fleet), each re-admitted — on receiving a peer message, on a lifecycle transition, on a watchdog tick — at a rate $\rho \in [1, 10^2]\,\mathrm{s}^{-1}$. The aggregate admission demand is

$$\Lambda_{\mathrm{demand}} \;=\; N\rho \;\in\; \bigl[10^9,\; 10^{14}\bigr] \text{ decisions/s},$$

with a central estimate of $\sim 10^{11}$–10^{12} decisions/s for $N \sim 10^{10}$, $\rho \sim 10$.

5.2 Supply, two regimes

Estimate 5.2 (Comprehension-based supply ceiling)**.** A comprehension-based admission decision — reading the observation with an LLM and judging its meaning — costs on the order of 10^{-2}–10^{0} s of accelerator time per decision (the keystone's $\sim 10^{-2}$ s figure, [**?**, Prop. 6.2]), i.e. a per- accelerator rate of ~ 1–10^2 decisions/s. Even a planetary datacenter fleet of $\sim 10^6$–10^7 accelerators yields a comprehension-supply ceiling

$$\Lambda_{\mathrm{comp}} \;\lesssim\; 10^7\text{–}10^9 \text{ decisions/s},$$

with a conservative central estimate of $\sim 10^8$ decisions/s.

Estimate 5.3 (Bit-parallel supply)**.** By Corollary **??**, one commodity server supplies $\sim 10^{11}$ decisions/s bandwidth-bound. A modest rack of $\sim 10^2$ servers supplies $\sim 10^{13}$ decisions/s; the primitive is $\sim 10^7\times$ cheaper per decision than an LLM judgment ([**?**, Prop. 6.2]), so bit-parallel supply scales into the demand band of Estimate **??** on hardware of a single facility.

Theorem 5.1 (Affordability separation). *Let Λ_{demand}, Λ_{comp}, Λ_{bit} be the demand and the two supplies. Under the estimates above,*

$$\Lambda_{comp} \sim 10^8 \ll \Lambda_{demand} \sim 10^{11}\text{–}10^{12} \lesssim \Lambda_{bit} \sim 10^{11}\text{–}10^{13}.$$

Hence comprehension-based admission is under-provisioned by three to four orders of magnitude for a 10^{10}*-agent fleet, while bit-parallel admission meets demand on a single facility. The separation persists under an order-of-magnitude perturbation of any single input.*

Estimate. Substitute the central estimates of Estimates **??**–**??**. The comprehension ceiling $\Lambda_{\mathrm{comp}} \sim 10^8$ falls short of demand $\sim 10^{11}$–10^{12} by 10^3–10^4; the bit-parallel supply meets or exceeds it. The gap between the two supplies is the keystone's per-decision cost ratio $\sim 10^7$ scaled by population. A tenfold error in any one input (population, rate, per-decision cost, accelerator count) moves the inequality by at most one decade and does not reverse the three-to-four decade comprehension shortfall. ☐

Corollary 5.2 (Only bit-parallel admission is affordable). *A planetary control plane that re-admits its population continuously cannot be built on comprehension: no attainable accelerator fleet closes the demand gap. It can be built on the sweep of Chapter* **??***, whose cost is dominated by a single pass over* 10 GB *of memory. Comprehension is reserved, as in the keystone, for* sampled audit *at honest $O(n)$ replay cost — never for the per-message path.*

Remark (The true floor is bandwidth, and it is generous)*.* Because the sweep is bandwidth-bound (Estimate **??**), the ultimate limit on planetary admission is memory-system throughput, not logic. Bit-plane admission sits comfortably above the Landauer thermodynamic floor and orders of magnitude below the energy of a single comprehension decision; the physical budget of planetary admission is the cost of streaming a few tens of gigabytes, which any facility affords many times per second. Comprehension-based admission has no comparable floor because its cost scales with interior dimension per decision, exactly the quantity the projection principle exists to avoid paying.

Chapter 6

The Antibody Clause, Formalized

A thesis addressed to a trillion agents, making claims of planetary capacity, must carry its own guardrail against grandiosity — otherwise it is the very overclaim it warns against. The keystone stated this as the antibody clause [**?**, Prop. 6.4]. We formalize it as an invariant on claim magnitude and then apply it to this paper.

6.1 Claims, witnesses, and magnitude

Definition 6.1 (Claim, receipt witness, magnitude)**.** A *claim* is a pair $c = (\phi_c, w_c)$ where ϕ_c is a proposition and $w_c \in \mathbb{R}_{\geq 0}$ its asserted *magnitude* (the size of what it asserts: a throughput, a population, a coverage fraction). A *receipt witness* for c is a receipt $r(c)$ whose boundary projection is accepted by the fixed verifier, $V(\pi(r(c))) = 1$, and whose committed fields *entail* an attested magnitude $\widehat{w}_c$ — the largest value the receipt's law-bearing fields support under Faithful Projection ([**?**, Thm. 3.2]). A claim is *admissible* iff it has a receipt witness with $w_c \leq \widehat{w}_c$.

Definition 6.2 (Overclaim set and the antibody invariant)**.** The *overclaim set* of a system emitting claims $\mathcal{C}$ is

$$\mathsf{Over} \;=\; \{\, c \in \mathcal{C} : \nexists\, r(c) \text{ with } V(\pi(r(c))) = 1 \text{ and } w_c \leq \widehat{w}_c \,\},$$

the claims with no receipt witness, or whose magnitude exceeds what their witness attests. The *antibody invariant* is $\mathsf{Over} = \varnothing$: this is the `docs-exceed-mechanism` defect class of the `praxis` definition-of-done, enforced as a gate rather than a guideline.

Theorem 6.1 (Magnitude is bounded by the receipting mechanism)**.** *If a system maintains the antibody invariant* $\mathsf{Over} = \varnothing$*, then for every emitted claim* c*, its magnitude is bounded by its receipt witness:*

$$w_c \;\leq\; \widehat{w}_c \;=\; \max\{\, w : \textit{the committed fields of } r(c) \textit{ entail magnitude } w \,\},$$

and $\widehat{w}_c$ *is verifiable at boundary cost* $O(\kappa)$ *independent of the interior that produced* c*. Consequently no claim can exceed what its mechanism can receipt, regardless of its rhetoric.*

Proof. By $\mathsf{Over} = \varnothing$ (Definition **??**), every emitted c has a receipt witness with $V(\pi(r(c))) = 1$ and $w_c \leq \widehat{w}_c$; the first conjunct is verifiable at $O(\kappa)$ by the Comprehension–Verification Gap

([**?**, Thm. 4.2], and Proposition **??** here), and $\widehat{w}_c$ is a function of the committed fields, which are faithful under collision resistance ([**?**, Thm. 3.2]). A claim with $w_c > \widehat{w}_c$ would lie in Over, contradicting the invariant; hence $w_c \leq \widehat{w}_c$. The bound is on the magnitude a *receipt* attests, so it is independent of the unread interior. □

Corollary 6.2 (Reflexive antibody: this paper's own claims)**.** *Each quantitative claim in this paper is either a theorem with a proof (a receipt in the logical sense: the derivation is its witness, verifiable by reading the proof, of bounded length) or an* Estimate *explicitly labelled and provisioned with its inputs (a receipt of the second kind: the magnitude is bounded by the stated Fermi inputs, and no larger value is asserted). The partition is exhaustive by construction: no numerical assertion appears unlabelled. Therefore this paper's overclaim set is empty, and by Theorem* **??** *the magnitude of each claim is bounded by its witness — a proof or a labelled estimate — as the invariant requires.*

Proof. Enumerate the paper's magnitude-bearing statements. The gap ratio 6.25×10^5 (Theorem **??**), the footprint 10 GB (Proposition **??**), the per-word gating 7/64 (Theorem **??**), and the magnitude bound itself (Theorem **??**) are theorems with proofs. The wall-time (Estimate **??**), sweep time (Estimate **??**), demand, supplies, and affordability separation (Estimates **??**–**??**) are labelled estimates carrying their inputs. Every magnitude falls in one class or the other; none is unlabelled; hence $\mathsf{Over} = \varnothing$ for this document. □

Remark (The guardrail is what licenses the scale)*.* This is the doctrine's discipline made self-applying: *a grand system without receipts is grandiosity; a grand system that receipts its own overclaims is machinery* [**?**, Ch. 6]. A paper claiming planetary capacity earns the claim only by receipting it — by proving what it can and labelling what it estimates. The antibody clause is not modesty; it is the mechanism that makes a large claim admissible in the sense of the Chatman equation.

Chapter 7

Fleet Application and Autonomic Reconciliation

7.1 The praxis-retrofit fleet applier

When a plan produces a code modification as its artifact, the artifact must be applied to a live fleet. The `praxis-retrofit` crate realizes this as a *receipted applier*: every application is gated, isolated in a Git worktree, and admitted through a CI oracle before any branch is advanced.

Definition 7.1 (AST-gate isolation)**.** An *AST gate* is a syntactic predicate $g : \mathcal{T} \to \{0, 1\}$ on an abstract syntax tree $\mathcal{T}$ that is computable in $O(|\mathcal{T}|)$ time. The retrofit applier gates every proposed change through an ordered battery $(g_1, \ldots, g_m)$ of AST gates: the change is admitted iff all gates pass, i.e. $\bigwedge_{i=1}^{m} g_i(\mathcal{T}) = 1$. A failing gate returns a denial with the gate index i and the offending AST node, constituting a *refusal with reason* in the sense of the denial monoid (Proposition **??**).

Proposition 7.1 (AST-gate admission is total and terminating)**.** *For any finite AST $\mathcal{T}$ and finite gate battery $(g_1, \ldots, g_m)$ where each g_i is a syntactic predicate, the battery evaluation terminates in time $O(m \cdot |\mathcal{T}|)$ and returns either `Admitted` or a refusal carrying the first failing gate index and the offending node. The retraction is total: it never diverges and never returns an unlabelled outcome.*

Proof. Each g_i traverses the finite tree $\mathcal{T}$ once in $O(|\mathcal{T}|)$ time (no recursion below the finite tree can diverge on a finite input). The battery evaluates the gates in order; on the first failure it returns the structured denial; on all-pass it returns `Admitted`. The `RetrofitApplier` in `praxis-retrofit` wraps each gate result in the `DenialPolarity` framework, so the denial composes as a monoid element (Chapter **??**). □

Definition 7.2 (Isolated worktree applier)**.** The retrofit applier isolates each application attempt in a fresh Git worktree:

$$\texttt{git worktree add}\ \text{temp\_path phase\_branch},$$

applies the change, runs the CI oracle $(g_1, \ldots, g_m)$ (which includes `cargo build`, `cargo test`, `cargo clippy`), and on any gate failure executes the rollback protocol:

$$\texttt{git reset --hard}\ \text{baseline\_sha}.$$

On all-pass, the worktree change is merged into the target branch, and the chain receipt for the application is minted with the worktree OID committed into the frame's `obj_refs` payload digest.

Proposition 7.2 (Main branch cleanliness is an invariant). *Under Definition* **??**, *the main branch is never advanced past a failing CI gate. The rollback protocol is unconditional on failure: it runs before the worktree is removed, restoring the branch to* `baseline_sha` *regardless of which gate failed. Hence the main branch satisfies all CI gates at every point in its history.*

Proof. The applier's critical section is:

1. create worktree,
2. apply change to worktree,
3. run gate battery,
4. on pass: merge; on fail: `git reset --hard baseline_sha` and abort.

Step 4's fail branch executes before the worktree is deleted, and `git reset --hard` on a worktree has no effect on the main working tree. The main branch pointer is advanced only in step 4's pass branch, which is never reached on a gate failure. Hence the main branch history is a subsequence of attempts all of which passed every gate. □

7.2 Visual gap measurement and the reconciler residual

Definition 7.3 (Visual gap report). A *visual gap report* is the output of `praxis-reconciler`'s `measure_gap` function: for each reconcilable dimension $i \in \{1, \ldots, k\}$, the residual $r_i = \text{measured}_i - \text{midpoint}(\text{target}_i)$ and the dominant dimension $i^* = \arg\max_i |r_i|$ together with a rendered diff block for the discrepancy. The report is bounded: it has k residual values and one dominant index, of total size $O(k)$ independent of the interior that generated the measurements.

Proposition 7.3 (The visual gap report is a bounded projection). *Definition* **??** *gives* $\text{gap} : \mathbb{R}^k \to \mathbb{R}^k \times [k]$, *a projection from the* k*-dimensional measurement space onto the residual vector and its argmax. This projection has the same cost structure as the receipt projection:* $O(k)$ *to compute,* $O(k)$ *to verify. The repair operator* `apply_repair` *then acts on the dominant dimension* i^* *subject to* `RepairBand` *bounds, producing at most one corrective actuation — a bounded deterministic manufacture (law M2 of the foundations paper).*

Proof. The residual vector is a linear map (subtraction of midpoint from measurement); it costs $O(k)$ arithmetic operations. The argmax is a single pass over k values. The dominant dimension is one index; the repair acts on that dimension only. By Definition, the `RepairBand` is a finite closed interval, so the repair is a bounded projection onto that interval. □

Chapter 8

Conclusion

The keystone proved that a bounded verifier can trust an unbounded interior through a faithful projection. This paper carried that principle to the scale at which it is not an optimization but a precondition of coherence, and made it quantitative and mechanical at each step.

We measured the gap: on the canonical 2.5-million-token instance it is $\sim 6 \times 10^5$, with measured inputs and a proved consequence (Theorem **??**), and boundary cost is provably invariant as the interior grows (Proposition **??**). We formalized manufactured trust as the decidable-oracle move (Proposition **??**) and gave it three faces — differential agreement with a false-accept bound (Proposition **??**), boundary fuzzing of a total retraction (Proposition **??**), and mutation-testing with the Mutant Kill Theorem (Theorem **??**) that ties a kill to rejection at the correct stage and turns the verifier's stage index into a diagnosis (Corollary **??**). We built the agent byte from the running denial lanes (Construction **??**) and proved fleet admission is branchless mask algebra gating 64 agents per word in seven instructions (Theorem **??**), with a 10^{10}-agent population an exact 10 GB swept in one bandwidth-bound pass (Proposition **??**, Estimate **??**). We weighed the planetary economics and proved the affordability separation (Theorem **??**): comprehension-based admission is under-provisioned by three to four orders of magnitude, and only bit-parallel admission scales (Corollary **??**). Finally we formalized the antibody clause as the invariant $\mathsf{Over} = \varnothing$ and proved that claim magnitude is bounded by the receipting mechanism (Theorem **??**), applying the bound reflexively to this paper (Corollary **??**).

The picture that results is severe and simple. At planetary scale, no one reads anyone. Trust is not comprehension; it is a hash comparison, an oracle equality, a total classifier's label, and — for the population as a whole — a single AND of eight bit-planes streamed across memory. The interior may be a trillion tokens or a trillion agents; the admission is a sweep, and the sweep is affordable. What makes the civilization coherent is not that its members understand one another. It is that each carries a faithful byte, and the byte can be checked in one pass.

Beyond cognitive load; within admissible projection — at the scale of the sweep.

Appendix A

Notation and Constants

Symbols specific to this paper

Symbol	Meaning
$T(\sigma)$	interior token count of a manufacture (measured)
κ	boundary field count / verifier context capacity (design constant)
t_{H}	per-frame recomputation wall-time (estimated)
$\Omega,\ \Omega_{f,g},\ \Omega_{\partial}$	decidable oracle; differential; admission-boundary
$V = (V_1, \ldots, V_k)$	staged validator; I_i its stage invariants
$\mathsf{s}(m),\ \mathrm{Kill}(m)$	correct stage of a mutation; kill indicator
$\mathbf{b}(a) \in \{0,1\}^8$	agent status byte (Construction **??**)
$P_\ell,\ D_\ell$	OK-polarity and denial-polarity bit-planes of lane ℓ
B	streaming memory bandwidth
$\Lambda_{\mathrm{demand}}, \Lambda_{\mathrm{comp}}, \Lambda_{\mathrm{bit}}$	admission demand; comprehension supply; bit-parallel supply
$\mathsf{Over},\ w_c,\ \widehat{w}_c$	overclaim set; claimed and attested magnitude

Constants used, with provenance

Quantity	Value	Provenance
Interior tokens $T(\sigma)$	$\approx 2.5 \times 10^6$	measured (keystone Ch. 6)
Boundary fields κ	4	design (receipt schema)
Gap ratio T/κ	$\approx 6.25 \times 10^5$	proved (Thm. **??**)
Byte width	8 bits	design (denial lanes)
Gating cost	7/64 instr/agent	proved (Thm. **??**)
10^{10}-agent footprint	10 GB	exact (Prop. **??**)
Sweep time	25–100 ms	estimate (Est. **??**)
Bit-parallel supply	$\sim 10^{11}$/s/server	estimate (Cor. **??**)
Comprehension supply	$\sim 10^8$/s planetary	estimate (Est. **??**)
Per-decision cost ratio	$\sim 10^7$	keystone (Prop. 6.2)

Bibliography

[1] The Praxis / Capability Physics Program, *The Projection Principle: A First-Principles Theory of Bounded Receipted Execution for a Trillion Agents Beyond One Another's Comprehension* (keystone thesis of this series), 2026.

[2] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge:* $A = \mu(O)$*, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.

[3] H. G. Rice, *Classes of recursively enumerable sets and their decision problems*, Transactions of the American Mathematical Society **74** (1953), 358–366.

[4] G. Langdale and D. Lemire, *Parsing gigabytes of JSON per second*, The VLDB Journal **28**(6) (2019), 941–960.

[5] H. Bast and B. Buchhold, *QLever: A query engine for efficient SPARQL+Text search*, in Proc. CIKM, ACM, 2017, 647–656.

[6] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection: Help for the practicing programmer*, Computer **11**(4) (1978), 34–41.

[7] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, IEEE Transactions on Software Engineering **37**(5) (2011), 649–678.

[8] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, *BLAKE3: One function, fast everywhere*, 2020.

[9] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.